Plenara

Technische Dokumentation · Stand der Kapitel 17. Oktober 2026 · gedruckt am 17. Oktober 2026

1. Technische Dokumentation Plenara (Ratsmonitor)
2. Glossar: verbindliche Begriffe
3. Abruf: Berichte aus den Ratsinformationssystemen holen
4. Pipeline: Tagespipeline, Extraktion, Verarbeitung, Nacharbeit, Altbestand
5. Suche: Suchkarten, Wortliste, Volltextindex
6. Auswertungen: plenara.X, Vorberechnung, Social Media
7. Datenmodell: Speicher, Tabellen, Trigger, Stände
8. Zugang, Konten, Oberfläche, KI-Weg, Tests

Quelle: `docs/technik/*.md` im Repository, erzeugt mit `node docs/technik/tools/build-pdf.mjs`. Die Markdown-Fassung ist maßgeblich; Querverweise zwischen den Kapiteln sind in dieser PDF Sprungmarken.

Inhalt

Technische Dokumentation Plenara (Ratsmonitor)

Kapitel

1. Was Plenara ist
2. Gesamtbild
3. Datenfluss auf einer Seite
4. Aufbau des Codes
5. Laufzeiten, Build, Deploy
6. Infrastruktur auf dem Server
7. Pflege dieser Dokumentation

Glossar: verbindliche Begriffe

1. Gegenstände der Daten
2. Abläufe und Betrieb
3. System, Konten, Oberfläche
4. Mehrdeutige Wörter und wie wir sie verwenden
5. Kennungen

Abruf: Berichte aus den Ratsinformationssystemen holen

1. Gebiet, Quelle, Leser, Server
2. Ein nächtlicher Abruf von Anfang bis Ende
3. Die Leser
4. Was der Abruf schreibt
5. Identität, Versionen, Marken
6. Schutz der Quellen und des eigenen Servers
7. Lokal und auf dem Server
8. Befunde beim Schreiben dieser Dokumentation (17.10.2026)

Pipeline: Tagespipeline, Extraktion, Verarbeitung, Nacharbeit, Altbestand

1. Überblick
2. Jobs und Zeitplaner
3. Die Tagespipeline Schritt für Schritt
4. Extraktion im Detail: die PDF-Ablage
5. Verarbeitung im Detail: Regeln je Bericht
6. Fehlerspeicher
7. Nacharbeit (`scripts/nacharbeit.mjs`)
8. OCR, tagsüber (`ocr`, `nightly.mjs --ocr-only --range daily`)
9. Altbestand (`scripts/altbestand.mjs`, Seite `/admin?seite=altbestand`)
10. Aufträge der Ablage (`pdf-job`, `scripts/pdf-store/job.mjs`)
11. Debug: Wege der Berichte, Nachtbericht
12. Befunde beim Schreiben dieser Dokumentation (17.10.2026)

Suche: Suchkarten, Wortliste, Volltextindex

1. Die Suchkarte (`search_cards`)
2. Die Indizes
3. Eine Anfrage an `GET /api/search`
4. Einordnung: welches Label die Suche zeigt
5. Was die Suche nicht tut
6. Bekannte Eigenheiten (Stand 17.10.2026)

Auswertungen: plenara.X, Vorberechnung, Social Media

1. Ein Weg für alles
2. Der Bau der Zahlen (`scripts/plenara-x/build-x-cases.mjs`, `server/integrations/x-cases.mjs`)
3. Die Abfrage `GET /api/x` (`app/api/x/route.ts`, `plenara-x-query.mjs`)
4. Signale (`/api/x/signale`, `plenara-x-signals.mjs`, `/analytics/signale`)
5. Die Tools (`/api/analytics/*`)
6. Startseite: Karte und Trends (`scripts/startseite-trends.mjs`)
7. Vorberechnung der Administration (`region-facts.mjs`, `region-series.mjs`, `admin-builds.mjs`)
8. Social Media (`social-topics.mjs`, Admin „Social Media“)
9. Befunde beim Schreiben dieser Dokumentation (17.10.2026)

Datenmodell: Speicher, Tabellen, Trigger, Stände

1. Die drei Speicher
2. Tabellen der Hauptdatenbank
3. Trigger-Kette und Datenstände
4. Schlüssel in `system_state`
5. Das Payload eines Berichts (`topics.payload`)
6. Kontendatei (`drizzle-accounts/`)
7. PDF-Ablage (`index.sqlite`)
8. Migrationen
9. Befunde beim Schreiben dieser Dokumentation (17.10.2026)

Zugang, Konten, Oberfläche, KI-Weg, Tests

1. HTTP-Schicht: wer darf was
2. Konten und Anmeldung (`server/accounts/`)
3. Oberfläche
4. KI-Weg (`ai-local-content-v2`)
5. Tests, Lint, Typen
6. Befunde beim Schreiben dieser Dokumentation (17.10.2026)

Technische Dokumentation Plenara (Ratsmonitor)

Stand: 17. Oktober 2026, gelesen am Code des Branchs `nachtlauf-fixes`. Die Dokumentation beschreibt, was der Code tut; wo Konzepte, Kommentare oder Betriebsdokumente etwas anderes sagen, steht das in den Abschnitten „Befunde“ der Kapitel. Alle Diagramme sind Mermaid-Blöcke und werden auf GitHub, in VS Code und in den meisten Markdown-Ansichten gezeichnet.

Kapitel

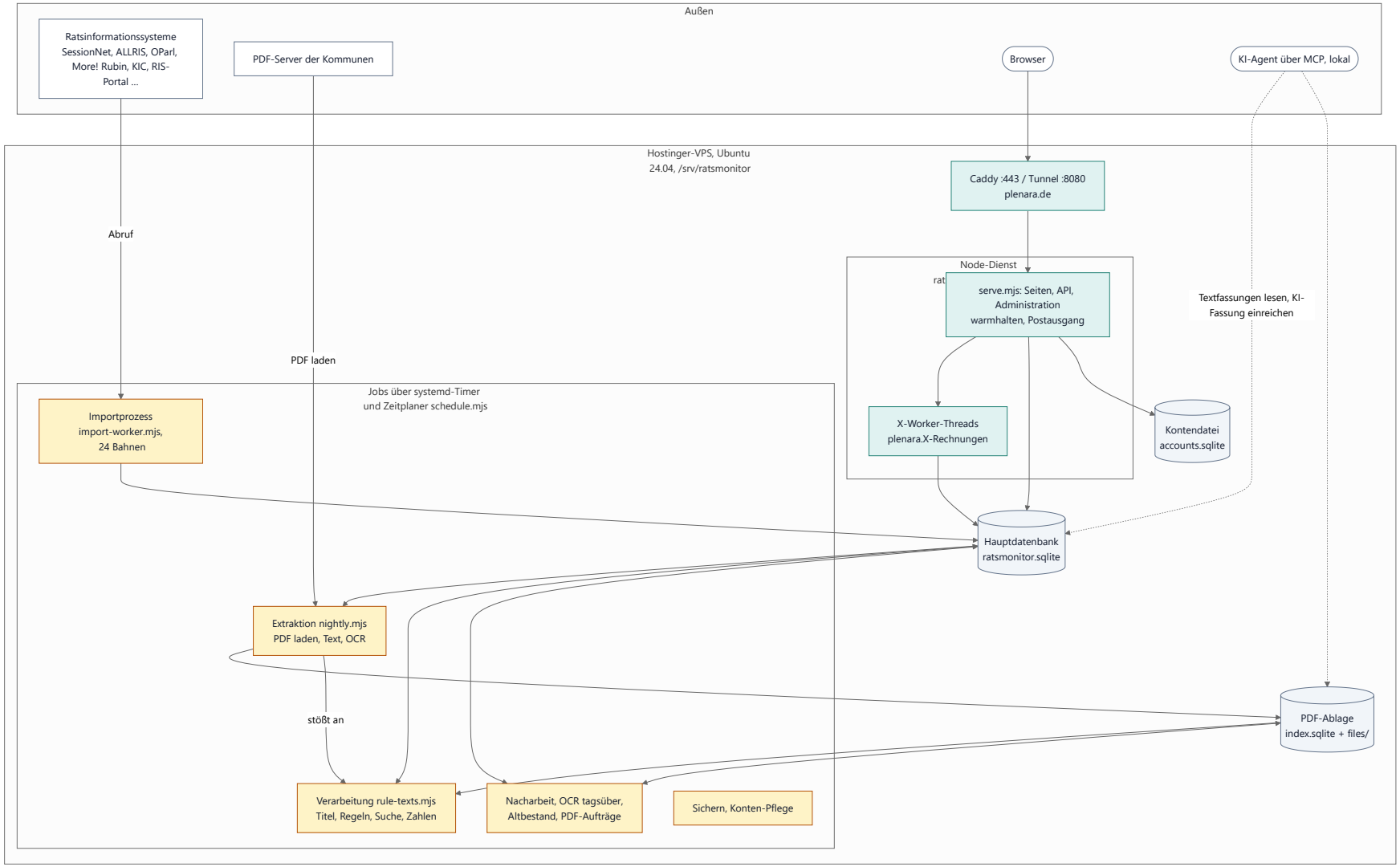
Kapitel	Frage
Glossar	Welches Wort meint was? Bericht, Suchkarte, Vorgang, Gebiet, Quelle, Stufe … mit Code- und Tabellennamen und den mehrdeutigen Wörtern
Abruf	Wie kommen Berichte aus den Ratsinformationssystemen in die Datenbank? Quellen, Leser, Abrufauftrag, Identität, Schutz
Pipeline	Was passiert jede Nacht und tagsüber? Tagespipeline, Extraktion (PDF-Ablage), Verarbeitung (Regeln), Nacharbeit, Altbestand, Fehlerspeicher, Zeitplaner
Suche	Wie findet die Suche? Suchkarten, Wortliste, Volltextindex, Entscheidungsbaum, Labels
Auswertungen	Woher kommen die Zahlen von plenara.X? `x_cases`, `x_signals`, `/api/x`, Tools, Vorberechnung der Administration
Datenmodell	Welche Tabellen gibt es, wer schreibt, wer liest, welche Trigger und Stände? Payload, `system_state`, Kontendatei, PDF-Ablage
Zugang und Oberfläche	Wer darf was? Konten, Stufen, Admin, API-Routen, Seiten, Admin-Reiter, KI-Weg, Tests

Betrieb und Einrichtung (Befehle, Migrationen, Umstellungen): `docs/betrieb/node-server.md`. Hinweise für die Arbeit am Code: `CLAUDE.md`. Frontend-Karte: `components/ratsmonitor/FRONTEND.md`. Produktvorteile und Slogan: `docs/produkt/`.

1. Was Plenara ist

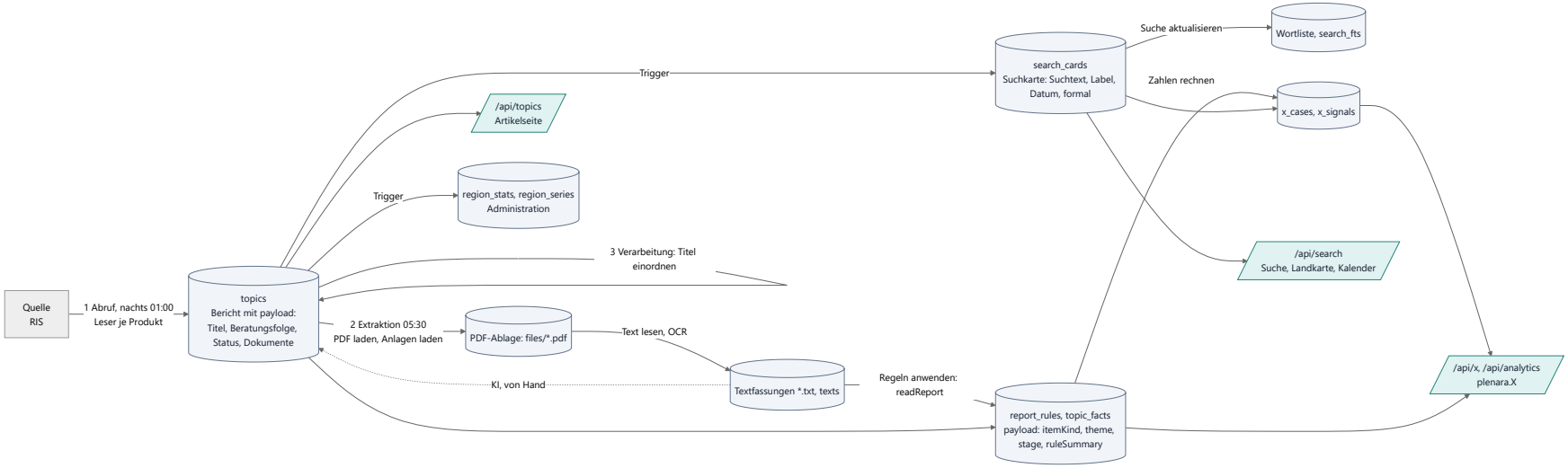
Plenara liest öffentliche Ratsinformationssysteme deutscher Kommunen (Gemeinden, Städte, Kreise, Gemeindeverbände: 5.324 Gebiete im Katalog, 4.041 mit angebundener Quelle), speichert je Vorlage oder Tagesordnungspunkt einen **Bericht** mit Beratungsfolge, Status und Dokumenten, lädt die PDFs, liest ihren Text, leitet mit festen Regeln Art des Punkts, Thema, Zusammenfassung und Werte ab und bietet darauf eine Suche mit Landkarte, Artikelseiten, Kalender und den Analyse-Teil **plenara.X**. Ein KI-Agent kann Berichte zusätzlich zusammenfassen, startet aber nie von selbst.

2. Gesamtbild


Außen
Ratsinformationssysteme SessionNet, ALLRIS, OParl, More! Rubin, KIC, RIS-Portal ...
PDF-Server der Kommunen
Browser
KI-Agent über MCP, lokal
Hostinger-VPS, Ubuntu 24.04, /srv/ratsmonitor
Caddy :443 / Tunnel :8080 plenara.de
Abruf
Node-Dienst
rat
serve.mjs: Seiten, API, Administration warmhalten, Postausgang
Textfassungen lesen, KI-Fassung einreichen
PDF laden
Jobs über systemd-Timer und Zeitplaner schedule.mjs
Importprozess import-worker.mjs, 24 Bahnen
X-Worker-Threads plenara.X-Rechnungen
Kontendatei accounts.sqlite
Hauptdatenbank ratsmonitor.sqlite
Extraktion nightly.mjs PDF laden, Text, OCR
stößt an
PDF-Ablage index.sqlite + files/
Verarbeitung rule-texts.mjs Titel, Regeln, Suche, Zahlen
Nacharbeit, OCR tagsüber, Altbestand, PDF-Aufträge
Sichern, Konten-Pflege

Ein Node-Prozess liefert Oberfläche, API und Administration (`dist/standalone/serve.mjs`, hinter Caddy). Daneben laufen als eigene Prozesse der **Importprozess** und die **Jobs**, alle auf derselben SQLite-Datei. Die Kontendatei sehen nur Webdienst und Kontojob. Der KI-Agent läuft nicht auf dem Server, sondern lokal über STDIO gegen eine Kopie oder die Datei.

3. Datenfluss auf einer Seite


Quelle
RIS
1 Abruf, nachts 01:00
Leser je Produkt
topics
Bericht mit payload:
Titel, Beratungsfolge,
Status, Dokumente
Trigger
/api/topics
Artikelseite
Trigger
region_stats, region_series
Administration
3 Verarbeitung: Titel
einordnen
2 Extraktion 05:30
PDF laden, Anlagen laden
PDF-Ablage: files/*.pdf
Text lesen, OCR
KI, von Hand
Textfassungen *.txt, texts
Regeln anwenden:
readReport
search_cards
Suchkarte: Suchtext, Label,
Datum, formal
Suche aktualisieren
Wortliste, search_fts
Zahlen rechnen
x_cases, x_signals
/api/search
Suche, Landkarte, Kalender
report_rules, topic_facts
payload: itemKind, theme,
stage, ruleSummary
/api/x, /api/analytics
plenara.X

In Worten: Der Abruf schreibt je Bericht eine Zeile in `topics`; Trigger leiten daraus die Suchkarte ab. Die Extraktion holt die PDFs der neuen Berichte und liest ihren Text. Die Verarbeitung ordnet Titel ein, wendet die Regeln je Bericht an (Art des Punkts, Thema, Regel-Zusammenfassung, Fakten), führt die Wortliste nach und baut die Vorgänge für plenara.X. Suche und Landkarte lesen nur Suchkarten, die Artikelseite den Bericht, plenara.X die Vorgänge. Alles, was scheitert, kommt in den Fehlerspeicher und wird tagsüber nachgearbeitet (Pipeline).

4. Aufbau des Codes

Ordner	Inhalt
`app/`	Next-App (vinext): `(monitor)/` Seiten der Oberfläche (Stubs, gerendert vom Client), `api/` alle Routen (78 `route.ts`), `admin/` die Administration, CSS-Dateien
`components/ratsmonitor/`	die eigentliche Oberfläche: `App.tsx`, `components/`, `pages/` (auch `analytics/`), `info/`, `lib/`, `state/`, `menu/`; Karte `FRONTEND.md`, Index `INDEX.md`
`components/admin-*.tsx`	Reiter und Bausteine der Administration; `components/ui/` shadcn
`server/services/`	Kernlogik: `sync.ts` (Abruf je Gebiet), `session.ts` (Zugang), `admin-http.ts`, `calendar-ics.ts`, `push.ts`, `read-cache.mjs`
`server/integrations/`	fachliche Module (ca. 150 Dateien): Leser je Ratsinformationsprodukt (`sessionnet.mjs`, `oparl*.mjs`, `allris*.mjs` …, Registry `readers.mjs`, Auswahl `collect-region.mjs`), Quellenkataloge (`*-sources.json`), Abrufauftrag (`pipeline-jobs.mjs`, `pipeline-worker.mjs`), Suche (`monitor-search.mjs`, `search-words.mjs`), Regeln (`report-rules.mjs`, `rule-summary.mjs`), plenara.X (`plenara-x-*.mjs`, `x-cases.mjs`, `analytics-*.mjs`), Administration (`admin-*.mjs`, `region-facts.mjs`), Betrieb (`schedules.mjs`, `work-failures.mjs`, `pdf-store-admin.mjs`, `altbestand-admin.mjs`), KI (`ai-local-work.mjs`, `ai-jobs.mjs`)
`server/node/`	Node-Laufzeit: `serve.mjs` (Einstieg), `d1-sqlite.mjs` (D1-Adapter), `cloudflare-workers.mjs` (Ersatzmodul), `import-worker.mjs`, `x-worker.mjs`, `pdf-store.mjs`, `mcp-runtime.mjs`, `ai-local-*.mjs`
`server/accounts/`	Konten: Anmeldung, Anmeldesitzungen, Zugang, Mail, TOTP
`server/repositories/`, `server/mappers/`	Lesezugriffe auf `topics`, Gebiete, Analysen; `seed.ts` Erstbefüllung
`server/mcp/`	MCP-Dienst (intern und Kunde)
`shared/`	Code für Server und Browser: Typen (`types.ts`), Gebietskatalog (`catalog.mjs`, `regions.ts`, `*-regions.json`, `*-population.json`), Regeln (`item-kind.mjs`, `themes.mjs`, `labels.mjs`, `stage.mjs`, `topic-facts.mjs`, `document-type.mjs`, `pdf-rank.mjs`), Begriffe (`pipeline-terms.mjs`), Stufen (`plans.mjs`), Qualität (`rule-quality.json`), Identität und Versionen (`topic-identity.mjs`, `article-record.mjs`, `article-version.mjs`)
`drizzle/`, `drizzle-accounts/`	Migrationen der Hauptdatenbank (0000 bis 0045) und der Kontendatei (0001, 0002); rohes SQL
`scripts/`	Skripte: Jobs (`schedule.mjs`, `rule-texts.mjs`, `nacharbeit.mjs`, `altbestand.mjs`, `startseite-trends.mjs`), PDF-Ablage (`pdf-store/`), plenara.X (`plenara-x/`), Konten (`accounts/`), Bau (`build-node.mjs`, `run-framework.mjs`, `node-migrate.mjs`), Katalogbau (`build-de.mjs`, `build-nds.mjs`, `build-nrw.py`, `source-discovery/`), Messungen (`*-study/`), KI (`ai-local.mjs`, `mcp-*.mjs`, `ai-batch/`), Wartung (`refresh-search-words.mjs`, `refresh-admin.mjs`, `reclassify-rules.mjs`)
`deploy/node/`	Server: `deploy.sh`, `ratsmonitor.service`, je Job `.service`/`.timer`/`.path`, Stopp-Einheiten, `Caddyfile` und `caddy/*.caddy` (Stufen), `backup.sh`, `accounts-*.sh`, `*.env.example`
`build/`	Vite-Plugins für Dev (`todo-dev-plugin.mjs`, `ai-local-dev-plugin.mjs`), `worker-build.mjs` (bündelt Importprozess und X-Worker)
`tests/`	175 Tests (`node --test tests/*.test.mjs`), Hilfen `tests/helpers/`, Fixtures
`data/`, `public/`	mitgelieferte Startdaten (NRW-Bestand, Münster, Billerbeck); statische Dateien, Geodaten, Schriften, `startseite-trends.json`, `sw.js`
`docs/`, `requirements/`	Betrieb, Produkt, Qualität, Konzepte und Pläne (Konzepte beschreiben teils ältere Stände)
`config/mcp/`	Startvorlagen des MCP-Diensts

5. Laufzeiten, Build, Deploy

Lokal (`npm run dev`)

`scripts/run-framework.mjs dev` startet vinext (Vite) mit dem Cloudflare-Plugin (Miniflare, D1-Bindung `DB` auf die Datei unter `.wrangler/state/v3/d1/`) auf Port 5173 und daneben: Wortliste nachführen (`RM_SKIP_SEARCH_WORDS=1` schaltet ab), Admin-Werte nachführen (`RM_SKIP_ADMIN_REFRESH=1`), `scripts/dev-jobs.mjs` (prüft alle 15 s, ob ein Job angefordert ist; geplante Zeiten laufen lokal nicht; `RM_SKIP_DEV_JOBS=1`), Konten einrichten (`scripts/accounts/dev-setup.mjs`, `RM_SKIP_ACCOUNTS=1`). Die Administration ist im Dev ohne Anmeldung offen. Für plenara.X mit häufigen Begriffen lokal besser `node scripts/node-local.mjs` (baut und startet den Node-Server auf der lokalen Datei, Port 3000).

Server (Node)

`npm run build:node` (`scripts/build-node.mjs`): `vinext build` mit `RM_TARGET=node` (ohne Cloudflare-Plugin, `output: standalone`, `cloudflare:workers` zeigt auf `server/node/cloudflare-workers.mjs`), kopiert `serve.mjs` nach `dist/standalone/`,

bündelt mit Vite `dist/standalone/worker/import-worker.mjs` und `worker/x-worker.mjs`, prüft, dass kein `cloudflare:workers` im Server-Paket und kein `node:sqlite` im Browser-Paket steckt. Der MCP-Dienst wird nicht gebaut, er läuft aus dem Quelltext.

Teil	Datei	Aufgabe
Einstieg	`server/node/serve.mjs`	Startprüfung (Datenbank mit `topics`, `search_cards`, `system_state`, `d1_migrations`; `RM_ACCESS`; mit `invite` Kontendatei, `ACCOUNTS_SECRET`, `SITE_URL`, `VINEXT_TRUST_PROXY`), Port 3000 nur Loopback, Node ≥ 22.16, X-Worker (`RM_X_WORKER`, `RM_X_WORKERS` 2), Warmhalten alle 10 min, Postausgang jede Minute, sauberes Beenden (20 s)
D1-Adapter	`server/node/d1-sqlite.mjs`	`prepare`, `bind` (ganze Zahlen als INTEGER, Booleans 1/0), `first`, `all`, `run`, `raw`, `batch` (eine Transaktion, `BEGIN IMMEDIATE`), `exec`; vor jeder Abfrage `setImmediate`; Langsam-Log ab `DATABASE_SLOW_MS`; WAL, `synchronous=NORMAL`
Ersatzmodul	`server/node/cloudflare-workers.mjs`	`env` als Proxy: `DB` (`DATABASE_FILE`), `ACCOUNTS` (`ACCOUNTS_FILE`), `AI_LOCAL_SOURCES` (`RM_PDF_STORE`), alles andere aus `process.env`

Deploy (`deploy/node/deploy.sh main`, nur `main` wird ausgerollt): `git archive` des Commits per SSH nach `/srv/ratsmonitor/app/releases/<sha>.build` → Werte aus `ratsmonitor.env` und `accounts.env` → `pnpm install --frozen-lockfile` → `build-node.mjs` → Migrationen der Hauptdatenbank nur prüfen (`node-migrate.mjs --check`; fehlen welche, bricht das Skript ab, eingespielt wird von Hand mit angehaltenem Importprozess) → Kontendatei sichern (age) und migrieren → Symlink `current` umstellen → `systemctl restart ratsmonitor` (laufende Importprozesse behalten ihren Stand) → Prüfung (`/`, `/api/search?level=city`, `/api/internal/health`) → `current` und die zwei neuesten anderen Stände bleiben. systemd-Einheiten installiert das Skript nicht; das steht in `docs/betrieb/node-server.md`.

Cloudflare

`npm run build` baut weiter mit dem Cloudflare-Plugin (`wrangler.local.json` nur lokal); der Code hält Kompatibilitätsstellen vor (`requestTier` ohne Kontendatei, `cf-connecting-ip`, `ADMIN_LEGACY_HEADER`). Produktiv läuft nur Node; eine Produktions-Konfiguration für Cloudflare gibt es im Repo nicht.

6. Infrastruktur auf dem Server

```
Browser ─▶ Caddy (plenara.de :443 mit Let's Encrypt, HSTS, noindex; Tunnel 127.0.0.1:8080) ─▶ Node 127.0.0.1:3000 ─▶ SQLite
```

Pfad	Inhalt
`/srv/ratsmonitor/app/releases/<sha>`, `app/current`	Programmstände, Symlink auf den laufenden
`/srv/ratsmonitor/ratsmonitor.env`	Umgebung (600), Vorlage `deploy/node/ratsmonitor.env.example`
`/etc/ratsmonitor/accounts.env`	`ACCOUNTS_FILE`, `ACCOUNTS_SECRET`, `MAIL_API_KEY`, `ACCOUNTS_AGE_RECIPIENT` (root, 600); nur Webdienst und Kontojob laden sie
`/srv/ratsmonitor/data/ratsmonitor.sqlite`	Hauptdatenbank
`/srv/ratsmonitor/data/pdf-store/`	PDF-Ablage (`RM_PDF_STORE`)
`/srv/ratsmonitor/data/trigger/<job>`, `stop-<job>`	Auslöser-Dateien der `.path`-Einheiten
`/srv/ratsmonitor/data/ocr-cache/`, `/srv/ratsmonitor/ocr-tools/`	OCR-Sprachdaten, tesseract.js und `@napi-rs/canvas` außerhalb des Programmstands
`/srv/ratsmonitor/backups/`	Sicherungen der Datenbank (zstd, 2 Stände), Index und Texte der Ablage, Protokolle
`/srv/ratsmonitor/accounts/`	Kontendatei, verschlüsselte Sicherungen (7 Stände), `deleted.log`; Rechte 700, für alle anderen Einheiten `InaccessiblePaths`
`/etc/caddy/Caddyfile`, `/etc/default/caddy`	Caddy (Fassungen in `deploy/node/caddy/`: Schritt 0 Tunnel mit alter Kennung und sslip mit Basic-Auth, Schritt 2 ohne Kennung, Schritt 3 Domain ohne Basic-Auth); die aktive Fassung mit `plenara.de` liegt nur auf dem Server

Umgebungsvariablen (`ratsmonitor.env.example`): `DATABASE_FILE`, `DATABASE_READONLY`, `DATABASE_BUSY_TIMEOUT_MS`, `DATABASE_SLOW_MS`, `HOST`, `PORT`, `NODE_ENV`, `TZ=UTC`, `VINEXT_TRUST_PROXY`, `NODE_OPTIONS`, `SITE_URL`, `RM_ACCESS` (`open`/`invite`), `ADMIN_HOSTS`, `ADMIN_LEGACY_HEADER`, `MAIL_TRANSPORT` (`http`/`log`/`outbox`), `LOGIN_TOTP`, `MAIL_API_URL`, `MAIL_FROM`, `MAIL_REPLY_TO`, `OPERATOR_EMAIL`, `ADMIN_SETUP_HASH` (alt), `IMPORT_TOKEN`, `OPENAI_API_KEY`/`OPENAI_MODEL` (optional, Modus `summaries`), `VAPID_*`, `RM_PDF_STORE`, `OCR_TOOLS`, `OCR_CACHE`, `RM_X_WORKERS`. Nicht in der Vorlage: `SHUTDOWN_GRACE_MS`, `RM_ALLOW_PUBLIC_BIND`, `RM_ALLOW_NO_DB`, `RM_SKIP_WARM`, `RM_X_WORKER`, `IMPORT_BUSY_TIMEOUT_MS` (Einheiten 30 s), `ACCOUNTS_BUSY_TIMEOUT_MS`, `RM_TRIGGER_DIR`, `ACCOUNTS_OPEN_SIGNUP`, `ACCOUNTS_IN_MAIN_DB`, `BACKUP_REMOTE`, `BACKUP_DIR`, `KEEP`, `ZSTD_LEVEL`, `RM_MCP_CONFIG`, `ROBOTS_POLICY`.

systemd (46 Dateien in `deploy/node/`): `ratsmonitor.service` (Webdienst, `User=ratsmonitor`, `Restart=always`, `MemoryHigh=6G`, `MemoryMax=8G`, `ProtectSystem=strict`) und je Job vier Teile: `ratsmonitor-<job>.service` (`node scripts/schedule.mjs <job>`, oneshot, eigene Speicher- und Zeitgrenzen), `.timer` (`OnCalendar=*:0/10`; `import` mit `OnUnitInactiveSec=10min`), `.path` (`PathModified` auf `data/trigger/<job>`, Sofortstart), `ratsmonitor-stop-<job>.path` → `ratsmonitor-stop@<job>.service` (als root `systemctl stop`). Jobs: `accounts`, `backup`, `import-nightly`, `startseite-trends`, `pdf-nightly`, `rule-texts`, `nacharbeit`, `ocr`, `altbestand`, `import`, `pdf-job` (Zeiten, Fenster, Grenzen in Pipeline, Abschnitt 2). Journal: 7 Tage (`journald-ratsmonitor.conf`).

7. Pflege dieser Dokumentation

- **PDF:** `node docs/technik/tools/build-pdf.mjs` baut alle Kapitel als `docs/technik/technische-dokumentation.pdf` (Markdown zu HTML, Diagramme über mermaid.js, Druck über Chrome mit playwright-core aus `docs/design/tools`; `--html` schreibt nur die HTML-Vorlage nach `tmp/technik-pdf/`). Nach Änderungen an den Kapiteln neu bauen.
- Begriffe der Pipeline nur in `shared/pipeline-terms.mjs` ändern, dann im Glossar nachziehen.
- Neue Tabelle oder neuer `system_state`-Schlüssel: Katalog im Datenmodell ergänzen.
- Neuer Job: `JOBS` in `schedules.mjs`, Einheiten in `deploy/node/`, Tabelle in Pipeline Abschnitt 2.
- Neuer Leser: `readers.mjs`, Tabelle in Abruf Abschnitt 3.
- Die „Befunde"-Abschnitte sind Stand 17.10.2026; was behoben ist, dort streichen.

Glossar: verbindliche Begriffe

Stand: 17. Oktober 2026. Teil der technischen Dokumentation.

Dieses Glossar legt für jeden Gegenstand **ein deutsches Wort** fest und nennt daneben den Namen im Code und in der Datenbank. Die Dokumentation benutzt nur die fett gesetzten Wörter. Wo der Code oder die Oberfläche ein anderes Wort benutzt, steht es in der Spalte „auch genannt", damit man es beim Lesen von Code, Admin-Seiten und älteren Konzepten wiedererkennt.

Die Begriffe der Server-Pipeline (Abläufe, Stufen, Schritte, Gründe) stammen aus `shared/pipeline-terms.mjs`; dort werden sie gepflegt, hier nur erklärt.

1. Gegenstände der Daten

Vom Groben zum Feinen: Gebiet → Quelle → Sitzung → Punkt → Bericht → Suchkarte → Vorgang.

Begriff	Im Code und in der Datenbank	Auch genannt	Bedeutung
Gebiet	`region`, `region_id`, `Region` in `shared/regions.ts`, Katalog `CATALOG` in `shared/catalog.mjs` (aus `nrw-regions.json`, `nds-regions.json`, `de-regions.json`)	Ort, Kommune, Gemeinde (Oberfläche)	Eine Gemeinde, Stadt, ein Kreis oder ein Gemeindeverband (Samtgemeinde, Amt, Verbandsgemeinde, Verwaltungsgemeinschaft) aus dem Katalog (5.324 Gebiete). Kennung: `nrw-<Schlüssel>`, `nds-<Schlüssel>`, `de-<Schlüssel>`, dazu sieben benannte Kennungen (`billerbeck`, `muenster`, `borken`, `coesfeld`, `recklinghausen`, `steinfurt`, `warendorf`). Arten `kind`: `city` (Gemeindeebene) oder `district` (Kreis). Der amtliche Schlüssel steht in `ags` (8 Stellen Gemeinde, 5 Kreis, 9 Gemeindeverband).
Mitgliedsgemeinde	`members` am Gebiet, `where()` in `server/integrations/member-place.mjs`, `shared/vg-groups.mjs`	–	Gemeinde innerhalb eines Gemeindeverbands. Der Verband ist das Gebiet mit der Quelle; die Mitgliedsgemeinde wird aus Gremium („Ortsgemeinderat Limbach") oder Titel erschlossen.
Quelle	`source` (`source.id`, `source.url`, `source.reader`), Kataloge `server/integrations/*-sources.json`, `source-catalog.mjs`, `source-overrides.json`	Ratsinformationssystem, RIS, Anbindung	Das Ratsinformationssystem eines Gebiets: Adresse plus die Angabe, welcher **Leser** es liest. Ein Gebiet hat höchstens eine Quelle; Kreise und Städte sind getrennte Gebiete mit eigener Quelle. Die Spalte `topics.source` enthält nicht die Quelle, sondern die Ebene (`city`/`district`).
Leser	`readers.mjs` (Registry), ein Modul je Produkt (`sessionnet.mjs`, `oparl.mjs`, `allris.mjs` …)	Adapter, Reader	Code, der ein bestimmtes Ratsinformationsprodukt liest und daraus `ImportData` (Berichte und Quellenstand) macht.
Server (einer Quelle)	`host`, `source_hosts`, `source-servers.mjs`, `body-identity.mjs`	Host	Der Rechner hinter einer oder mehreren Quellen (ein Anbieter betreibt oft viele Gebiete). Trägt die **Serverstufe** des Fehlerspeichers. Nicht zu verwechseln mit dem eigenen **Node-Server**.
Sitzung	`meeting` in den Lesern; im Bericht `events[].date` und `events[].committee`	Termin	Zusammenkunft eines Gremiums an einem Tag. Es gibt keine eigene Tabelle; Sitzungen leben in den Beratungsfolgen der Berichte und in den Marken des Abrufs (`import-marks:<gebiet>`). Nicht verwechseln mit der **Anmeldesitzung** eines Kontos.
Gremium	`committee`, `search_cards.gremium`	Ausschuss, Rat	Rat, Ausschuss, Ortsrat, Verbandsversammlung.
Punkt (Tagesordnungspunkt)	`agendaItem`, Kennungsteil `top`, Art in `payload.itemKind` (`shared/item-kind.mjs`)	TOP	Ein Punkt der Tagesordnung einer Sitzung. Seine **Art des Punkts** ist `subject` (**Sachpunkt**), `formal` (**Formalie**: Eröffnung, Niederschrift, Tagesordnung), `collection` (**Sammelpunkt**: Mitteilungen, Anfragen) oder `routine` (**Routine**: Wahlen, Besetzungen, Einwohnerfragen). Nur ein Sachpunkt bekommt ein Thema.
Vorlage	`paper`, Kennungsteil `vo`, `reference` (Drucksachennummer), `paperUrl` in `shared/topic-identity.mjs`	Drucksache, Beschlussvorlage, Antrag	Das Schriftstück, über das beraten wird. Eine Vorlage wandert durch mehrere Sitzungen (Beratungsfolge).
Bericht	`topic`, Tabelle `topics` (`id`, `region_id`, `source`, `event_date`, `updated_at`, `status`, `payload`, generiert `active`), Typ `StoredTopic` in `server/types.ts`	Topic; Artikel/`article` (`article_versions`, `article_analyses`, `saved_articles`, `ArticleCard`, Artikelseite); „Vorgang" in der Oberfläche der Suche; Adresse `/beschluss/<id>`	**Die Einheit des Bestands.** Eine Vorlage mit ihrer Beratungsfolge, oder, wenn die Quelle keine Vorlagen-Kennung hergibt, ein einzelner Punkt einer Sitzung. Kennung je Leser: bei SessionNet, ALLRIS, More! Rubin, KIC u. a. `<quelle>-vo-<nr>` (Vorlage) oder `<quelle>-top-<sitzung>-<nr>` (Punkt), bei OParl `<quelle>-oparl-<hash>`. Alles, was die Pipeline tut, tut sie je Bericht.

Begriff	Im Code und in der Datenbank	Auch genannt	Bedeutung
Beratungsfolge	`events[]` (`TopicEvent`: `date`, `committee`, `status`, `result`, `decision`, `attendance`)	Stationen, Beratungsweg	Die Sitzungen, in denen ein Bericht behandelt wurde oder wird, mit Ergebnis je Station.
Status	`status` (`TopicStatus` in `shared/types.ts`: `announced`, `consulting`, `recommended`, `approved`, `rejected`, `postponed`, `info`, `unknown`)	Verfahrensstand	Stand des Berichts als Ganzes, abgeleitet aus der jüngsten Station. Texte dazu in `STATUS` (`shared/types.ts`).
Ausgang	`events[].result`, `events[].decision`, `NO_RESULT` in `analytics-news.mjs`	Ergebnis, Beschluss	Was eine Station ergab (beschlossen, abgelehnt, vertagt, Kenntnis). Platzhalter der Importe („Ergebnis … nicht belegt") sind kein Ausgang.
Dokument	`documents[]` (`SourceDocument`: `title`, `url`, `kind`)	PDF, Unterlage	Ein verlinktes Schriftstück am Bericht. **Hauptdokument** = `primaryPdf` (die Vorlage selbst; in der Ablage `pdfs.role` leer); **Anlage** = jedes weitere PDF (`role='other'`, Schritt „Anlagen laden"); **Sitzungsunterlage** = `sessionDocument` (Niederschrift oder Einladung der ganzen Sitzung, an 5 und mehr Berichten; wird nie zusammengefasst). **Dokumentart** = `documentType` aus `shared/document-type.mjs` (`paper`, `decision`, `attachment`, `minutes`, `invitation`, `other`), berichtigt durch den Textanfang (`texts.head_type`).
Textfassung	`texts` in `index.sqlite`, Datei `files/<ab>/<sha256>.txt`, Seiten getrennt durch `\f`	Textebene, Text	Der aus einem PDF gelesene Text. **Textebene** heißt der Teil, den das PDF selbst enthält; **OCR** liest Seiten ohne Textebene (Scans).
Suchkarte	`search_cards` (eine Zeile je aktivem Bericht, gepflegt von Triggern auf `topics`), `search`, `search_exact`, `label`, `formal`, `date`, `next_date`, `gremium`	Karte (mehrdeutig, siehe Abschnitt 4), Treffer	Die Projektion eines Berichts für die Suche: Suchtext, Label, Datum „Stand heute", Gremium, Formalie ja/nein. Suche, Trends, Social Media und plenara.X lesen Suchkarten, nie `topics.payload`.
Vorgang	`x_cases` (eine Zeile je Vorgang), `report_rules.case_key` = Gebiet + `titleKey(title)` (`shared/topic-facts.mjs`)	Case, Sachvorgang; in der Suchoberfläche heißt auch ein einzelner Bericht „Vorgang"	**In plenara.X:** alle Berichte eines Gebiets mit demselben normalisierten Titel, also derselbe Antrag im Ausschuss und im Rat, einmal gezählt. **Sachvorgang** = Vorgang, dessen Punkt ein Sachpunkt ist; nur Sachvorgänge zählen in den Auswertungen.
Thema, **Unterthema**	`theme`, `subs` (`shared/themes.mjs`: `THEMES`, `SUBTOPICS`, `THEMES_VERSION`), `report_rules.theme/subs`, `payload.theme`	Sachgebiet	Eines von 10 Themen und bis zu 7 Unterthemen je Thema, aus Titel, Betreff und Beschlussvorschlag eines Sachpunkts.
Label	`label` (Suchkarte), `classification` (Bericht), Katalog `LABELS` in `shared/labels.mjs` (`labels-v3` = 10 Themen + `unklar` „Sonstiges"), View `topic_effective_labels`	Einordnung, Kategorie	Das Thema, das Suche, Filter und Landkarte zeigen. Rangfolge: **Einordnung von Hand** → **KI-Label** → Thema aus den Regeln → **Titelregel** (`classifyTopic`, `title-rules-v7`). Ältere Labels (`labels-v2`, 16 Werte) werden fest übersetzt.
Was geschieht	`stage` (`shared/stage.mjs`), `report_rules.stage`, `x_signals.phase`	Stand (bei Signalen), Art des Vorgangs	Was mit einem Vorgang passiert, quer zu den Themen: Bericht, Bauantrag, Vergabe, Planung, Beschluss … Seit 15.10.2026 nicht mehr im Atlas, aber Stand der Signale Solar, Schulen, Mobilität und Eingang des Newsradars.
Fakt	`topic_facts` (`topic_id`, `kind`, `value`, `detail`, `method`), Regeln in `shared/topic-facts.mjs` (`FACTS_VERSION`)	Wert, Baustein	Ein ausdrücklich genannter Wert oder eine Einordnung aus Titel oder Text eines Berichts: Hebesatz, Gebühr, Fraktion, Verfahrensschritt eines Bebauungsplans … **Werte für die Karte** sind die Fakten über dem Qualitätstor (`shared/map-facts.mjs`).
Signal	`x_signals` (`case_key`, `signal`, `art`, `phase`), Regeln in `plenara-x-signals.mjs`	Kategorie	Ein Ereignis für eine Branche (Vergabe, Wohnbau, Wind, Solar, Wärme, Schulen, Straßen, Wasser, Breitband, Mobilität, Gewerbe) mit **Art** (Filter) und **Stand**.
Regel-Zusammenfassung	`payload.ruleSummary` (`RuleSummary`: `short`, `long`, `score`), `server/integrations/rule-summary.mjs`	Zusammenfassung (Regel)	Satzauswahl aus dem Hauptdokument ohne Modell, kurz und lang.
KI-Fassung	`payload.shortSummary`, `longSummary` mit `generatedBy`, `contentAnalysis`, `labelAssessments.ai`; Kopie in `article_analyses`; Arbeitsposten `ai_work_items`	KI-Zusammenfassung, KI-Label	Kurzfassung (ein Satz), Langfassung und KI-Label, die ein Agent über den internen MCP-Dienst einreicht. Regeln und KI überschreiben einander nie.
Einordnung von Hand	`labelAssessments.review`	Review	Ein Label, das ein Mensch gesetzt hat; geht allem anderen vor. Gelesen von View und `storedClassification`; einen Schreiber gibt es im Code derzeit nicht (`manual-analysis.mjs` ist trotz des Namens die Titelregel).

Begriff	Im Code und in der Datenbank	Auch genannt	Bedeutung
Quellenstand	`source_coverage` (eine Zeile je Gebiet, `payload` vom Typ `Coverage`)	Abdeckung, Coverage	Je Gebiet: Zeitraum von/bis, letzter Abruf, letzter Erfolg, Sitzungen, vollständig ja/nein, Probleme, nächster Versuch.
Stand (Datenstand)	`data_revisions` (`content`, `search`, `coverage`), `region_revisions`, Parameter `revision` der Suche, `search-words` in `system_state`	Revision, Datenstand	Ein Zähler je Art, der mit jeder Änderung steigt. Zwischenspeicher, Blättern, ETags und das Nachführen der Wortliste vergleichen ihn. In der Oberfläche heißt das Datum des letzten Abrufs ebenfalls „Stand“.

2. Abläufe und Betrieb

Begriff	Im Code	Bedeutung
Ablauf	`FLOWS` in `shared/pipeline-terms.mjs`	Einer von dreien: **Tagespipeline** (jede Nacht), **Nacharbeit** (tagsüber, automatisch), **Altbestand** (auf Knopfdruck).
Stufe (eines Ablaufs)	`PHASES`: `abruf`, `extraktion`, `verarbeitung`	Die drei Stufen je Bericht: **Abruf** (Berichte holen), **Extraktion** (PDFs, Text, OCR), **Verarbeitung** (Einordnung, Regeln, Suche, Zahlen, KI). Nicht die Stufe eines Kontos (Abschnitt 3).
Schritt	`STEPS` (`id`, `name`, `phase`, `aliases`)	Benannter Teil eines Laufs: Sichern, Abruf, PDF laden, Anlagen laden, Text lesen, OCR, OCR-Nachladen, Dokumentart, Grenze halten, Stand zählen, Titel einordnen, Regeln anwenden, Suche aktualisieren, Zahlen rechnen, KI, Wege zählen. Protokolle tragen diese Namen.
Abruf (Import)	`runSync` in `server/services/sync.ts`, Prozess `import-worker.mjs`	Die Sitzungen und Berichte einer Quelle holen und in `topics` schreiben. „Import“ ist dasselbe Wort im Code.
Abrufauftrag	`admin-pipeline-job` in `system_state`, `pipeline-jobs.mjs`	Gespeicherter Auftrag: Liste der Gebiete (Posten), Zeitfenster, Fortschritt, Sperren. Angelegt von der Admin-Seite, vom nächtlichen Job oder von Hand; abgearbeitet vom **Importprozess**.
Importprozess	`dist/standalone/worker/import-worker.mjs` (Quelle `server/node/import-worker.mjs`)	Eigener Node-Prozess neben dem Webserver, der Abrufaufträge in 24 **Bahnen** abarbeitet.
Teilabruf	`chunk`, `meeting-marks`, `resumable`	Ein Abruf eines Gebiets innerhalb der Zeitgrenze; der nächste Lauf setzt hinter den Marken fort.
PDF-Ablage	`RM_PDF_STORE`, `server/node/pdf-store.mjs`, `scripts/pdf-store/`, `index.sqlite`, `files/<ab>/<sha256>.pdf` und `.txt`	Speicher der Original-PDFs (jede Datei einmal, nach Prüfsumme) und ihrer Textfassungen, nur auf dem Node-Server.
Grenze	`BUDGET_GB` und Score in `shared/pdf-rank.mjs`, `keeper` in `store-lib.mjs`	Feste Größe der Ablage (80 GB). Eine neue Datei verdrängt die schlechteste oder bleibt nur als Text (`text-only`). Im Code „Budget“.
Regeln je Bericht	`readReport` in `server/integrations/report-rules.mjs`, `scripts/pdf-store/read-reports.mjs`, Schlüssel in `report_rules`	Ein Durchlauf je Bericht in fester Reihenfolge: Hauptdokument, Art des Punkts, Thema, Regel-Zusammenfassung, Fakten. Jede Regel liest nur Quellen.
Wortliste	`search_words`, `search_postings`, `search_word_areas`, `search_word_facets`, `refreshSearchWords` in `search-words.mjs`	Index der Suche: Wörter mit Kartenzahl, Karten-IDs je Wort und vorberechnete Zahlen je Gebiet, Thema und Status.
Zahlen für plenara.X	`x_cases`, `x_signals`, `scripts/plenara-x/build-x-cases.mjs`, `x-cases.mjs`	Vorberechnete Vorgänge und Signale, Schritt „Zahlen rechnen“.
Tagesbereich	`inDailyRange`, `dailyBoundary`, `DAILY_SINCE` in `work-failures.mjs`	Sitzungen der letzten 7 Tage (und künftige), frühestens 12.10.2026. Nur ihn bearbeitet die Nacharbeit; alles andere ist **Altbestand**.
Fehlerspeicher	`work_failures`, `source_hosts`, `recordFailure`, `clearFailure`, `classify`	Jeder Fehlschlag mit **Grund** (`class`), Versuchen und nächstem Termin. Gründe: Quelle bremst (`blocked`), Zugriff verweigert (`denied`), vorübergehend (`transient`), fehlt dauerhaft (`gone`), zu langsam (`slow`), Sperre im Haus (`busy`), Fehler im eigenen Code (`internal`).
Serverstufe	`source_hosts.state`: `ok` (liefert), `bremsen`, `gesperrt`, `pruefen`	Wartestufe eines Servers; wartende Server lassen alle Abrufer aus. Ein fälliger Server bekommt erst eine **Stichprobe** (`SAMPLE`, 3 Einträge).
Folgekette	`followUp` in `scripts/nacharbeit.mjs`	Gelingt ein Schritt, laufen für dieselben Berichte gleich die nächsten Stufen (Abruf → PDF laden → Anlagen → Text → OCR → Regeln).
Job	`JOBS` in `server/integrations/schedules.mjs`, Einheiten `ratsmonitor-<job>.timer/.path/.service`	Ein zeitgesteuerter Lauf auf dem Server (backup, import-nightly, startseite-trends, pdf-nightly, rule-texts, nacharbeit, ocr, altbestand, import, pdf-job, accounts).
Zeitplaner	`scripts/schedule.mjs`, Einstellungen `schedules`, Läufe `schedule-runs`, Laufbuch `schedule-log`	Entscheidet alle 10 Minuten, ob ein Job dran ist; führt ihn aus und vermerkt den Lauf.
Auslöser	systemd `.timer` (alle 10 Minuten) und `.path` (beobachtet `data/trigger/<job>`), `job-trigger.mjs`	Was den Zeitplaner aufruft: der Timer oder ein Knopf in der Administration (schreibt die Auslöser-Datei).
Anhalten	`schedule-stop`, `trigger/stop-<job>`, `ratsmonitor-stop@.service`, `onStop` in `scripts/pdf-store/state.mjs`	Laufenden Job beenden, ohne auf das Ende des Schritts zu warten. Nie `systemctl stop …timer`.
Auftrag (der Ablage)	`pdf-store-job` in `system_state` (`{queue, running: {chain,ocr}, recent}`), `JOB_KINDS` in `pdf-store-admin.mjs`: `run`, `retry`, `others`, `texts`, `ocr`	Ein Knopf im Reiter PDF-Import oder auf der Seite Altbestand, den der Job `pdf-job` als Lauf ausführt.

Begriff	Im Code	Bedeutung
Bahn	Ablage: `laneOf` → `chain` (Laden) und `ocr`; Abruf: 24 parallele Bahnen des Importprozesses	Zwei Bedeutungen: In der Ablage sind es die zwei Spuren, die nebeneinander laufen dürfen; im Abruf die parallelen Arbeiter eines Auftrags.
Lauf	`schedule-runs`, `pdf-store-runs`, `import_runs`, Ergebnis `RESULT_TERMS` (fertig, mit Fehler, angehalten, ausgelassen)	Eine Ausführung eines Jobs oder Auftrags mit Beginn, Ende, Ergebnis.
Sperre	`import-lock.mjs`, `search-words-lease`, `nightly.lock`, `ocr-lane.lock`, `eval.lock`, `admin-builds`	Verhindert, dass zwei Läufe dasselbe schreiben. Jede Sperre hat eine Frist, die ein lebender Lauf verlängert.
Lebenszeichen	`pdf-store-heartbeat`, Fortschritt `pdf-store-progress-*`, Journal des Importprozesses	Regelmäßig geschriebener Stand, an dem die Administration erkennt, ob ein Dienst läuft.
Programmstand	`/srv/ratsmonitor/app/releases/<commit>`, `current`, `work_failures.release`	Ein ausgerollter Commit.
Vorberechnung (Admin)	`region_stats`, `region_series`, `admin_agg`, `admin-stored.mjs`, `refreshRegionFacts`	Werte je Gebiet und globale Auswertungen, die die Admin-Seiten lesen statt zu rechnen.

3. System, Konten, Oberfläche

Begriff	Im Code	Bedeutung
Hauptdatenbank	`ratsmonitor.sqlite` (`DATABASE_FILE`), im Code `env.DB` mit D1-Schnittstelle, lokal die Datei unter `.wrangler/`, Migrationen `drizzle/`, Buchführung `d1_migrations`	Alle Berichte, Suchkarten, Regeln, Zahlen, Betriebszustände. Eine Datei, SQLite, WAL.
Kontendatei	`accounts.sqlite` (`ACCOUNTS_FILE`), `env.ACCOUNTS`, Migrationen `drizzle-accounts/`, `accounts_migrations`	Konten, Anmeldungen, gespeicherte Suchen. Getrennt von der Hauptdatenbank, nie in Kopien, Export oder MCP. Im Dev-Server liegen die Tabellen in der lokalen Hauptdatenbank.
Betriebszustand	Tabelle `system_state` (`key`, `value` als JSON)	Schlüssel-Wert-Speicher für Aufträge, Stände, Zähler, Einstellungen, Ergebnisse. Schlüssel siehe Datenmodell.
Dev-Server	`npm run dev` → `scripts/run-framework.mjs dev`, Port 5173, Miniflare mit lokaler D1	Lokale Entwicklung. Startet Nachführen der Wortliste und der Admin-Werte im Hintergrund.
Node-Server	`dist/standalone/serve.mjs` (Quelle `server/node/serve.mjs`), Port 3000, hinter Caddy	Der Produktionsprozess auf dem VPS: Seiten, API, Administration. Daneben laufen Importprozess, Jobs und X-Worker-Threads.
D1-Adapter	`server/node/d1-sqlite.mjs`, `server/node/cloudflare-workers.mjs`	Lässt den für Cloudflare D1 geschriebenen Code unverändert auf `node:sqlite` laufen.
Konto	`users`, `plan_grants`, `user_roles`	Ein Nutzer mit E-Mail-Adresse, Stufe und Rollen.
Anmeldesitzung	`sessions` (Kontendatei), Cookie, `server/accounts/sessions.mjs`	Angemeldeter Zustand eines Browsers. Im Code `session`; im Text immer „Anmeldesitzung", damit sie nicht mit der Sitzung eines Gremiums verwechselt wird.
Stufe (eines Kontos)	`TIERS` in `shared/plans.mjs`: `guest`, `basic`, `pro`, `enterprise`; `LIMITS`, `FEATURES`; `plan_grants`; `sessions.view_tier`	Was ein Konto darf. Ohne `plan_grants`-Zeile gilt Gast. Nicht die Stufe eines Ablaufs (Abschnitt 2). Im Produkt „Tarif".
Gast	`guest`, `guest-search.mjs`	Nicht angemeldet oder ohne freigeschaltete Stufe: 10 Treffer, keine Filter.
Admin	Rolle `admin` in `user_roles`, `adminState()` in `server/services/session.ts`, `ADMIN_HOSTS`, zweiter Faktor `user_totp`	Rolle am Konto; gilt über den Tunnel ohne weiteren Code, sonst mit TOTP.
Zugangssperre	`RM_ACCESS=invite`, `middleware.ts`, `server/accounts/access.mjs`	Sperrt alles außer Anmeldung, Rechtstexten und `/api/me`.
Ansehen als	`sessions.view_tier`	Ein Admin sieht die Seite wie eine andere Stufe.
Postausgang	`mail_outbox`, `server/accounts/mail.mjs`, `MAIL_TRANSPORT`	Alle Mails gehen über diese Tabelle; der Kontojob verschickt sie.
Suche	`GET /api/search`, `server/integrations/monitor-search.mjs`, Oberfläche `state/search.tsx`	Die Begriffs- und Filtersuche über Suchkarten.
Landkarte	`components/map/MapPanel.tsx`, `lib/geo/mapEngine.ts`, `badgeCounts`, `areaCounts`	Die Deutschlandkarte mit Trefferzahlen je Gebiet. Nur „Landkarte", nie „Karte" allein.
Artikelseite	`pages/DetailPage.tsx`, Adresse `/beschluss/<id>`, Datenquelle `getTopic`	Die Detailseite eines Berichts mit Beratungsfolge, Dokumenten, Zusammenfassungen.
plenara.X	`/analytics/*`, `GET /api/x`, `/api/x/signale`, `/api/analytics/*`, `pages/analytics/`	Der Analyse-Teil (Stufe Enterprise): **Atlas**, **Signale**, **Fakten** und die **Tools** (Trends mit Ausbreitung, Themennetz, Ortsvergleich, Beschlüsse, Beratungswege, Fraktionsprofil, Newsradar).
Agent (KI)	Client des MCP-Dienstes `scripts/mcp-internal.mjs`, Werkzeuge `ai_list`, `ai_claim`, `ai_read`, `ai_renew`, `ai_release`, `ai_submit`	Ein Modell mit Werkzeugzugriff, das Berichte liest und KI-Fassungen einreicht. Startet nie von selbst.
Arbeitsposten	`ai_work_items`	Ein Bericht in der Liste des Agenten mit Zustand (bereit, reserviert, erledigt, veraltet).
Kundendienst (MCP)	`scripts/mcp-customer.mjs`, `mcp-customer-data.mjs`	Reine Recherche über gespeicherte Daten für Kunden; keine interne Verbindung.

Begriff	Im Code	Bedeutung
Admin-Seite	`/admin?seite=<name>`, `app/admin/page.tsx`, `components/admin-*.tsx`	Ein Reiter der Administration.

4. Mehrdeutige Wörter und wie wir sie verwenden

Wort	Bedeutungen	Regel in dieser Dokumentation
Karte	Suchkarte (`search_cards`) und Landkarte (`MapPanel`)	**Suchkarte** für die Tabelle, **Landkarte** für die Karte. „Karte" allein nur in Zitaten aus Code und Admin.
Vorgang	In der Suche ein einzelner Bericht („Vorgänge gefunden", `/beschluss/<id>`); in plenara.X die Zusammenfassung gleicher Titel je Gebiet (`x_cases`)	**Bericht** für die Zeile in `topics`, **Vorgang** nur für `x_cases`. Die Oberfläche der Suche bleibt bei „Vorgang" (für den Leser ist es derselbe Antrag).
Artikel	Älterer Name für Bericht in Code und Tabellen (`article_versions`, `article_analyses`, `saved_articles`, `ArticleCard`, `exportArticle`)	**Bericht**; Tabellennamen bleiben, wie sie sind.
Stufe	Stufe eines Ablaufs (Abruf, Extraktion, Verarbeitung), Stufe eines Kontos (Gast bis Enterprise), Stufe eines Servers (liefert bis prüfen), Wartestufe (`level`)	Immer mit Zusatz: **Stufe des Ablaufs**, **Stufe des Kontos** (oder Tarif), **Serverstufe**, **Wartestufe**.
Sitzung	Sitzung eines Gremiums; Anmeldesitzung eines Kontos (`sessions`)	**Sitzung** nur für Gremien, **Anmeldesitzung** für Konten.
Bahn	24 Bahnen des Abrufs; Bahnen Laden und OCR der Ablage	Mit Zusatz: **Bahn des Abrufs**, **Bahn der Ablage**.
Label, Thema	Label = angezeigtes Thema nach Rangfolge; Thema = Ergebnis der Themenregel (`payload.theme`)	**Thema** für die Regel, **Label** für das Angezeigte. Seit `labels-v3` ist der Katalog derselbe (10 Themen plus Sonstiges).
Stand	Datenstand (`data_revisions`); Stand der Signale (`phase`); Datum des letzten Abrufs in der Oberfläche; „Stand zählen" (Ablage)	**Datenstand** für die Zähler, sonst mit Zusatz.
Import, Abruf	Dasselbe: Code sagt `import`, Pipeline-Begriffe sagen „Abruf"	**Abruf** im Text, `import` im Code.
Job, Auftrag, Lauf	Job = zeitgesteuerte Einheit; Auftrag = gespeicherter Wunsch (Abrufauftrag, Auftrag der Ablage, Bauauftrag der Admin-Vorberechnung); Lauf = eine Ausführung	So wie links.
Server	Rechner einer Quelle (`source_hosts`); eigener Node-Server; VPS	**Server** einer Quelle nur mit Zusatz oder im Kontext des Fehlerspeichers; sonst **Node-Server** oder **VPS**.
Analyse	`article_analyses` (KI-, Regel- und Hand-Ergebnisse je Bericht); plenara.X-Auswertungen; Admin-Reiter „Analyse"	**Auswertung** für plenara.X, **Analyse** nur für die Tabelle und ihre Varianten.

5. Kennungen

Kennung	Form	Beispiel	Wo
Gebiet	`nrw-`, `nds-`, `de-` plus amtlicher Schlüssel, oder benannte Kennung	`de-09162000`, `nds-03241001`, `billerbeck`	`region_id` überall
Bericht	je Leser; `<quelle>-vo-<nr>`, `<quelle>-top-<sitzung>-<nr>`, `<quelle>-oparl-<hash>`, Münster `papers-…`	`billerbeck-vo-12345`	`topics.id`, `search_cards.id`, `report_rules.topic_id`, `topic_facts.topic_id`, `ai_work_items.topic_id`
Vorgang	Gebiet + `|` + normalisierter Titel (`titleKey`: klein, alles außer Buchstaben und Ziffern wird Leerzeichen); ungelesene Berichte Gebiet + `|` + `lower(title)`	`billerbeck|bebauungsplan nr 12`	`report_rules.case_key`, `x_cases.case_key`, `x_signals.case_key`
PDF	SHA-256 des Inhalts	`3f9a…`	`pdfs.sha256`, `texts.sha256`, Dateiname in `files/<ab>/`
Fehlerspeicher	Schritt + Gegenstand	`pdf` + Adresse, `import` + Gebiet, `rules` + Bericht, `ocr` + Prüfsumme	`work_failures (step, key)`
Wortliste	Wort (Kleinbuchstaben, Umlaute gefaltet, ab 3 Zeichen)	`waermeplanung`	`search_words.word`
Konto	`u_` + Zufall	`u_66FW…`	`users.id`
Datenstand	Art + Zahl	`search` 4711	`data_revisions`

Abruf: Berichte aus den Ratsinformationssystemen holen

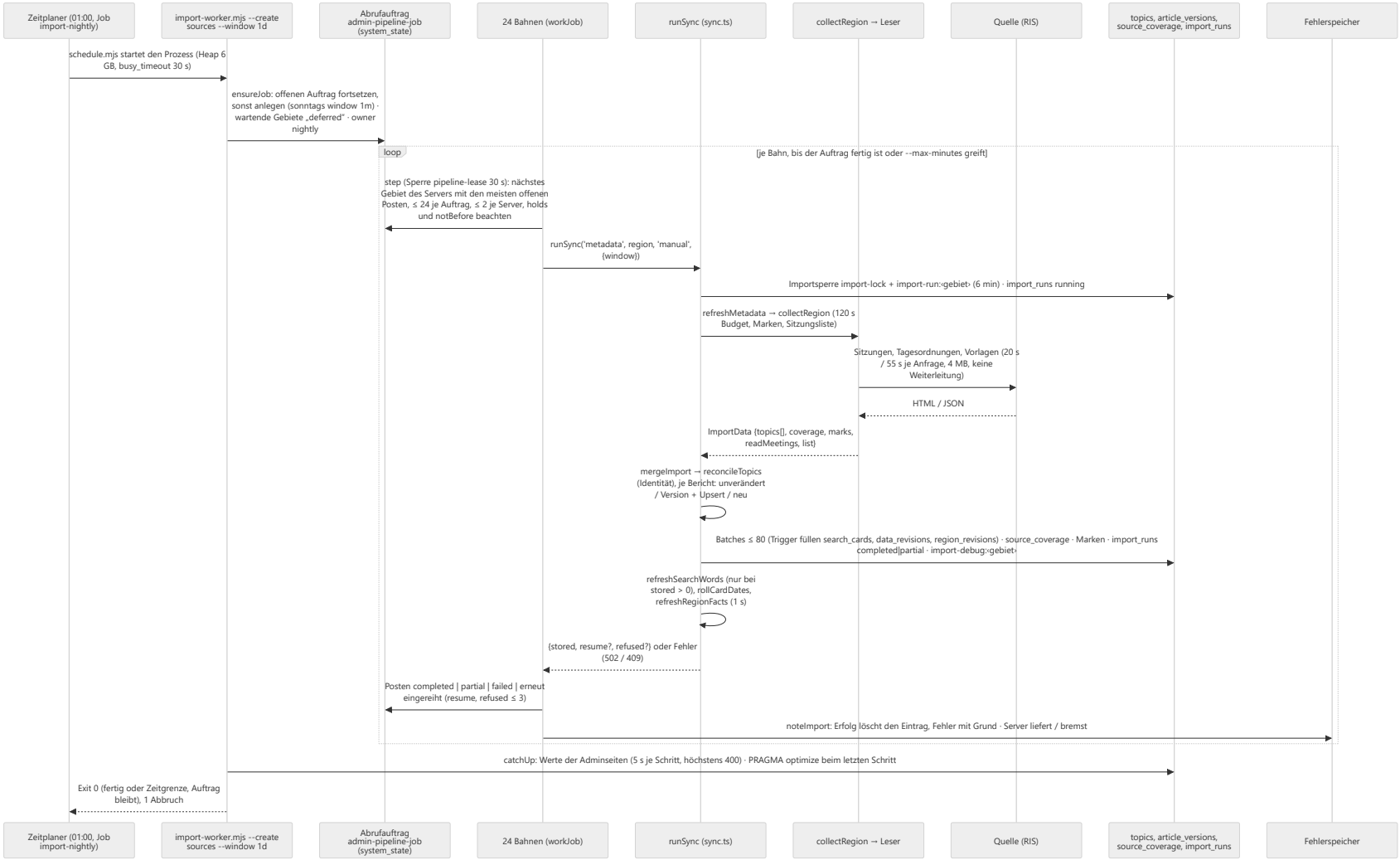
Stand: 17. Oktober 2026, gelesen am Code des Branchs `nachtlauf-fixes`. Teil der technischen Dokumentation. Begriffe: Glossar. Der Abruf ist die Stufe 1 der Pipeline; im Code heißt er durchgehend `import`.

1. Gebiet, Quelle, Leser, Server

Begriff	Was es ist	Woher
Gebiet	Eintrag im Katalog `CATALOG` (`shared/catalog.mjs`), 5.324 Gebiete: NRW `nrw-<Schlüssel>` (dazu sieben benannte Kennungen wie `billerbeck`, `muenster`), Niedersachsen `nds-<Schlüssel>`, übrige Länder `de-<Schlüssel>`; Arten `city` (Gemeindeebene, auch Gemeindeverbände mit `members`) und `district` (Kreis)	`nrw-regions.json`, `nds-regions.json`, `de-regions.json` (BKG VG250, Stand 04.10.2026), Einwohner aus Wikidata
Quelle	Katalogeintrag mit derselben Kennung, der sagt, wie das Gebiet gelesen wird: `url`, `adapter` oder `method` (`oparl`, `pending`), `extension` (asp/php bei SessionNet), `body` (OParl-Körperschaft), `oparlFallback`, `consent`	`NRW_SOURCES` in `server/integrations/source-catalog.mjs` aus sieben JSON-Dateien (`nrw-`, `nds-`, `de-`, `expanded-`, `nearby-`, `statewide-`, `citystate-sources.json`); bei gleicher Kennung gilt die zuletzt genannte Datei. Dazu sechs fest eingetragene Altquellen (`regions.mjs`) und Münster. `canImport` gilt für 4.041 Gebiete (alles außer `method:'pending'`)
Leser	Modul, das ein Ratsinformationsprodukt liest (Abschnitt 3); Registry `READERS` in `readers.mjs`, Auswahl in `collect-region.mjs`	
Server	Gruppe von Hosts nach registrierbarer Domain plus IP (`source-servers.mjs`, `source-servers.json`); `provider(region)` liefert ihn. Grenze 2 gleichzeitige Abrufe je Server, Pausen (`holds`) und Serverstufe in `source_hosts` hängen daran	
Körperschaft	OParl-Körperschaft innerhalb eines gemeinsamen Systems, gewählt über `source.body`, Gemeindeschlüssel oder normalisierten Namen; muss eindeutig sein (`body-identity.mjs`)	

Die übrigen JSON-Dateien im Ordner dienen nicht dem Abruf: `source-overrides.json` fließt beim Katalogbau in `nrw-sources.json` ein (`scripts/apply-source-overrides.mjs`), `source-consents.json` dem Katalogbau und dem Dashboard, `source-robots.json` und `source-atlas.json` nur Admin und Lückenatlas.

2. Ein nächtlicher Abruf von Anfang bis Ende


Zeitplaner (01:00, Job import-nightly)
import-worker.mjs --create sources --window 1d
Abrufauftrag admin-pipeline-job (system_state)
24 Bahnen (workJob)
runSync (sync.ts)
collectRegion → Leser
Quelle (RIS)
topics, article_versions, source_coverage, import_runs
Fehlerspeicher
schedule.mjs startet den Prozess (Heap 6 GB, busy_timeout 30 s)
ensureJob: offenen Auftrag fortsetzen, sonst anlegen (sonntags window 1m) · wartende Gebiete „deferred" · owner nightly
loop
[je Bahn, bis der Auftrag fertig ist oder --max-minutes greift]
step (Sperre pipeline-lease 30 s): nächstes Gebiet des Servers mit den meisten offenen Posten, ≤ 24 je Auftrag, ≤ 2 je Server, holds und notBefore beachten
runSync('metadata', region, 'manual', {window})
Importsperre import-lock + import-run:‹gebiet› (6 min) · import_runs running
refreshMetadata → collectRegion (120 s Budget, Marken, Sitzungsliste)
Sitzungen, Tagesordnungen, Vorlagen (20 s / 55 s je Anfrage, 4 MB, keine Weiterleitung)
HTML / JSON
ImportData {topics[], coverage, marks, readMeetings, list}
mergeImport → reconcileTopics (Identität), je Bericht: unverändert / Version + Upsert / neu
Batches ≤ 80 (Trigger füllen search_cards, data_revisions, region_revisions) · source_coverage · Marken · import_runs completed|partial · import-debug:‹gebiet›
refreshSearchWords (nur bei stored > 0), rollCardDates, refreshRegionFacts (1 s)
{stored, resume?, refused?} oder Fehler (502 / 409)
Posten completed | partial | failed | erneut eingereiht (resume, refused ≤ 3)
noteImport: Erfolg löscht den Eintrag, Fehler mit Grund · Server liefert / bremst
catchUp: Werte der Adminseiten (5 s je Schritt, höchstens 400) · PRAGMA optimize beim letzten Schritt
Exit 0 (fertig oder Zeitgrenze, Auftrag bleibt), 1 Abbruch

1. **Auslöser:** der Zeitplaner startet um 01:00 `import-worker.mjs --create sources --window 1d --weekly-window 1m --weekly-day 0 --max-minutes 240 (Job import-nightly). Die Argumente liest parseArgs in pipeline-worker.mjs; windowForTonight` nimmt sonntags (deutsche Zeit) `1m`. Ist `RM_PDF_STORE` gesetzt, wird die Ablage eingebunden (`useDocumentStore`).
2. **Auftrag anlegen oder fortsetzen** (`ensureJob`): ein offener Auftrag wird fortgesetzt, außer er ist pausiert oder hat sich seit 6 Stunden nicht bewegt. `pipelineAction({action:'create', stage:'metadata', regions:'sources', window, replace:true, deferred})` legt je Gebiet einen Posten an (`queued`, `deferred` für Gebiete mit wartendem Eintrag im Fehlerspeicher oder wartendem Server, `unavailable`), jeder mit `server: provider(region)`. Gespeichert als JSON in `system_state` `admin-pipeline-job`, danach `owner: 'nightly'`.
3. **24 Bahnen** (`workJob`): jede Bahn ruft in einer Schleife `pipelineAction({action:'step'})`. „Warteschlangenschritt läuft bereits" heißt 2,5 s warten; ein ersetzter Auftrag beendet die Bahn; andere Fehler 5 s Pause, nach 5 in Folge gibt die Bahn auf. Jede Minute ein Protokolleintrag mit den Zählern.
4. **Ein Schritt** (`step` in `pipeline-jobs.mjs`, Sperre `pipeline-lease` 30 s): Posten, die über 5 Minuten „laufen", werden `unknown`; `next()` wählt ein wartendes Gebiet des Servers mit den meisten offenen Posten, der noch Platz hat (höchstens 24 je Auftrag, 2 je Server, `holds` und `notBefore` beachtet); der Posten wird `running`. Der Import selbst läuft außerhalb der Sperre.
5. ** `runSync` ** (`server/services/sync.ts`): Zeitraum prüfen, mitgelieferte Startdaten einspielen (`ensureData`, nur beim ersten Mal), bisherigen Quellenstand lesen, Importsperre nehmen (`acquireImport`: `import-lock` gemeinsam plus `import-run:<gebiet>`, 6 Minuten, nach dem Sammeln erneuert; 409 mit `busy: 'area'|'stock'`), Aufzeichnung anlegen (`createTrace`), `import_runs` mit `running` schreiben (Läufe älter als 30 Tage löschen), dann `refreshMetadata`.
6. ** `refreshMetadata` :** alle gespeicherten Berichte des Gebiets laden, Sitzungsadressen sammeln, Marken (`import-marks:<gebiet>`) und gemerkte Sitzungsliste (`import-list:<gebiet>`, 6 h) aus `system_state` lesen, `collectRegion(region, {maxDurationMs: 120000, window, marks, trace, oparlProbeAt})` aufrufen.
7. **Leser wählen und lesen** (`collect()` in `collect-region.mjs`, Abschnitt 3). Ergebnis `ImportData`.
8. **Zusammenführen:** Sperre erneuern; `mergeImport` (`merge-import.mjs`) versieht neue Berichte mit `metadata` und ruft `reconcileTopics` (Abschnitt 5); je Bericht `preserveAnalysis`, dann der Vergleich „unverändert?", bei Änderung eine Zeile in `article_versions` und ein Upsert in `topics`. Geschrieben wird in Batches zu höchstens 80 Anweisungen; eine Identitätsgruppe bleibt in einem Batch.
9. **Quellenstand und Protokoll:** `importHealth` bewertet den Versuch, `source_coverage` wird per Upsert geschrieben; Marken und Sitzungsliste gespeichert; nur für Münster `latest-decision` und Push; `import_runs` auf `completed` oder `partial`; Aufzeichnung unter `import-debug:<gebiet>` (höchstens 1.500 Anfragen).
10. **Nacharbeiten in `runSync` :** `refreshSearchWords` (nur bei `stored > 0`, `onlyIfBuilt`, höchstens 20.000 Karten), `rollCardDates`, `refreshRegionFacts` (nur ohne `resume`, 1 s Budget); im `finally` `invalidateReads()` und Sperre freigeben. Bei einer Ausnahme wird `source_coverage` mit dem gescheiterten Versuch geschrieben, Antwort 502, bei Bedarf mit `refused`.
11. **Ergebnis im Auftrag:** 409 wird je nach Fall wiederholt, zurückgestellt oder der Auftrag wartet; `refused` pausiert den Server (`hold`, 60 s bis 1 h, Standard 5 Minuten), das Gebiet folgt höchstens 3-mal; andere Fehler `failed`; `resume` reiht das Gebiet erneut ein (Meldung „(Teil n)", höchstens `MAX_RESUMES` × Jahre des Zeitraums); sonst `completed` oder `partial`. Danach `noteImport` in den Fehlerspeicher. Der Schritt, der den Auftrag beendet, ruft `PRAGMA optimize`.
12. **Ende:** Bahnen enden bei `completed`, `cancelled`, pausiert, ersetzt oder `stop()` (Zeitgrenze); laufende Abrufe laufen zu Ende. Dann `catchUp` für die Werte der Adminseiten. Exit 0 fertig oder Zeitgrenze (der Auftrag bleibt), 1 Abbruch.
13. **Fortsetzung:** der Job `import` (alle 10 Minuten, `--run --max-minutes 240`) arbeitet einen offenen Auftrag weiter, lässt den Nachtauftrag aber aus, solange er sich in den letzten 10 Minuten bewegt hat (`nightlyBusy`). Gescheiterte Gebiete holt die Nacharbeit über `import-worker.mjs --direct --regions …` nach (Pipeline, Abschnitt 7).

Weitere Einstiege

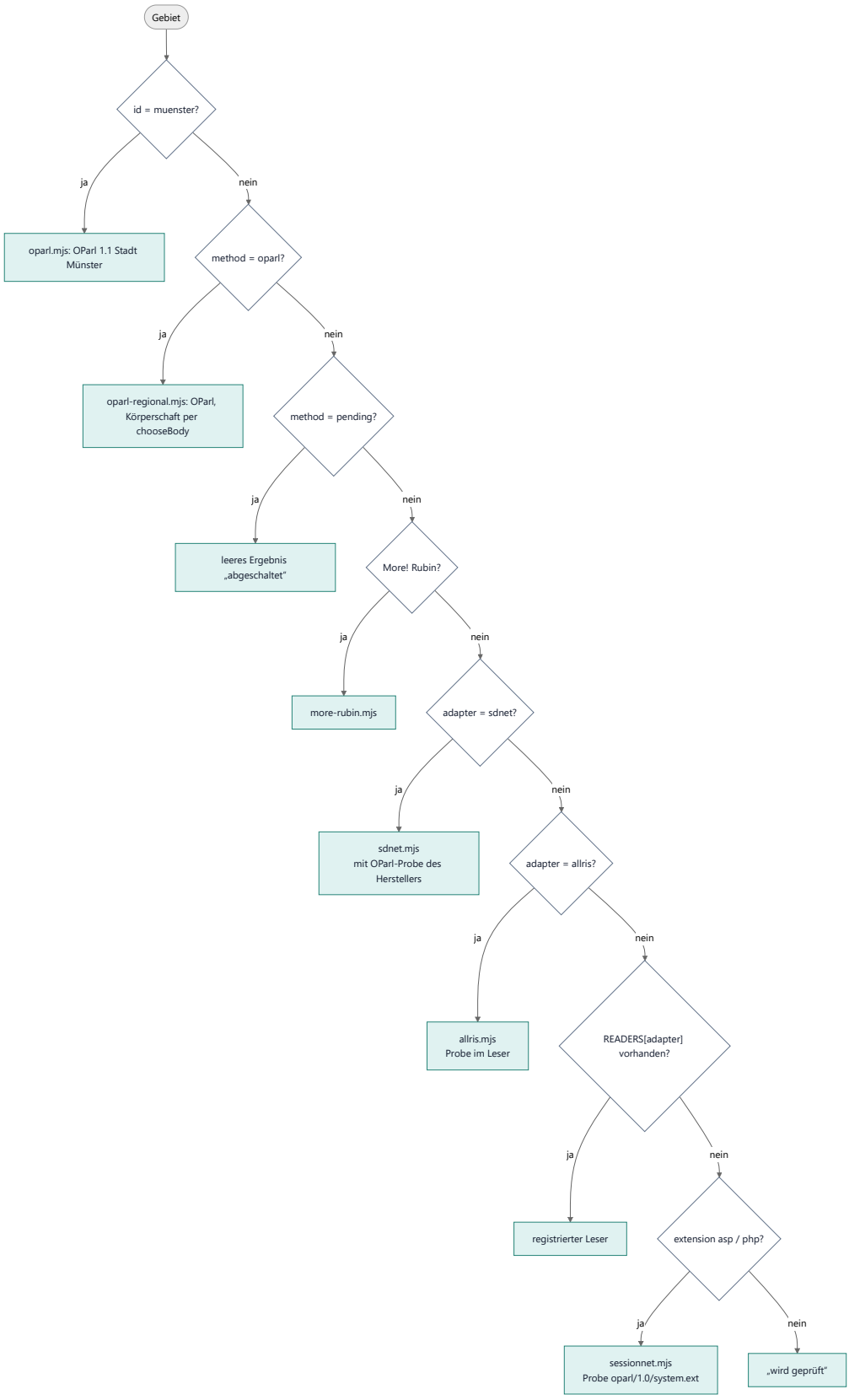
Weg	Was passiert
Admin-Reiter „Abruf & Verarbeitung" (`components/admin-processing.tsx`)	legt mit `create` an und arbeitet im Browser mit `run` ab: bis 8 Bahnen je Anfrage, je 90 s (`POST /api/admin/pipeline`, dieselbe Ausführungsfunktion wie der Prozess). „Im Hintergrund abrufen" legt den Auftrag nur an; der Job `import` holt ihn binnen 10 Minuten. Prozess und Browser dürfen denselben Auftrag treiben (jedes Gebiet genau einmal)
`POST /api/admin/import`	ein Gebiet, Standardzeitraum
`POST /api/internal/sync` (Bearer `IMPORT_TOKEN`)	ein Gebiet; älterer Weg von außen über `scripts/run-imports.mjs` (Gebiet für Gebiet, 3 Versuche, danach `/api/internal/admin-refresh`)

Weg	Was passiert
`import-worker.mjs --direct --regions … --window …`	ohne Auftrag, direkt je Gebiet (Nacharbeit, Altbestand)
Seite Altbestand	rückwirkender Abruf je 8 Gebiete, 1 bis 24 Monate

Aktionen des Auftrags: `create`, `cancel`, `pause`, `resume`, `status`, `step`, `run`. Struktur: Kopf `{id, stage, window, scope, createdAt, updatedAt, status queued|running|completed|cancelled, paused?, holds?, owner?, items[]}`, Posten `{region, status queued|running|completed|partial|failed|unknown|unavailable|deferred, processed, message, server, startedAt?, notBefore?, refusals?, resumes?}`. Zeiträume `1d|1w|1m|3m|12m|24m`, Standard `12m` (`shared/history-window.mjs`).

3. Die Leser

Auswahl (`collect()` in `collect-region.mjs`)



OParl-Probe: Viele Produkte bieten neben HTML eine OParl-Schnittstelle. Ein negatives Ergebnis der Probe gilt 30 Tage (`PROBE_DAYS`, `oparlProbeAt` im Quellenstand); `oparlFallback` im Katalog überspringt sie; ein positives Ergebnis bricht mit „Adapterfreigabe erforderlich" ab, damit die Quelle bewusst auf `method:'oparl'` umgestellt wird.

Leser und Produkte

Zahlen: Einträge im Katalog dieser Arbeitskopie (4.035 Quellen), einmal ausgezählt.

Leser (`adapter`)	Produkt	Modul	Einträge
Münster (fest)	OParl 1.1 Stadt Münster	`oparl.mjs`	1
`method:'oparl'`	OParl beliebiger Hersteller	`oparl-regional.mjs` (Öffentlichkeitsprüfung `public-agenda.mjs`)	261
(`extension` asp/php, auch `sessionnet`)	SessionNet (Somacos) bis 5.x, HTML-Seiten `si0040`, `si0057/si0056/to0040`, `vo0050`	`sessionnet.mjs` (Anwesenheit `sessionnet-details.mjs`)	1.358
`sessionnet6`	SessionNet 6, JSON `api/v1`	`sessionnet6.mjs`	16
`allris`	ALLRIS 4 (CC e-gov), `si010/to010/vo020`	`allris.mjs`	390
`allris3`	ALLRIS 3 (asp), fragt zuerst OParl	`allris3.mjs`	137
`sdnet`	SD.NET RIM (Sternberg), öffentliche Seiten	`sdnet.mjs`	92
`sdnet-rim4`	SD.NET RIM 4, selbst gehostet	`sdnet-rim4.mjs`	98
`more-rubin`	More! Rubin (`api.php`, JSON)	`more-rubin.mjs`	346
`ris-portal`	RIS-Portal comundus regisafe (Liferay)	`ris-portal.mjs`	542
`kic`	KIC-RIS (auch komuna), Gast-API	`kic.mjs`	404
`komfa`	KOMFA-RIS (kommunalfabrik)	`komfa.mjs`	12
`councilservice`	Sitzungsdienst mein-intra.net	`councilservice.mjs`	52
`ti-generator`	TI-Generator	`ti-generator.mjs`	33
`cron-ratsinfo`	cron Ratsinfo für TYPO3	`cron-ratsinfo.mjs`	6
`typo3-bi`	TYPO3-Bürgerinformationssystem (risportal)	`typo3-bi.mjs`	1
`hw-typo3`	Ratsinformationsmodul Hirsch & Wölfl	`hw-typo3.mjs`	4
`meeting-mobile`	Meeting Mobile / RIS Web (Domino)	`meeting-mobile.mjs`	4
`provox-iip`	Provox IIP	`provox-iip.mjs`	1
`ecics-ris`	Gemeinde-RIS ecics	`ecics-ris.mjs`	10
`ris-ng`, `hhdocuments`, `rio-sys`, `hitcom-ris`	kleine Systeme über den gemeinsamen Rahmen `small-common.mjs`	je ein Modul	0, 1, 1, 1
`webcontact-ratsinfo`	Ratsinfo webcontact (Nuxt-JSON)	`webcontact-ratsinfo.mjs`	1
`muenchen-risi`, `sim-hannover`, `piwi`, `pio`, `parlis`	Eigenentwicklungen großer Städte (München, Hannover, Wiesbaden, Offenbach, Frankfurt)	je ein Modul	je 1
`website`	Website einer Kommune ohne RIS: Bekanntmachungen, PDFs, RSS/iCal/WP-REST/JSON-LD/Sitemap	`website.mjs` (Hilfen `website-feeds.mjs`, `website-text.mjs`)	256
`ckan`	CKAN-Portal	`ckan.mjs`	0
`hamburg-transparenz`, `berlin`, `oparl-bezirke`	Transparenzportal Hamburg, Abgeordnetenhaus Berlin und Bezirke (nur mit Freigabe `consent`)	`citystates.mjs`	1, 1, 0
`method:'pending'`	abgeschaltet	–	1

`nrw-snapshot.mjs` ist kein Leser, sondern der mitgelieferte NRW-Bestand für Startdaten und Rückfall (`data/nrw-seed*`).

Was ein Leser liefert

`ImportData` (`server/types.ts`): `topics: StoredTopic[]`, `coverage` (`Coverage` plus `unchangedMeetings`, `resumable`, `warnings`, `method`, `oparlProbeAt`), `marks` (gelesene Sitzungen), `readMeetings`, `list` (Sitzungsliste für die Fortsetzung).

Beispiel SessionNet (`sessionnet.mjs`):

1. Kalendermonate `si0040` vom Beginn des Zeitraums bis zum nächsten Monat, 3 parallel; Sitzungen vor dem Zeitraum fallen weg.
2. Je Sitzung, neueste zuerst, höchstens 400 je Abruf (`MAX_MEETINGS`): Tagesordnungsseite; nur öffentliche Punkte („Ö …“).
3. Ein Fingerabdruck der Tagesordnung entscheidet, ob die Seiten dahinter neu gelesen werden (Marken, Abschnitt 5).
4. Die Vorlagenseite `vo0050` liefert PDF-Links (`documents`) und Feldpaare (`sourceData.records`, darunter die Beratungsfolge als Text).

5. Punkte mit gleicher Kennung werden gruppiert, ihre Sitzungen als `events` gesammelt; der Bericht wird gebaut.

Kennung eines Berichts: SessionNet `<quelle>-vo-<kvonr>` (Vorlage) oder `<quelle>-top-<ksinr>-<nummer>` (Punkt ohne Vorlage); dasselbe Muster bei allris, allris3, sdnet, more-rubin, sessionnet6, kic, ti-generator, councilservice; mit eigenem Infix bei ris-portal (`-rp-`), komfa (`-kf-`), website (`-web-<datum>-<hash>`), hw-typo3 (`-hw-`); oparl-regional `<quelle>-oparl-<sha256 der Vorlagen- oder Punktadresse, 20 Zeichen>`; Münster der Pfad der OParl-Kennung (`papers-…`).

Was eine Zeile in `topics` ist: ein Bericht. Verlinkt die Quelle eine Vorlage, ist es die Vorlage mit allen gelesenen Sitzungen; sonst ein einzelner Tagesordnungspunkt einer Sitzung. Münster nimmt zusätzlich neue Vorlagen ohne Sitzung auf. Zeilen mit `identity.mergedInto` sind Aliase (`active = 0`).

`events[]` (`TopicEvent`): je **gelesener** Sitzung, in der der Punkt öffentlich auf der Tagesordnung stand, ein Eintrag `{date, committee, status, description, result, url, attendance, decision}`; über Abrufe hinweg vereinigt nach `[url, date, committee]`. Die Beratungsfolge auf der Vorlagenseite wird bei SessionNet nicht zu `events`, sondern bleibt in `sourceData.records`.

`status`: je Station nach Regeln des Lesers, der Bericht übernimmt den Status der letzten Station. SessionNet (`itemStatus`): künftige Sitzung → `consulting` (mit Vorlage) oder `announced`; „vertagt" → `postponed`; „Kenntnis" → `info`; „empfohlen" → `recommended`; „beschlossen/abgelehnt" gilt nur bei einem beschließenden Gremium (`DECIDING_BODY` in `source-fields.mjs`) als `approved` / `rejected`, sonst `recommended`; sonst `unknown`. OParl über `authoritative` / `role`; eine vergangene Sitzung ohne Ergebnis wird `unknown`.

`documents[]` (`SourceDocument {title, url, kind}`): PDFs, die HTML-Tagesordnung (`html`), der OParl-Datensatz (`oparl`). Das Hauptdokument (`primaryPdf`) wird beim Abruf nicht berechnet und nicht gespeichert; es entsteht erst in der Extraktion und in den Regeln (Pipeline).

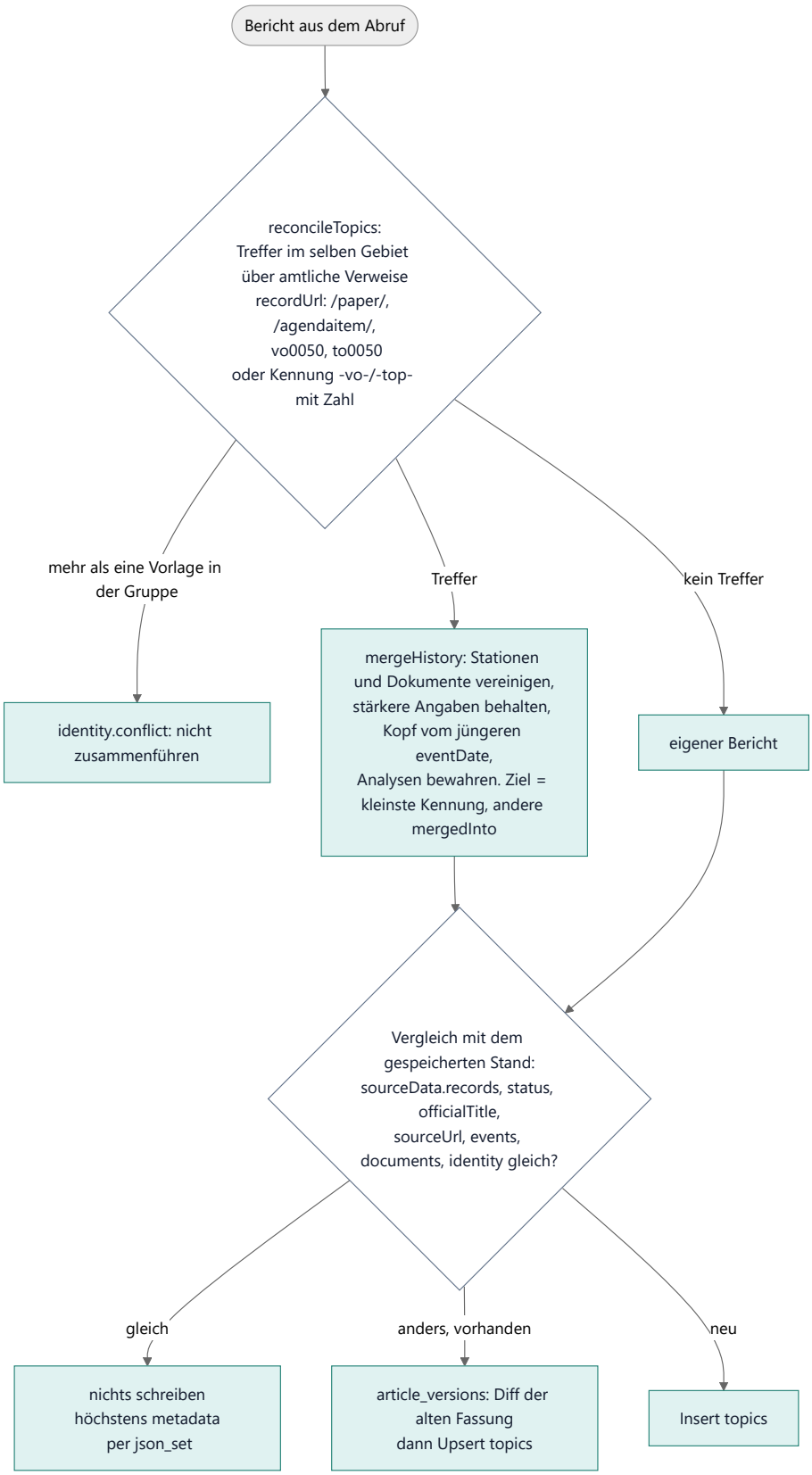
4. Was der Abruf schreibt

Ziel	Inhalt	Wann
`topics`	echte Spalten `id`, `region_id`, `source` (`city/district`, die Ebene, nicht die Quelle), `event_date`, `updated_at`, `status`; `payload` = das ganze `StoredTopic` (Abschnitt 5 im Datenmodell); generiert `active`	Upsert; bei Konflikt nur `event_date`, `updated_at`, `status`, `payload`
`article_versions`	vorige Fassung als Diff (`diff-v1`, `shared/article-version.mjs`, Wiederherstellung `restoreVersions`)	nur wenn sich der Bericht ändert
`source_coverage`	je Gebiet ein JSON: `Coverage` des Lesers zusammengeführt plus `importHealth` (`lastAttemptAt`, `lastSuccessAt`, `lastCompleteAt`, `attemptStatus`, `failureCount`, `nextRetryAt`), `importedAt`, `window`, `oparlProbeAt`, `apiCheck`	jeder Versuch, auch gescheiterte
`import_runs`	`running` zu Beginn, am Ende `completed/partial/failed` mit `details` (mode, region, trigger, window, count …)	jeder Lauf; Läufe älter als 30 Tage werden gelöscht
`system_state`	`import-marks:<gebiet>`, `import-list:<gebiet>`, `import-debug:<gebiet>`, `latest-decision` (Münster)	
Trigger	`search_cards` (+ `search_fts`, `search_cards_gone`), `data_revisions` `content` und `search`, `region_revisions`; `source_coverage` erhöht `coverage`	mit jedem Schreiben
Fehlerspeicher	`work_failures` (`step='import'`, `key=<gebiet>`, `host=provider`), `source_hosts`	über `noteImport`

Reihenfolge in `runSync`: `import_runs` → Batches aus `article_versions` und `topics` → `source_coverage` → Marken und Liste → `latest-decision` → `import_runs` → `import-debug` → `refreshSearchWords` → `rollCardDates` → `refreshRegionFacts` → Sperre frei.

Der Abruf löscht nie. Ein leerer Abruf behält den alten Bestand, ein kürzerer Zeitraum verkürzt die belegte Spanne nicht.

5. Identität, Versionen, Marken



- **Zusammengeführt wird nur über amtliche Datensatz-Verweise** (`shared/topic-identity.mjs`, `IDENTITY_VERSION official-records-v1`), nie über Titel, Drucksachennummer oder PDFs: `recordUrl` erkennt `/paper/`, `/agendaitem/`, `vo0050`, `to0050`; dazu aus Kennungen abgeleitete Verweise (`-vo-`, `-top-` mit rein numerischem Rest). Nur im selben Gebiet.
- **Ziel und Aliase:** Ziel ist die kleinste Kennung der Treffer; die übrigen erhalten `identity.mergedInto` (`active = 0`). `getTopic` folgt `mergedInto`. Mehr als eine Vorlage in der Gruppe ergibt `identity.conflict` ohne Zusammenführung.

- ** `mergeHistory` :** vereinigt Stationen und Dokumente, behält Anwesenheit und Beschluss, wenn der neue Abruf schwächere Angaben liefert, nimmt die Kopfdaten vom Stand mit dem jüngeren `eventDate` ; Analyse- und KI-Felder bleiben über `preserveArticleContent` (`shared/article-record.mjs`) erhalten; ändert sich die Quelle, werden `contentAnalysis` und `labelAssessments.ai` auf `stale` gesetzt.
- **Unverändert, versioniert, neu:** unverändert, wenn `sourceData.records` (ohne Abrufzeitpunkte), `status` , `officialTitle` , `sourceUrl` , `events` , `documents` und `identity*` gleich sind (dann höchstens `metadata` per `json_set`); sonst eine Zeile in `article_versions` und Upsert; neu nur Insert.
- **Marken** (`meeting-marks.mjs` , `import-marks:<gebiet>`): je Sitzung `[Datum, Fingerabdruck, Lesezeit, Phase, Punkte]` . Innerhalb von 6 Stunden „vertraut“ (nicht einmal die Tagesordnung wird neu geholt); sonst gilt eine Marke nur bei unveränderter Phase, nicht innerhalb von 14 Tagen nach der Sitzung, bei einem Alter unter 45 bis 75 Tagen und solange die Berichte noch gespeichert sind. Die gemerkte OParl-Sitzungsliste (`import-list:<gebiet>`) gilt 6 Stunden.
- **Fortsetzbarkeit:** der Leser setzt `coverage.resumable` , wenn das Zeitbudget endete; `runSync` meldet `resume` , der Auftrag reiht das Gebiet erneut ein; vertraute Marken lassen gerade Gelesenes aus.

6. Schutz der Quellen und des eigenen Servers

Mechanismus	Wo	Regel
Sperren	`import-lock.mjs` , `pipeline-lease`	gemeinsame Importsperre plus `import-run:<gebiet>` , 6 Minuten, nach dem Sammeln erneuert; 409 `busy: area|stock` ; der Auftrag selbst unter `pipeline-lease` (30 s)
Zeit	`sync.ts` , `request-budget.mjs` , `--max-minutes`	je Abruf 120 s Sammeln; je Anfrage 20 s bzw. 55 s; Wiederholung nur bei Timeout oder 502 bis 504 und nur mit Restbudget; danach „Zeitbudget“ und fortsetzbar
Abweisung	`refusalGate` , paced in `request-budget.mjs` ; `holds` im Auftrag	nach dem ersten 429 oder einer Firewall-Seite keine Anfrage mehr an die Quelle; danach mindestens 600 ms Abstand; der Auftrag pausiert den Server 60 s bis 1 h, höchstens 3-mal
Last	`PER_PROVIDER` , `robots-policy.mjs`	höchstens 2 gleichzeitige Abrufe je Server
robots.txt	`robots.mjs` , `robots-policy.mjs`	wird vermerkt, nicht befolgt, außer mit `ROBOTS_POLICY=obey` (dann prüft der Website-Leser)
Weiterleitungen, Adressen	`no-redirect.mjs` , `sessionnet.mjs` `allowed` , `body-bytes.mjs`	keine automatische Weiterleitung, eigener User-Agent `VorOrt-PoliticalTopics/0.5` ; SessionNet folgt höchstens 3 Weiterleitungen von Hand auf dieselbe Herkunft; Antworten bis 4 MB
Gesundheit der Quelle	`import-health.mjs` , `source_coverage.nextRetryAt`	Rückstellung 2^n Stunden, höchstens 24 h; wirksam nur bei `trigger === 'scheduled'`
Fehlerspeicher	`noteImport` → `work-failures.mjs`	Grund je Gebiet, Serverstufe je `provider` ; wartende Gebiete und Server stehen im Nachtauftrag als „Zurückgestellt“ (Pipeline, Abschnitt 6)

7. Lokal und auf dem Server

- **Lokal:** Abrufe meist über die Admin-Seite im Browser (`run`); der Dev-Server (`npm run dev`) startet daneben `dev-jobs.mjs` , das alle 15 s `schedule.mjs <job>` aufruft, aber nur bei „jetzt ausführen“ oder offener Arbeit (geplante Zeiten laufen lokal nicht). Der Importprozess läuft lokal nur nach `node scripts/build-node.mjs` mit `DATABASE_FILE=<sqlite> node dist/standalone/worker/import-worker.mjs --create --regions billerbeck --window 1d`. Datenbank: die Miniflare-Datei unter `.wrangler/` .
- **Server:** Jobs `import-nightly` (01:00, `--create`) und `import` (alle 10 Minuten, `--run`), beide über Zeitplaner und systemd (`ratsmonitor-import*.service` , Speicherrahmen 8 GB, höchstens 5 h, `IMPORT_BUSY_TIMEOUT_MS` 30 s, niedrige CPU- und Plattenpriorität). Ein Deployment startet nur den Webdienst neu; ein laufender Importprozess läuft mit seinem alten Stand zu Ende. Vor Migrationen Importprozess und Timer anhalten (`docs/betrieb/node-server.md`).

8. Befunde beim Schreiben dieser Dokumentation (17.10.2026)

Nur gelesen, nicht zur Laufzeit geprüft:

1. **Fenster des Nachtlaufs:** `schedules.mjs` nutzt `--window 1d --weekly-window 1m` ; `docs/betrieb/node-server.md` (Abschnitt Importprozess) und der Kommentar in `ratsmonitor-import-nightly.service` sagen noch „eine Woche zurück“, der Timer-Kommentar nennt alte Uhrzeiten (Sicherung 03:30, Wortliste 04:15).
2. **Exit-Code 2** („pausierter frischer Auftrag“) kann kaum eintreten: `ensureJob` zählt pausierte Aufträge als veraltet und ersetzt sie.
3. ** `RM_PDF_STORE` im Importprozess** hat im Metadaten-Abruf keine Wirkung: der Pfad ruft weder `readDocument` noch `enrichDocument` auf (die nur der Modus `summaries` nutzt). `docs/betrieb/node-server.md` beschreibt, der Import lege jedes gelesene PDF ab; das trifft so nicht zu.

4. **Zwei Schlüsselarten in `source_hosts`:** der Abruf schreibt den Servergruppennamen (`provider`), die PDF-Schritte den Host der Adresse; `deferredRegions` sucht nach `provider(id)`.
5. **`nextRetryAt`** aus `import-health` wirkt nur bei `trigger === 'scheduled'`; Prozess und Admin-Seite rufen mit `'manual'` auf.
6. **`refreshSummaries`** (Modus `summaries` von `runSync`) ist kein Teil des Metadaten-Abrufs: nur von Hand über `POST /api/admin/analyse und nur mit OPENAI_API_KEY (Methode openai-summary-v1, ai-summary.mjs`). Das widerspricht dem Satz in `requirements/ai-processing.md`, es gebe keinen Weg zu automatischen Modellanfragen, sobald der Schlüssel gesetzt ist; in der Vorlage `ratsmonitor.env.example` ist er leer.
7. Ob andere Leser (sim-hannover „Beratungsverlauf", sdnet-rim4 `vorgang`, allris `vo020`) Stationen aus der Beratungsfolge der Vorlage bilden, ist ungeprüft; `MAX_MEETINGS = 400` gilt nur für SessionNet.

Pipeline: Tagespipeline, Extraktion, Verarbeitung, Nacharbeit, Altbestand

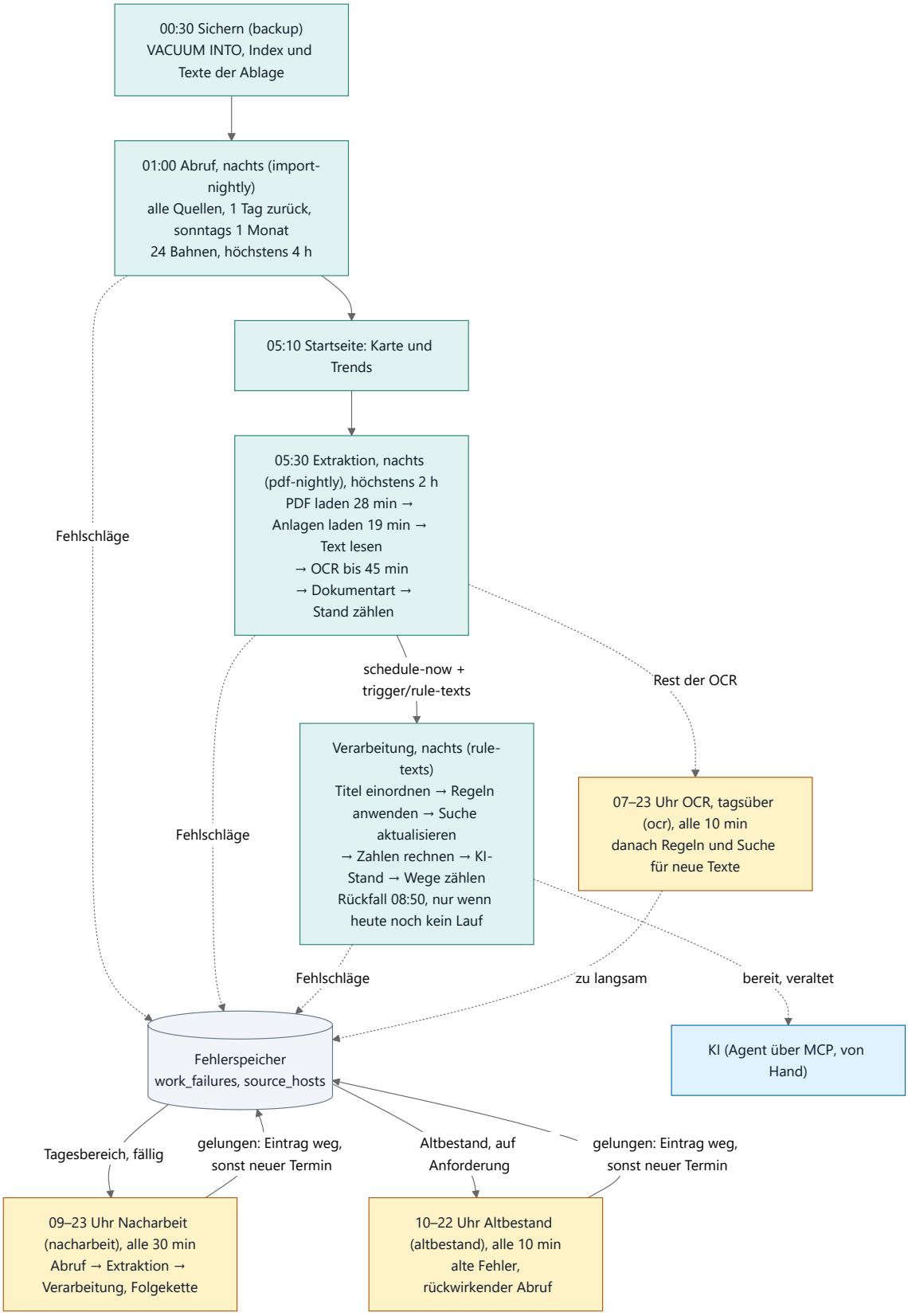
Stand: 17. Oktober 2026, gelesen am Code des Branchs `nachtlauf-fixes`. Teil der technischen Dokumentation. Begriffe: Glossar. Der Abruf selbst (Stufe 1) hat ein eigenes Kapitel: Abruf. Konzept und Begründungen: `requirements/betriebsplan-konzept.md`; interaktives Ablaufbild mit allen Verzweigungen: `requirements/betriebsplan-ablauf.html`; Prozesstapete als PDF: `requirements/betriebsplan-prozesstapete.pdf` (erzeugt von `requirements/betriebsplan-tapete.mjs`).

1. Überblick

Drei **Abläufe**, drei **Stufen** je Bericht, benannte **Schritte** (`shared/pipeline-terms.mjs`; dieselben Wörter stehen in Protokollen, Admin-Seiten und hier):

Ablauf	Wann	Was
Tagespipeline	jede Nacht, feste Reihenfolge	holt und verarbeitet, was neu ist; was scheitert, kommt in den Fehlerspeicher, die Pipeline wartet nicht
Nacharbeit	tagsüber, automatisch, nur Tagesbereich	versucht erneut, was gescheitert ist, Stufe für Stufe, mit Folgekette
Altbestand	nur auf Knopfdruck, 10 bis 22 Uhr	Älteres rückwirkend abrufen, extrahieren, verarbeiten; alte Fehler aufarbeiten

Stufe	Schritte (Schlüssel in STEPS)	Skript
–	Sichern (backup)	deploy/node/backup.sh
Abruf	Abruf (import)	import-worker.mjs → runSync
Extraktion	PDF laden (pdf), Anlagen laden (others), Text lesen (text), OCR (ocr), OCR-Nachladen (ocr-fetch), Dokumentart (doctype), Grenze halten (budget)	scripts/pdf-store/fetch.mjs, text.mjs, budget.mjs
–	Stand zählen (stats)	scripts/pdf-store/stats.mjs
Verarbeitung	Titel einordnen (title), Regeln anwenden (rules), Suche aktualisieren (search), Zahlen rechnen (xcases), KI (ai)	scripts/reclassify-rules.mjs, scripts/pdf-store/read-reports.mjs, scripts/refresh-search-words.mjs, scripts/plenara-x/build-x-cases.mjs, Agent über scripts/mcp-internal.mjs
–	Wege zählen (trace)	scripts/pipeline-trace.mjs



Reihenfolge der Nacht und warum: Sichern vor dem Abruf (ein Stand vor den Änderungen des Tages); Abruf vor der Extraktion (neue Berichte bringen neue PDFs); Text und OCR vor der Verarbeitung (die Regeln haben morgens den ganzen Text); Regeln vor Suche und Zahlen (Zusammenfassungen ändern den Suchtext).

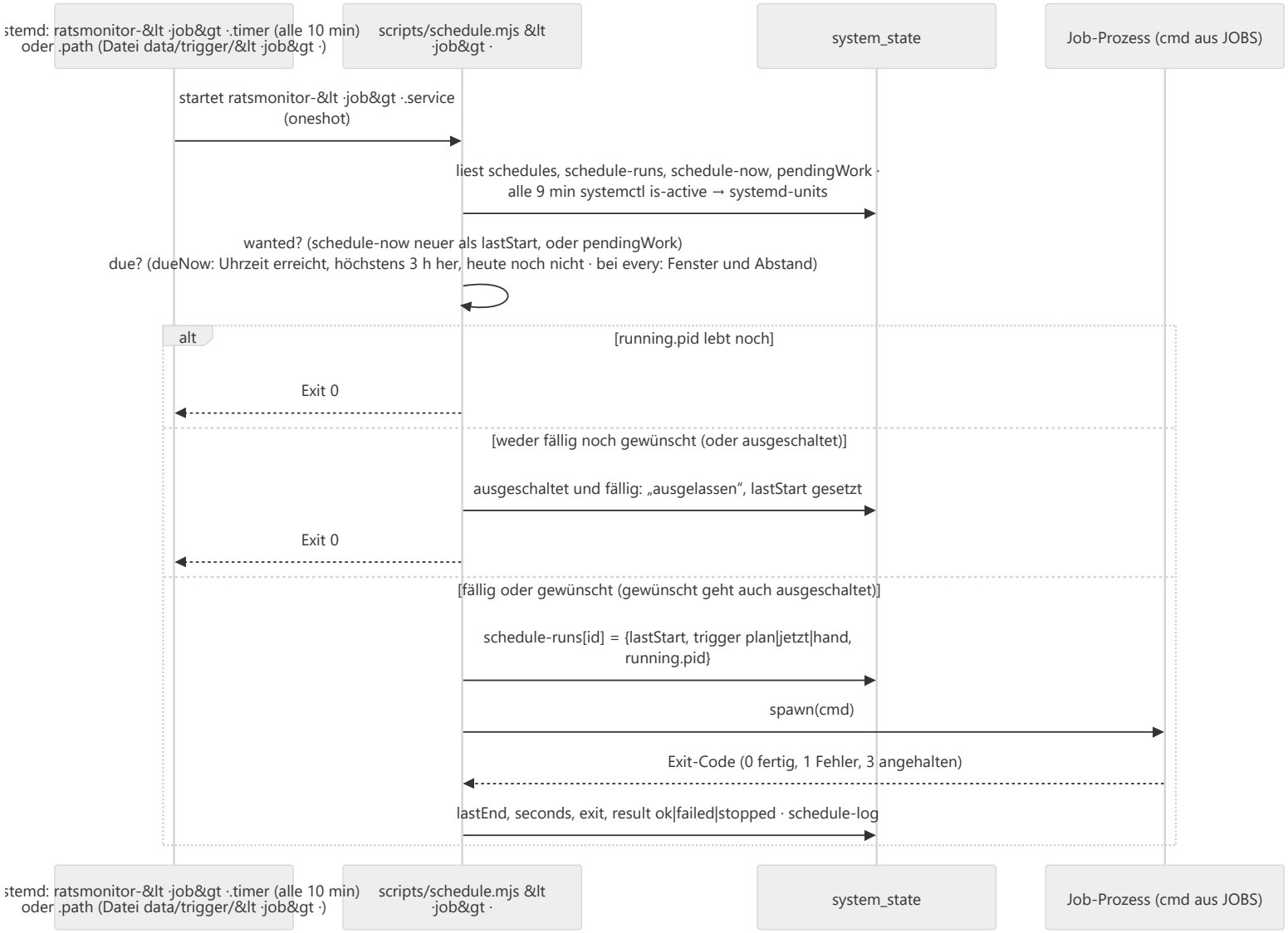
2. Jobs und Zeitplaner

Die Jobs (`JOBS` in `server/integrations/schedules.mjs`)

Job (id)	Name im Admin	Ablauf	Zeit / Fenster	Grenze	Befehl
accounts	Konten: Pflege und Sicherung	–	00:10	Sekunden	bash deploy/node/accounts-job.sh
backup	Sichern	Tagespipeline	00:30	Platz 1,5 × Datenbank	bash deploy/node/backup.sh
import-nightly	Abruf, nachts	Tagespipeline	01:00	4 h	import-worker.mjs --create sources --window 1d --weekly-window 1m --weekly-day 0 --max-minutes 240
startseite-trends	Startseite: Karte und Trends	Tagespipeline	05:10	Minuten	scripts/startseite-trends.mjs
pdf-nightly	Extraktion, nachts	Tagespipeline	05:30	2 h	scripts/pdf-store/nightly.mjs --kind nightly --daily --minutes 120 --budget-gb 80 --ocr-workers 2
rule-texts	Verarbeitung, nachts	Tagespipeline	direkt nach pdf-nightly; 08:50 nur als Rückfall (fallback: nicht, wenn heute schon gelaufen)	meist unter 1 h	scripts/rule-texts.mjs
nacharbeit	Nacharbeit	Nacharbeit	alle 30 min, 09:00 bis 23:00	20 min, 500 Einträge	scripts/nacharbeit.mjs --range daily --minutes 20
ocr	OCR, tagsüber	Nacharbeit	alle 10 min, 07:00 bis 23:00	3 h	nightly.mjs --kind ocr --ocr-only --range daily --minutes 180 --ocr-workers 2 --until 23:00
altbestand	Altbestand	Altbestand	alle 10 min, 10:00 bis 22:00, nur mit Anforderung	2 h	scripts/altbestand.mjs --minutes 120 --until 22:00
import	Abrufaufträge	Altbestand	alle 10 min, nur mit Auftrag	4 h	import-worker.mjs --run --max-minutes 240
pdf-job	PDF-Aufträge und Messungen	Altbestand	alle 10 min; Aufträge 10:00 bis 21:30	3 h je Auftrag, Dienst bis 12 h	scripts/pdf-store/job.mjs --minutes 180 --budget-gb 80

Einstellungen (ein/aus, Uhrzeit in 10-Minuten-Schritten deutscher Zeit, Abstand) liegen in `system_state` `schedules`; Läufe in `schedule-runs` (Start, Ende, Dauer, Ergebnis, laufende PID), Laufbuch `schedule-log` (höchstens 400), Wünsche `schedule-now`, Anhalten `schedule-stop`, Zustand der systemd-Einheiten `systemd-units`.

Wie ein Job startet


stemd: ratsmonitor-&lt ·job&gt ·.timer (alle 10 min) oder .path (Datei data/trigger/&lt ·job&gt ·)
scripts/schedule.mjs &lt ·job&gt ·
system_state
Job-Prozess (cmd aus JOBS)
startet ratsmonitor-&lt ·job&gt ·.service (oneshot)
liest schedules, schedule-runs, schedule-now, pendingWork · alle 9 min systemctl is-active → systemd-units
wanted? (schedule-now neuer als lastStart, oder pendingWork)
due? (dueNow: Uhrzeit erreicht, höchstens 3 h her, heute noch nicht · bei every: Fenster und Abstand)
alt
[running.pid lebt noch]
Exit 0
[weder fällig noch gewünscht (oder ausgeschaltet)]
ausgeschaltet und fällig: „ausgelassen", lastStart gesetzt
Exit 0
[fällig oder gewünscht (gewünscht geht auch ausgeschaltet)]
schedule-runs[id] = {lastStart, trigger plan|jetzt|hand, running.pid}
spawn(cmd)
Exit-Code (0 fertig, 1 Fehler, 3 angehalten)
lastEnd, seconds, exit, result ok|failed|stopped · schedule-log
stemd: ratsmonitor-&lt ·job&gt ·.timer (alle 10 min) oder .path (Datei data/trigger/&lt ·job&gt ·)
scripts/schedule.mjs &lt ·job&gt ·
system_state
Job-Prozess (cmd aus JOBS)

- **„Jetzt ausführen“** (`POST /api/admin/zeitplaene {action:'now'}`): `runScheduleNow` setzt `schedule-now[id]`, `triggerJob(id)` schreibt die Datei `<RM_TRIGGER_DIR | trigger neben DATABASE_FILE>/<job>`; `ratsmonitor-<job>.path` (`PathModified`) startet den Dienst sofort. Läuft der Dienst schon, passiert nichts. `pendingWork` zählt ebenfalls als Wunsch: beim `pdf-job` eine offene Zählung, Speichermessung oder ein wartender Auftrag, beim `altbestand` eine offene Anforderung im Fenster.
- **„Anhalten“**: `requestScheduleStop` schreibt `schedule-stop[id]` (nur Anzeige „wird angehalten“) und `triggerStop(id)` die Datei `trigger/stop-<job>`; `ratsmonitor-stop-<job>.path` startet `ratsmonitor-stop@<job>.service` (root), das `systemctl stop ratsmonitor-<job>.service` `ausführt.` schedule.mjs `gibt SIGTERM an das Kind weiter und vermerkt` stopped`. Jobs nie mit `systemctl stop …timer` stoppen: der Job startet dann nie wieder (Befund 16.10.2026); die Debug-Seite und der Nachtbericht warnen bei inaktiven Einheiten.
- Laufeinträge schreibt der Zeitplaner mit bis zu 6 weiteren Versuchen à 10 s, weil die Verarbeitung die Schreibsperre länger als `busy_timeout` halten kann.

3. Die Tagespipeline Schritt für Schritt

3.1 Sichern (`backup`, 00:30)

`deploy/node/backup.sh`: älteste Sicherung löschen (nie mehr als 2 Stände je Art), Datenbank per `VACUUM INTO` kopieren (die Website läuft weiter), prüfen, mit zstd packen; dazu `index.sqlite` der Ablage (`.backup`) und ihre Textfassungen als `tar.zst` (sonntags voll, sonst nur seit der letzten Sicherung geänderte); mit `BACKUP_REMOTE` (rclone) eine Kopie außer Haus. PDFs werden nicht gesichert, sie lassen sich neu laden. Bei Fehler bleibt die vorige Sicherung.

3.2 Abruf, nachts (`import-nightly`, 01:00)

`import-worker.mjs --create sources --window 1d --weekly-window 1m --weekly-day 0`: ein Abrufauftrag über alle angebundenen Quellen, 1 Tag zurück plus die schon angesetzten künftigen Sitzungen; sonntags einen Monat zurück, damit Niederschriften und Beschlüsse, die Wochen nach der Sitzung erscheinen, ankommen. Gebiete mit wartendem Eintrag im Fehlerspeicher und Gebiete auf wartenden Servern stehen im Auftrag als „Zurückgestellt“ (`deferredRegions`); die Nacharbeit holt sie. 24 Bahnen, je Server höchstens 2. Jeder Ausgang geht über `noteImport` (`pipeline-jobs.mjs`) in den Fehlerspeicher: Erfolg löscht den Eintrag und setzt den Server auf „liefert“; ein teilweiser Abruf mit Netzfehler steht als „vorübergehend, teilweise: …“ ohne Server. Der Nachtauftrag trägt `owner: nightly`; der Job „Abrufaufträge“ lässt ihn aus, solange er sich bewegt (`nightlyBusy`, 10 Minuten ohne Stand). Nach jedem Gebiet: Wortliste, Datum „Stand heute“ und Werte der Adminseiten nachführen. Einzelheiten: Abruf.

3.3 Startseite: Karte und Trends (`startseite-trends`, 05:10)

Rechnet je Beispielbegriff der Startseite die Ausbreitung wie `/api/analytics/diffusion` und legt sie in `system_state` `startseite-trends` ab (`/api/startseite-trends`). Ohne Eintrag zeigt die Startseite den mitgelieferten Stand `public/data/startseite-trends.json`.

3.4 Extraktion, nachts (`pdf-nightly`, 05:30, `nightly.mjs --daily`)

Sperre `nightly.lock` (Bahn „Laden“ der Ablage). Zeit 120 min, `--fetch-share 0.4`, Reserve 25 min. Jeder Schritt ist ein Kindprozess (`--max-old-space-size=4096`); scheitert einer, laufen die übrigen weiter, der Lauf endet dann mit Exit 1.

1. **Stichtag:** `since = pdf-daily-last.startedAt` (ohne Eintrag: 48 h zurück), `boundary = dailyBoundary` (7 Tage, frühestens 12.10.2026).
2. **PDF laden:** `fetch.mjs --since-updated <since> --event-from <boundary> --origin daily --budget-gb 80 --max-minutes 28`: Hauptdokumente aller Berichte, die seit `since` geschrieben wurden (`updated_at`), und aller Berichte mit Sitzung ab `boundary`. Zwei Abfragen, beide über Indizes. Wartende Server werden ausgelassen, fällige bekommen eine Stichprobe von 3.
3. **Anlagen laden:** dasselbe mit `--others`, höchstens 19 min. In der Grenze stehen Anlagen unter allen Hauptdokumenten.
4. **Grenze halten:** nur wenn die Ablage über 80 GB liegt: `budget.mjs --backfill`, dann `budget.mjs --budget-gb 80 --apply`.
5. **Text lesen:** `text.mjs --max-minutes <Rest − 25>`: Textebene aller abgelegten PDFs ohne Text (nicht nur die dieses Laufs).
6. **OCR:** die Dateien, deren Text in diesem Lauf entstand und Seiten ohne Text hat, Hauptdokumente zuerst, dann wenigste Seiten: `text.mjs --ocr --shas … --origin daily --workers 2 --file-minutes 5 --max-minutes min(45, Rest − 15)`. Entfällt bei weniger als 5 min Rest. Der Rest kommt im Job „OCR, tagsüber“ dran.
7. **Dokumentart:** `budget.mjs --backfill` (`texts.head_type` nachtragen).
8. **Stand zählen:** `stats.mjs --budget-gb 80` → `pdf-store-stats`, Fehlerliste als CSV.

9. Wenn nicht angehalten: `pdf-daily-last = {startedAt, finishedAt, failed}`.
10. **Verarbeitung anstoßen**, wenn `rule-texts` eingeschaltet ist: `schedule-now['rule-texts']` setzen und `triggerJob('rule-texts')`.

Fortschritt: `pdf-store-progress-chain` vor jedem Schritt und minütlich aus den Schritten; Protokoll des Laufs in `pdf-store-runs` (höchstens 50). Anhalten: `pdf-store-stop.chain` (alle 2 s geprüft) oder SIGTERM → SIGTERM an den Schritt, nach 60 s SIGKILL, Lauf „angehalten“, Exit 3.

3.5 Verarbeitung, nachts (`rule-texts`, direkt danach, `scripts/rule-texts.mjs`)

Jeder Schritt läuft als Kindprozess; Schritte mit `fatal:false` beenden den Lauf bei Fehler nicht. Ergebnis je Schritt in `system_state` `rule-texts-last` (`{startedAt, finishedAt, result, all, steps[{name, seconds, ok, fatal, summary}], facts, warnings}`), Anzeige im Reiter „Themen & Texte“. Der Wunsch „Alle neu erstellen“ (`rule-texts-all`) wird beim Start gelesen und gelöscht.

Nr.	Schritt	Was	Aufruf
0	Titel einordnen	Titelregel für Berichte mit fehlender oder veralteter `classification` (Pakete zu 500 unter `import-lock`): schreibt `payload.classification`, `labelAssessments.rule`, `analysisFeatures` und `article_analyses` (`rule-label`)	`scripts/reclassify-rules.mjs` → `analysePending` (`manual-analysis.mjs`)
1	Regeln anwenden	Regeln je Bericht für alle aktiven Berichte, deren Eingaben sich geändert haben (Abschnitt 5); bei „alle neu“ `--all`	`scripts/pdf-store/read-reports.mjs`
–	Fakten zählen	Berichte mit Fakten, davon aus Text, veraltete Fassung	SQL im Skript
2	Suche aktualisieren	`rollCardDates`, dann Wortliste nachführen; bei „alle neu“ `--full`	`scripts/refresh-search-words.mjs --quiet`
3	Zahlen rechnen	`x_cases`, `x_signals` bauen, warmhalten, Zählung je Thema, Unterthema, Fakt, `factCoverage`	`scripts/plenara-x/build-x-cases.mjs`
4	KI-Stand zählen	Zustände der Arbeitsposten, Berichte mit neuem Text seit der KI-Fassung, ohne KI-Fassung → `ai-counts`	im Skript
5	Wege zählen	Pipeline-Debug (Abschnitt 10), nicht fatal	`scripts/pipeline-trace.mjs`

Exit 0 fertig, 1 Fehler, 3 angehalten. Stürzt eine Regel bei einem Bericht ab, wird er als „Fehler im eigenen Code“ (`rules`, `internal`) vermerkt und der Lauf geht weiter.

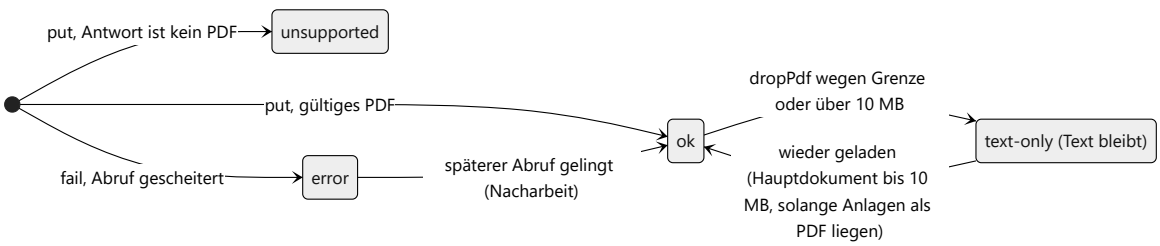
4. Extraktion im Detail: die PDF-Ablage

Nur auf dem Node-Server (`RM_PDF_STORE`, Standard `tmp/pdf-store`, Server `/srv/ratsmonitor/data/pdf-store`); `server/node/pdf-store.mjs` öffnet sie, `scripts/pdf-store/store-lib.mjs` enthält die gemeinsamen Teile der Skripte. Der Server erzeugt alles aus seinen eigenen PDFs, übertragen wird nichts.

Speicher

- `index.sqlite` (WAL, `busy_timeout` 30 s): `pdfs` (eine Zeile je **Adresse**: `url`, `sha256`, `bytes`, `pages`, `status`, `http`, `error`, `fetched_at`, `region_id`, `topic_id`, `doc_type`, `formal`, `event_date`, `role`), `texts` (eine Zeile je **Datei**: `sha256`, `status`, `method` text/ocr/mixed, `version`, `pages`, `ocr_pages`, `ocr_needed`, `chars`, `error`, `made_at`, `head_type`), `meta` (`doc-type-version`, `failures-seeded`), `text_hashes`, `facts_keys` (Altlast, ersetzt durch `report_rules`), `ocr_refetch` und `ocr_slow` (Altlasten, einmal in den Fehlerspeicher übertragen).
- `files/<ab>/<sha256>.pdf` und `.txt` (Seiten getrennt durch `\f`); gleiche Bytes unter mehreren Adressen liegen einmal. Geschrieben wird als `.tmp`, dann umbenannt.
- Sperren und Hilfsdateien im Ordner: `nightly.lock` (Bahn Laden), `ocr-lane.lock` (Bahn OCR), `ocr.lock` (OCR-Treiber), `ocr-beat-<pid>.json`, `step-summary-<pid>.json` u. a.

Status einer Adresse und Rolle



- `pdfs.role`: `'other'` = Anlage (übriges PDF), leer = Hauptdokument. Welches PDF Hauptdokument ist, entscheidet `primaryPdf` (`shared/document-type.mjs`): nur PDFs, keine Unterlage an 5 und mehr Berichten, bevorzugt mit Text, Rang nach Dokumentart.
- `wants(url,{retryDays,textOnlyBelow})` entscheidet, ob eine Adresse geladen wird: ohne Eintrag ja; `ok`/`unsupported` nein; `error` erst nach `retryDays` (mit Fehlerspeicher nie, das macht die Nacharbeit); `text-only` nur Hauptdokumente bis 10 MB, solange noch Anlagen als PDF liegen.

Score und feste Grenze

Score 0 bis 1 je Datei (`shared/pdf-rank.mjs`) = Dokumentart (Vorlage, Beschluss 1; Niederschrift 0,7; Anlage, Sonstiges 0,4; Einladung 0,1) × Formalie (0,5, nicht bei Niederschriften) × Inhalt (Zeichen plus 800 je OCR-Seite gegen 4.000, mit der Wurzel) × Größe (voll bis 100 KB, darüber mit der Kubikwurzel). Über 10 MB: Score 0, nur Text. Anlagen stehen immer unter allen Hauptdokumenten (`order = score − 1`).

Grenze `BUDGET_GB` = 80 für PDFs und Texte zusammen (`keeper` in `store-lib.mjs`): jede neue Datei wird sofort bewertet (Textebene gleich gelesen); ist die Grenze erreicht, verdrängt sie die schlechteste behaltene Datei, wenn sie besser ist, sonst fliegt sie selbst raus. Vor dem Löschen nur die Textebene, keine OCR. Ist der Lauf angehalten, wird nichts mehr verdrängt. `budget.mjs --apply` ist nur das Sicherheitsnetz und löscht nur Dateien, die schon eine Textfassung haben.

PDF laden (`fetch.mjs`)

- Abruf über `fetchDocument` (`document-proxy.mjs`): Kennung `VorOrt-PoliticalTopics/0.5 (public council information)`, nur https und öffentliche Ziele, Weiterleitungen von Hand geprüft, 10 s für Kopf und Inhalt, über 30 MB „Dokument zu groß“. Standard 32 Abrufe gleichzeitig, 3 je Server, 200 ms Abstand.
- Was gesammelt wird: je Bericht das Hauptdokument; mit `--others` die Anlagen; mit `--urls` die Adressen aus einer Datei (Nacharbeit); mit `--since-updated`/`--event-from` der Tagesbereich; sonst Monat für Monat zurück (`--months`, Aufträge des Altbestands).
- Server: wartende werden ausgelassen, fällige bekommen 3 Adressen als Stichprobe, der Rest wartet in `held` und folgt nach dem ersten Erfolg. Liefert ein Server 429 oder 503 oder 3 Fehler in Folge, wird seine Reihe für den Lauf geleert.
- Fehlerspeicher: Fehler und `unsupported` als Schritt `pdf`; Erfolg löscht den Eintrag und setzt einen wartenden Server auf ok.
- Anhalten bricht laufende Abrufe ab; sie zählen nicht als Fehler. Am Ende eine Zusammenfassung (`stepSummary`).

Text lesen und OCR (`text.mjs`)

- **Textebene:** `unpdf`/pdf.js `getTextContent`; eine Seite „braucht OCR“ bei weniger als 20 Zeichen ohne Leerraum. Gespeichert mit `method:'text'`, `version: TEXT_VERSION` (1; eine neue Version liest alles neu). Defekte PDFs als `text`/`internal` in den Fehlerspeicher.
- **OCR:** tesseract.js und `@napi-rs/canvas` aus `OCR_TOOLS`, Sprachdaten in `OCR_CACHE`, Sprache `deu`, Maßstab bis 2,5, höchstens 12 Mio. Pixel je Seite. Nur Seiten, die OCR brauchen; am Ende `method` ocr oder mixed, `ocr_needed=0`.
- **Reihe** `ocrQueue`: Dateien mit `ocr_needed>0` (Status `ok` oder `text-only`); Reihenfolge Hauptdokument vor Anlage, abgelegte vor verdrängten, neuester Sitzungstag zuerst, dann wenigste OCR-Seiten. `text.mjs` nimmt heraus, was im Fehlerspeicher wartet (`ocr`/`ocr-fetch`; `slow` nur mit `--include-slow`), filtert `--shas` und `--range daily|backlog` (`ocrRange`: Sitzungstag, sonst Ladetag, gegen `dailyBoundary`).
- **Treiber und Pakete:** der Elternprozess hält `ocr.lock` (läuft schon einer, endet er sofort als „ausgelassen“) und startet Pakete zu 25 Dateien als Kindprozesse (`--max-old-space-size=2048`, RSS-Grenze 2,5 GB). Ein **Wächter** beendet ein Paket ohne Lebenszeichen (`ocr-beat-<pid>.json`) nach 180 s mit SIGKILL; seine laufenden Dateien gelten als zu langsam. Ein Paket ohne Fortschritt beendet den Lauf. Grenzen: 90 s je Seitengruppe, `--file-minutes` je Datei (5; Nacharbeit 15). 2 Worker in den Jobs.
- **Verdrängte Dateien** (`text-only`) lädt `ocrWithRefetch` kurz nach (gleiche Prüfsumme), liest sie und löscht sie gleich wieder; Nachladefehler stehen als `ocr-fetch` im Fehlerspeicher, Zeitüberschreitung als `ocr`/`slow`. Ein Server mit 3 Fehlern in Folge oder wartend im Fehlerspeicher wird im Paket ausgelassen.

Dokumentart (`budget.mjs --backfill`)

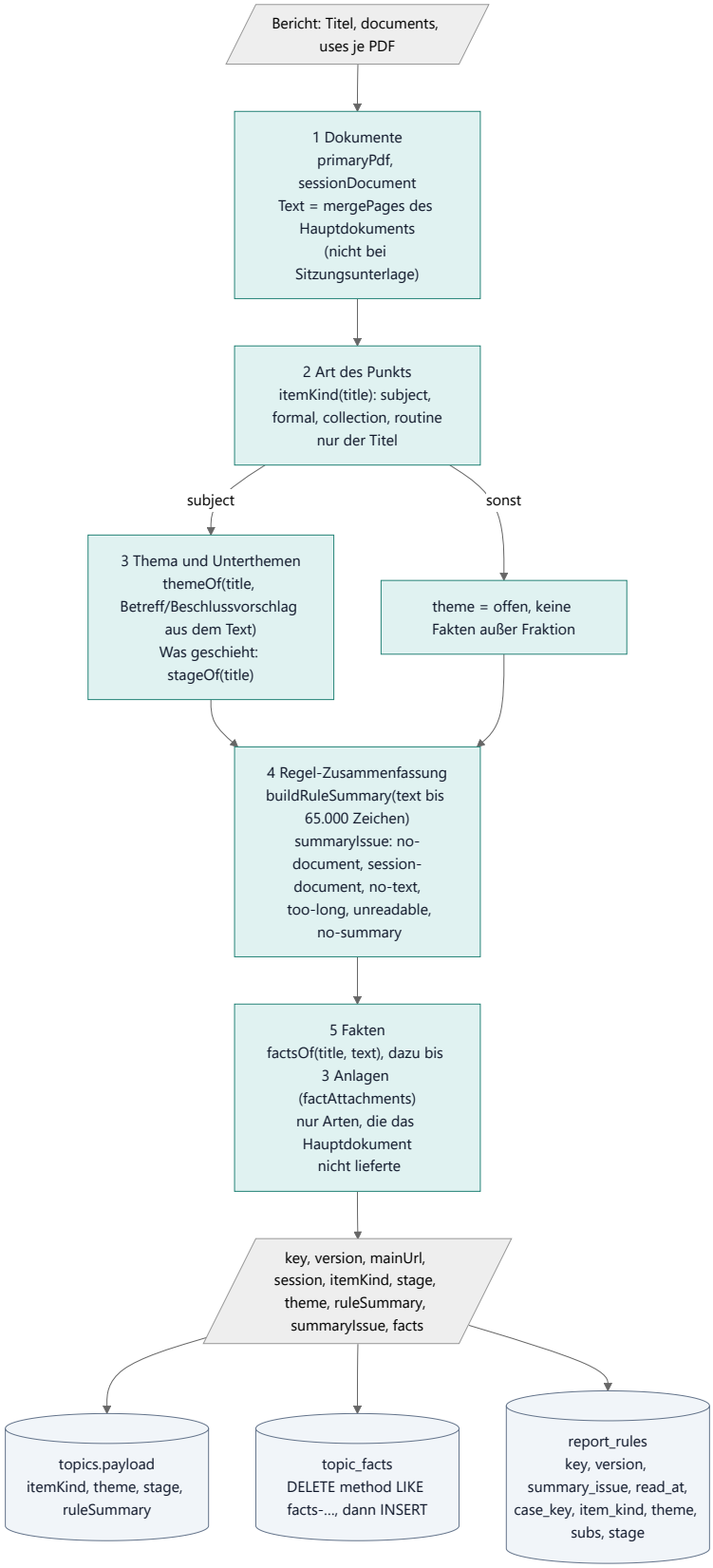
`documentType(title)` aus dem ersten Wort des Titels, berichtigt durch `documentTypeFromText` über die ersten 600 Zeichen (`texts.head_type`; `''` ohne Merkmal, `NULL` noch nicht bestimmt). Bei neuer `DOC_TYPE_VERSION` (6) werden alle Werte neu geschrieben, sonst nur fehlende.

Wie Abruf und KI die Ablage nutzen

`import-worker.mjs` schaltet mit `useDocumentStore(openPdfStore(RM_PDF_STORE))` den Lesezugriff ein; `readDocument` (`documents.mjs`) liest erst die Textfassung (`getText`, über 150 Seiten Fehler, sonst `mergePages` bis 65.000 Zeichen), dann das PDF (`get`), sonst aus dem Netz (30 s, 12 MB) und legt es mit `put` ab, ohne Gebiet und Rang. Der KI-Stapel (`scripts/ai-batch/lib.mjs` `fetchSource`) geht denselben Weg.

5. Verarbeitung im Detail: Regeln je Bericht

`readReport` (`server/integrations/report-rules.mjs`) ist eine reine Funktion: feste Reihenfolge, jede Regel liest nur Quellen (Titel, Dokumentliste, Text des Hauptdokuments, passende Anlagen), nie das Ergebnis einer anderen. Plan: `requirements/regel-schritt-plan.md`.



- **Schlüssel:** `readKey` = Hash über `READ_VERSION`, Titel, Adresse des Hauptdokuments, Sitzungsunterlage ja/nein und Textstand des Hauptdokuments und der Anlagen. `read-reports.mjs` liest nur Berichte, deren Schlüssel vom gespeicherten in `report_rules.key` abweicht; `--all` verwirft alle Schlüssel, `--ids <datei>` liest die genannten immer (Nacharbeit, OCR tagsüber).
- **Versionen:** `READ_VERSION` = `read-v4:doc6:item-kind-v4:themes-v3:stage-v4:sum3:facts-v27`, zusammengesetzt aus `DOC_TYPE_VERSION`, `ITEM_KIND_VERSION`, `THEMES_VERSION`, `STAGE_VERSION`, `RULE_SUMMARY_VERSION`, `FACTS_VERSION`. Jede Änderung liest beim nächsten Job alle Berichte neu (Server etwa 40 Minuten für 1,5 Mio. Berichte). `THEMES_VERSION` steht

zusätzlich fest im View `topic_effective_labels`: ein Wechsel braucht eine neue Migration. Die Titelregel (`LABEL_VERSION`, `CLASSIFIER_VERSION`) gehört nicht dazu; sie läuft als Schritt „Titel einordnen“.

- **Schreiben:** Pakete zu 500 in `BEGIN IMMEDIATE`; Payload nur bei Änderung; eine alte `ruleSummary` wird gelöscht, außer bei `no-text`, `too-long`, `unreadable`; `topic_facts` mit `method = 'facts-v27:text|title'`; am Ende Datenstand `content` + 1. KI-Felder (`shortSummary`, `longSummary`, `labelAssessments.ai`, `contentAnalysis`) bleiben unberührt.
- **Vorgang:** `case_key = region_id + '|' + titleKey(coalesce(officialTitle, title))`; gleiche Titel je Gebiet sind ein Vorgang.
- **Auch der Modus `summaries`** von `runSync` (`refreshSummaries` in `sync.ts`: nur von Hand über `POST /api/admin/analyse` und nur mit `OPENAI_API_KEY`, nicht der nächtliche Abruf) ruft `readReport` mit dem dort gelesenen Text und schreibt `itemKind`, `theme`, `stage` und `topic_facts`, aber nicht `report_rules`; der Job liest diese Berichte später noch einmal.
- **Regel-Zusammenfassung** (`rule-summary.mjs`): nur Originalsätze, nichts wird umformuliert. Der Text wird in Beschluss, Sachverhalt und Ende geteilt; gewählt werden bis zu 2 Beschlusssätze, eine Einleitung, ein Satz mit Betrag, ein Begründungssatz, dann weitere nach Gewicht. Kurz = ein Satz (Ziel 30, höchstens 40 Wörter), lang = Ziel 100, höchstens 140 Wörter, in Dokumentreihenfolge mit Lücken „[…]“. Güte 0 bis 100 aus Struktur, Einleitung, Titelabdeckung, Satzqualität, Länge, Lesbarkeit, Betrag; gespeichert nur, wenn beide Fassungen existieren und ihr Mittel mindestens 60 erreicht. Keine Zusammenfassung bei persönlichen Schreiben, Niederschriften oder fehlender Struktur.
- **Qualität:** `shared/rule-quality.json` (von Hand nach jeder Messung, ehrliche Zahl einer frischen Server-Stichprobe), Belege in `docs/qualitaet/plenara-x/regeln-2026-10-14.md`, Anzeige im Reiter „Themen & Texte“.

Was danach aus den Ergebnissen wird: Suchkarten und Wortliste (Suche), Vorgänge und Signale (Auswertungen).

6. Fehlerspeicher

Migration `drizzle/0040`, Modul `server/integrations/work-failures.mjs`. Ohne die Tabellen tun alle Funktionen nichts.

`work_failures` (ein Eintrag je Schritt und Gegenstand; ein neuer Fehlschlag ersetzt den alten mit `tries + 1`, `first_at` bleibt):

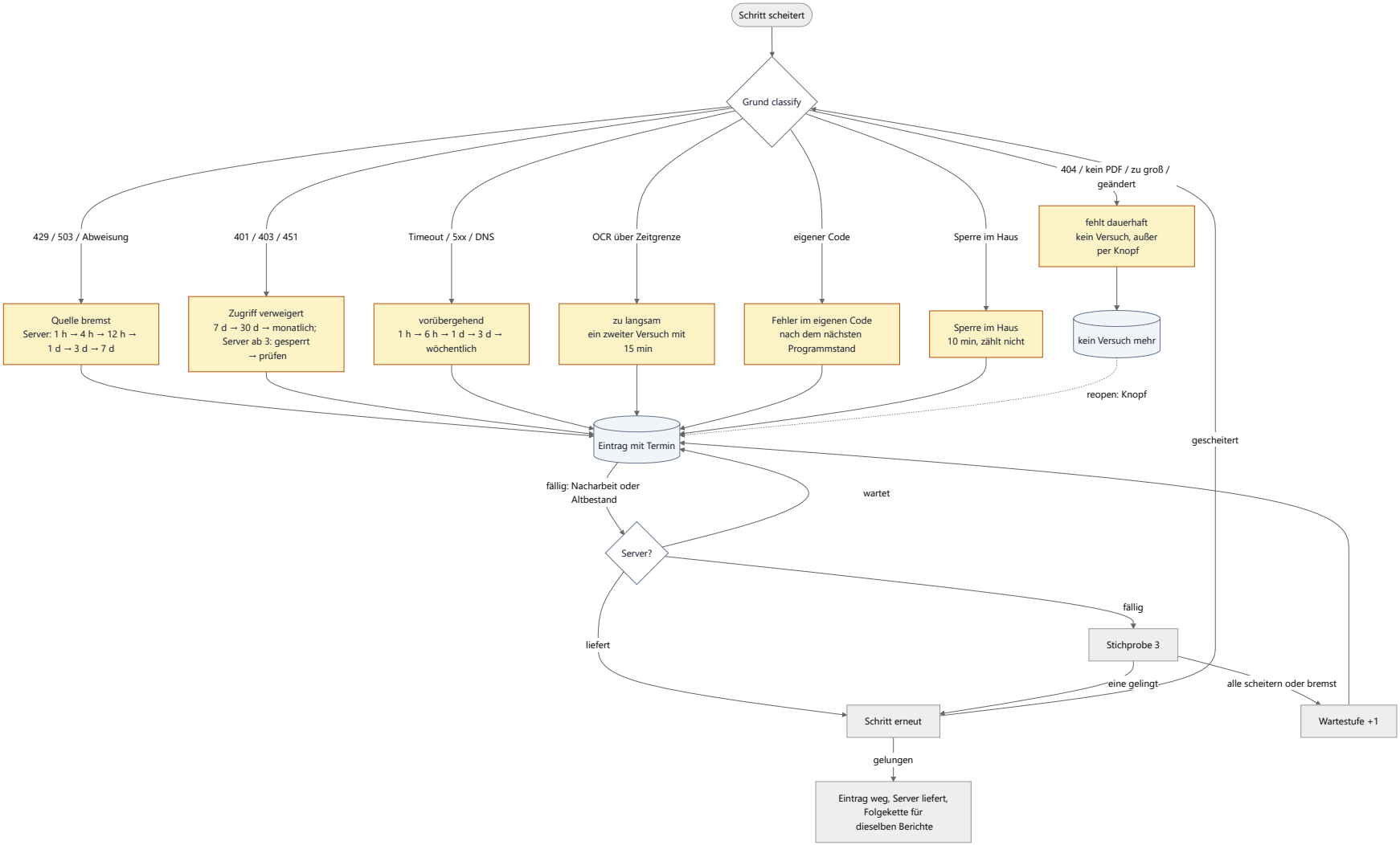
Spalte	Inhalt
`step`, `key`	`import` (Gebiet), `pdf` (Adresse), `text`, `ocr`, `ocr-fetch` (Prüfsumme), `rules` (Bericht); Stufe über `PHASES`
`host`	Server der Quelle (für die Serverstufe)
`class`, `reason`, `http`	Grund, Meldung, HTTP-Code
`tries`, `first_at`, `last_at`, `next_try_at`	Versuche, erster und letzter Fehlschlag, nächster Termin (NULL: kein automatischer Versuch)
`event_date`, `origin`	Sitzungstag; Herkunft `daily` oder `backlog`
`release`	Programmstand bei „Fehler im eigenen Code“

`source_hosts` (je Server): `state` (`ok`, `bremsen`, `gesperrt`, `pruefen`), `fails` (zählende Fehler in Folge), `level` (Wartestufe), `next_try_at`, `last_ok_at`, `last_fail_at`, `reason`.

Gründe und Wiedervorlage (`classify`, `nextTry`, `hostFailed`)

Grund (`class`)	Erkennung (Reihenfolge der Prüfung)	nächster Versuch des Eintrags	Wirkung auf den Server
zu langsam (`slow`)	vom Aufrufer gesetzt (OCR über 90 s je Seite oder über der Dateigrenze)	1. Mal sofort, ab dem 2. kein Versuch (nur `reopen`)	keine
Sperre im Haus (`busy`)	409, „database is locked“, „Import läuft bereits“	+10 min, `tries` steigt nicht	keine
Quelle bremst (`blocked`)	`refused`, 429, 503, Retry-After, Abweisungsseite	Termin des Servers, sonst +1 h	zählt; sofort `bremsen`: 1 h, 4 h, 12 h, 1 d, 3 d, 7 d; ein längeres Retry-After geht vor (60 s bis 7 d)
fehlt dauerhaft (`gone`)	`changed`, `unsupported`, 404, 410, „zu groß“, „kein PDF“, übrige 4xx	kein Versuch; wieder offen per Knopf oder bei neuer Adresse	keine
Zugriff verweigert (`denied`)	401, 403, 451	7 d, 30 d, danach alle 30 d	zählt; ab 3 in Folge `gesperrt`: 7 d, 30 d, dann `pruefen` (Stichprobe alle 30 d)
vorübergehend (`transient`)	5xx außer 503, Zeitüberschreitung, DNS, Verbindung, TLS, „unvollständig“; alles Unpassende	1 h, 6 h, 1 d, 3 d, danach wöchentlich	zählt; ab 3 in Folge `bremsen`

Grund (`class`)	Erkennung (Reihenfolge der Prüfung)	nächster Versuch des Eintrags	Wirkung auf den Server
Fehler im eigenen Code (`internal`)	Ausnahme in Parser, Regel oder Leser, defektes PDF	fällig, sobald `release` nicht mehr der laufende Programmstand ist	keine


Schritt scheitert
Grund classify
429 / 503 / Abweisung
401 / 403 / 451
Timeout / 5xx / DNS
OCR über Zeitgrenze
eigener Code
Sperre im Haus
404 / kein PDF / zu groß / geändert
Quelle bremst
Server: 1 h → 4 h → 12 h → 1 d → 3 d → 7 d
Zugriff verweigert
7 d → 30 d → monatlich; Server ab 3: gesperrt → prüfen
vorübergehend
1 h → 6 h → 1 d → 3 d → wöchentlich
zu langsam
ein zweiter Versuch mit 15 min
Fehler im eigenen Code
nach dem nächsten Programmstand
Sperre im Haus
10 min, zählt nicht
fehlt dauerhaft
kein Versuch, außer per Knopf
kein Versuch mehr
reopen: Knopf
Eintrag mit Termin
gescheitert
fällig: Nacharbeit oder Altbestand
wartet
Server?
fällig
liefert
Stichprobe 3
eine gelingt
alle scheitern oder bremst
Schritt erneut
Wartestufe +1
gelungen
Eintrag weg, Server liefert, Folgekette für dieselben Berichte

- **Serverstufe:** `hostWaiting` (nicht ok, Termin in der Zukunft oder ohne Termin): alle Abrufer lassen den Server aus, seine Einträge bleiben offen und zählen keine Versuche. `hostDue` (Termin erreicht): Stichprobe von `SAMPLE` = 3 Einträgen; gelingt einer, liefert der Server wieder (`hostOk` setzt alle Zähler zurück); scheitern drei oder bremst er, steigt die Wartestufe. „Freigeben" (Seiten Pipeline-Debug und Altbestand, `releaseHost`) zieht die Prüfung vor.
- **Tagesbereich:** `DAILY_SINCE` = 12.10.2026, `DAILY_DAYS` = 7; `dailyBoundary` ist das Spätere aus (heute − 7 Tage) und `DAILY_SINCE`; ein Eintrag liegt im Tagesbereich, wenn `first_at ≥ DAILY_SINCE`, der Sitzungstag (sonst `first_at`) ≥ Grenze und `origin <> 'backlog'`. Der Bereich wird bei jeder Abfrage neu bestimmt: ein `daily`-Eintrag rutscht nach 7 Tagen von selbst in den Altbestand. Herkunft beim Schreiben: `backlog`, wenn der Aufrufer es sagt oder der Eintrag außerhalb liegt, sonst `daily`; ein vorhandenes `daily` bleibt `daily`.
- **Fällige Einträge** (`dueEntries`): Termin erreicht oder `internal` mit anderem Programmstand; Reihenfolge Abruf, Extraktion, Regeln, darin neuester Sitzungstag, wenigste Versuche, ältester letzter Versuch; wartende Server ausgelassen; je Server höchstens 20 (ein fälliger Server 3); höchstens 500.
- **Wer schreibt:** Abruf über `noteImport` (`pipeline-jobs.mjs`): kein Ergebnis → `internal`, `refused` → `blocked`, Status ≠ 200 → `classify`, teilweise mit Netzfehler → `transient` ohne Server, 409 → nichts, Erfolg → `clearFailure` mit Server. Die Skripte der Ablage über `scripts/pdf-store/failures.mjs` (eigene schreibende Verbindung): `pdf` aus `fetch.mjs`, `text`/`internal` und `ocr`/`slow` und `ocr-fetch` aus `text.mjs`, `rules`/`internal` aus `read-reports.mjs`. `seedFromStore` überträgt beim ersten Lauf die alten Fehler der Ablage als `backlog` (Abruffehler, `ocr_refetch`, `ocr_slow`, Server nach der alten 90-%-Regel).
- Kein Fehler im Sinn des Speichers: ein Abruf über der Zeitgrenze (die nächste Nacht setzt hinter dem Cursor fort) und „Quelle reicht nicht" bei der KI (`ai_work_items`).

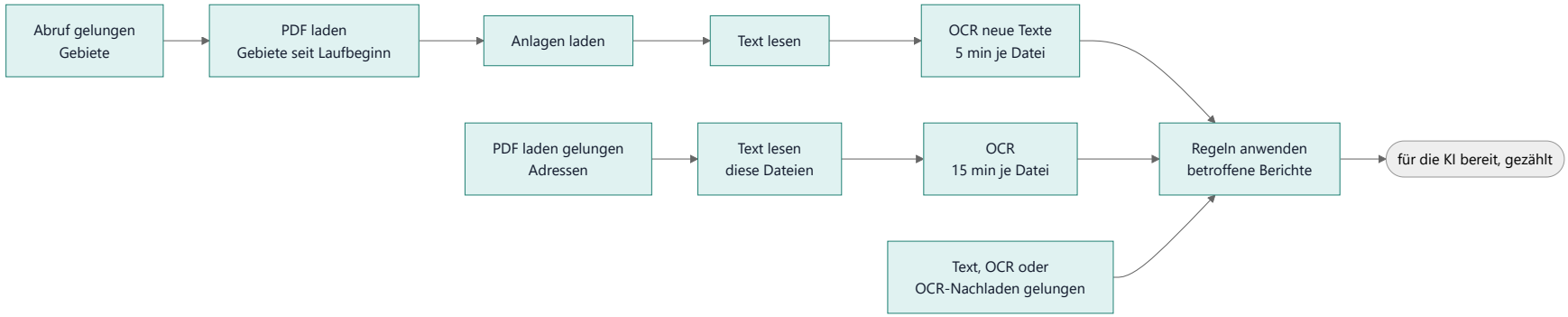
7. Nacharbeit (`scripts/nacharbeit.mjs`)

Job `nacharbeit`, alle 30 Minuten zwischen 09:00 und 23:00, `--range daily --minutes 20`.

1. Ohne Fehlerspeicher Exit 0. Sperre `nightly.lock` nehmen (`takeLane`); hält sie die Nachtkette, ein Auftrag der Ablage oder der Altbestand, lässt der Lauf aus.
2. `dueEntries(daily, 500, perHost 20, includeSlow)` holen; `planBatches` bündelt nach Schritt in der Reihenfolge `import`, `pdf`, `text`, `ocr-fetch`, `ocr`, `rules`.
3. Je Schritt (solange mindestens 1 Minute übrig): Schlüssel in eine Datei schreiben und denselben Befehl wie nachts aufrufen:

Schritt	Befehl
Abruf	`import-worker.mjs --direct --regions … --window <1w, 1m, 3m, 12m oder 24m je nach Alter des ersten Fehlschlags> --origin … --no-catch-up`
PDF laden	`fetch.mjs --urls … --origin … --budget-gb 80` (Hauptdokumente und Anlagen; die Rolle bleibt)
Text lesen	`text.mjs --shas …`
OCR, OCR-Nachladen	`text.mjs --ocr --include-slow --shas … --workers 2 --file-minutes 15`
Regeln anwenden	`read-reports.mjs --ids …`

4. Ausgang je Eintrag: Eintrag weg = gelungen; `last_at` geändert = neuer Termin (oder endgültig, wenn `next_try_at` NULL); unverändert = nicht drangekommen, `next_try_at` + 1 h, damit er nicht jede halbe Stunde wiederkommt.
5. **Folgekette** (`followUp`): nach gelungenem Abruf für die Gebiete PDF laden (`--since-updated` Laufbeginn), Anlagen laden, Text lesen, OCR der neuen Texte, Regeln der Berichte mit `updated_at ≥ Beginn`; nach gelungenem PDF laden Text lesen, OCR, Regeln; nach Text oder OCR nur die Regeln. Die KI startet nicht; gezählt wird `readyForAi`.
6. Ergebnis in `nacharbeit-last`; Exit 1 bei `failed`.


Abruf gelungen
Gebiete
PDF laden
Gebiete seit Laufbeginn
Anlagen laden
Text lesen
OCR neue Texte
5 min je Datei
PDF laden gelungen
Adressen
Text lesen
diese Dateien
OCR
15 min je Datei
Regeln anwenden
betroffene Berichte
für die KI bereit, gezählt
Text, OCR oder
OCR-Nachladen gelungen

8. OCR, tagsüber (`ocr`, `nightly.mjs --ocr-only --range daily`)

Eigene Einheit `ratsmonitor-ocr.service` (Anhalten trifft nur die OCR), alle 10 Minuten von 07:00 bis 23:00, höchstens 3 Stunden. Sperre `ocr-lane.lock` (Bahn OCR), `text.mjs` zusätzlich `ocr.lock`.

1. OCR des Tagesbereichs: `text.mjs --ocr --range daily --workers 2 --file-minutes 5`.
2. Danach, wenn auf der Seite Altbestand angefordert (`altbestand-requests.ocr` ohne `stoppedAt`), zwischen 10:00 und 21:55 und mit mindestens 5 Minuten Rest: OCR des Altbestands mit `--range backlog --file-minutes 15 --include-slow --origin backlog`.
3. Hat kein Schritt Dateien gelesen: Exit 0 ohne Protokolleintrag.
4. Dokumentart nachtragen; dann **Regeln anwenden** für die Berichte, deren Text seit Laufbeginn per OCR entstand (`read-reports.mjs --ids`), und Suche aktualisieren. Entfällt, solange „Verarbeitung, nachts“ läuft (dann kommt der Text dort an).

9. Altbestand (`scripts/altbestand.mjs`, Seite `/admin?seite=altbestand`)

Die Seite legt nur Wünsche in `altbestand-requests` ab (`altbestandAction` in `altbestand-admin.mjs`) und stößt die Jobs an:

Aktion	Wirkung
`import`	`{regions, window 1m/3m/12m/24m, requestedAt, done:[], failed:[]}`: Gebiete rückwirkend abrufen (alle angebundenen oder eine Liste); ersetzt einen früheren Wunsch; `import-stop` setzt `stoppedAt`
`rework`	`{requestedAt, phase}`: alte Fehler erneut versuchen; mit `reopen` werden auch wartende und endgültige Einträge sofort fällig
`ocr` / `ocr-stop`	OCR des Altbestands anfordern (führt der Job `ocr` aus) bzw. anhalten
`pdf`	Auftrag der Ablage (`run` Ältere Monate laden, `others` Anlagen laden, `texts` Text lesen) für den Job `pdf-job`
`rules`	Verarbeitung jetzt bzw. alle Berichte neu (`rule-texts`)
`release`	Server freigeben (`releaseHost`)

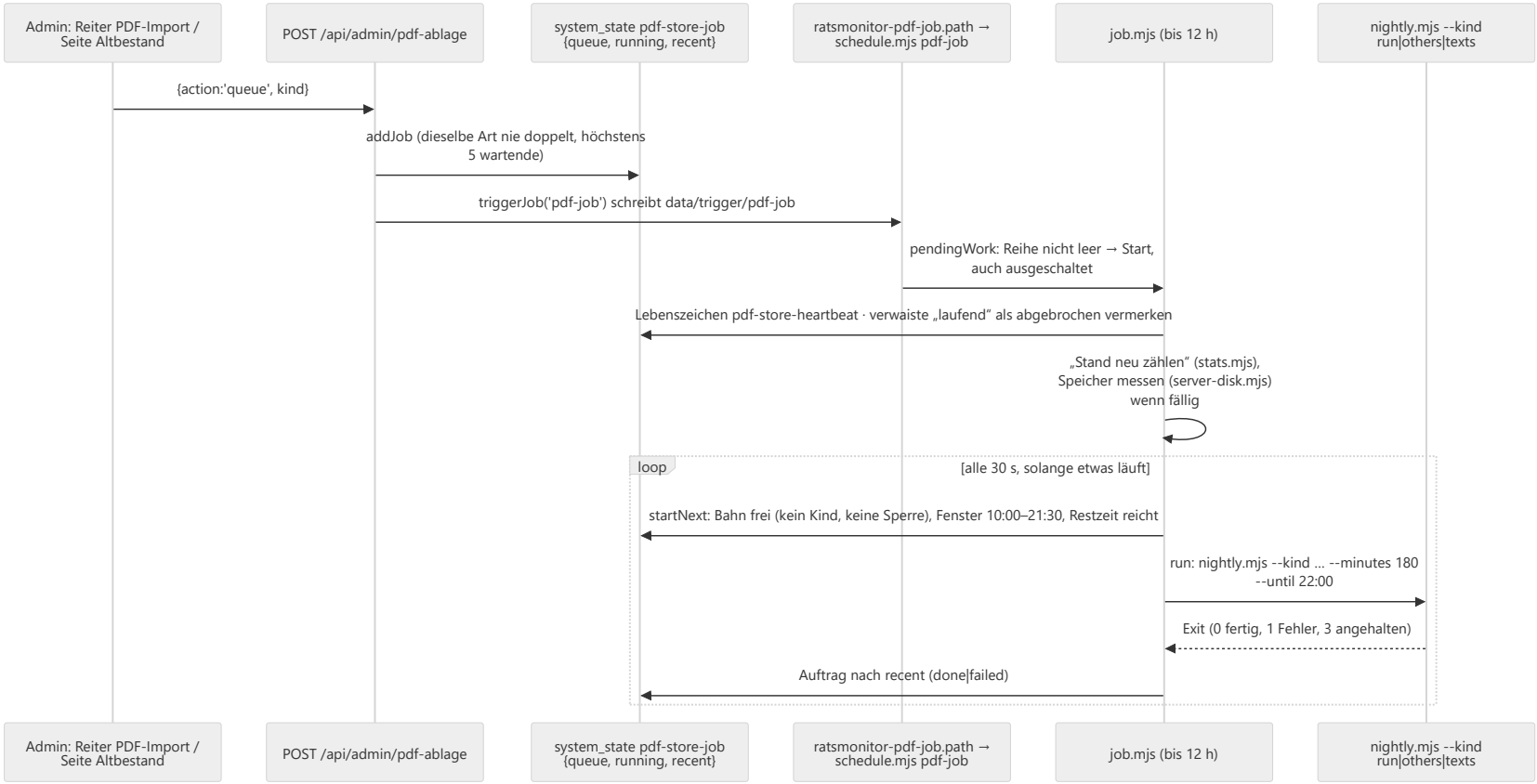
Job `altbestand` (alle 10 Minuten, 10:00 bis 22:00, `--minutes 120 --until 22:00`):

1. Ohne offenen `import`- oder `rework`-Wunsch Exit 0; Restzeit unter 5 Minuten: Ende; ohne Fehlerspeicher: Ende.
2. **Vorrang der Nacharbeit:** gibt es fällige Einträge im Tagesbereich und lief die Nacharbeit seit 25 Minuten nicht, lässt der Lauf aus. Sperre `nightly.lock`; belegt: aus.
3. **Alte Fehler** (`rework`): solange mindestens 2 Minuten übrig, `nacharbeit({range:'backlog', includeSlow, minutes: min(30, Rest)})` in Runden; keine fälligen Einträge mehr: `finishedAt`; eine Runde ohne Ausgang beendet die Schleife. Ergebnis `altbestand-rework-last`.
4. **Rückwirkend abrufen** (`import`): je 8 offene Gebiete `import-worker.mjs --direct --regions … --window <window> --origin backlog --no-catch-up ; gescheitert ist ein Gebiet mit Eintrag import` seit Paketbeginn; für gelungene die Folgekette mit `origin: backlog`; `done`/`failed` fortschreiben, bis nichts mehr offen ist, dann `finishedAt`. Fortsetzbar über Tage, anhaltbar.
5. `altbestand-last` schreiben.

Der Knopf „Alte Fehler erneut versuchen“ im Reiter PDF-Import (Auftrag `retry`) macht die wiederholbaren Abruffehler (`pdf`, `ocr-fetch` mit `transient`, `denied`, `blocked`) und alle wartenden Server sofort fällig, setzt `rework` und stößt `altbestand` und `nacharbeit` an; geladen wird dabei nichts.

10. Aufträge der Ablage (`pdf-job`, `scripts/pdf-store/job.mjs`)

Vom Knopf zum Lauf:


Admin: Reiter PDF-Import / Seite Altbestand
POST /api/admin/pdf-ablage
system_state pdf-store-job {queue, running, recent}
ratsmonitor-pdf-job.path → schedule.mjs pdf-job
job.mjs (bis 12 h)
nightly.mjs --kind run|others|texts
{action:'queue', kind}
addJob (dieselbe Art nie doppelt, höchstens 5 wartende)
triggerJob('pdf-job') schreibt data/trigger/pdf-job
pendingWork: Reihe nicht leer → Start, auch ausgeschaltet
Lebenszeichen pdf-store-heartbeat · verwaiste „laufend" als abgebrochen vermerken
„Stand neu zählen" (stats.mjs), Speicher messen (server-disk.mjs) wenn fällig
loop
[alle 30 s, solange etwas läuft]
startNext: Bahn frei (kein Kind, keine Sperre), Fenster 10:00–21:30, Restzeit reicht
run: nightly.mjs --kind … --minutes 180 --until 22:00
Exit (0 fertig, 1 Fehler, 3 angehalten)
Auftrag nach recent (done|failed)
Admin: Reiter PDF-Import / Seite Altbestand
POST /api/admin/pdf-ablage
system_state pdf-store-job {queue, running, recent}
ratsmonitor-pdf-job.path → schedule.mjs pdf-job
job.mjs (bis 12 h)
nightly.mjs --kind run|others|texts

Besonderheiten: Auftrag `ocr` wird nur übergeben (`altbestand-requests.ocr`, `schedule-now.ocr`, `triggerJob('ocr')`); Auftrag `retry` lädt nichts, sondern macht Einträge und Server fällig (Abschnitt 9). Zwei Bahnen laufen nebeneinander: Laden (`run`, `retry`, `others`, `texts`, auch die Nachtkette, Nacharbeit und Altbestand) und OCR (`ocr`); Sperren `nightly.lock` und `ocr-lane.lock`. Anhalten je Bahn über `pdf-store-stop[lane]`. Die Anzeige hält einen Auftrag ohne Rückmeldung seit 30 Minuten für `stale`; ohne Lebenszeichen seit 20 Minuten sperrt die Seite die Knöpfe („kein Ausführer", lokal immer).

11. Debug: Wege der Berichte, Nachtbericht

- **Wege zählen** (`scripts/pipeline-trace.mjs`, Schritt am Ende von `rule-texts`, nicht fatal): für aktive Berichte mit `updated_at` in den letzten 24 h bzw. 7 Tagen eine Kette von Stufen: PDF ja/nein → Hauptdokument, nur Sitzungsunterlage oder keines → Status in der Ablage samt Fehlerklasse → Text → OCR (none/waiting/done) → Regeln (read/stale/never) → Regel-Zusammenfassung → Titelregel → KI (`ai_work_items.state`); dazu `work_failures` nach Schritt, Grund und Herkunft, `import_runs` nach Status. Ergebnis in `pipeline-trace`.
- **Seite Pipeline-Debug** (`/admin?seite=debug`, `pipeline-debug.mjs`): liest nur `system_state` (Laufbuch, Läufe, `rule-texts-last`, `nacharbeit-last`, `ai-counts`, `pipeline-trace`, `pdf-daily-last`, `admin-pipeline-job`, `altbestand-*`, `systemd-units`) und die Tabellen des Fehlerspeichers; Aktionen `release` und `reopen` im Tagesbereich, danach `triggerJob('nacharbeit')`. Vorübergehend, bis der Betrieb läuft.
- **Nachtbericht** (`scripts/pipeline-report.mjs`, nur lesend, ohne Deploy über ssh): Schalter, Auslöser, Laufbuch je Job, Abruf, Extraktion, Verarbeitung, Nacharbeit, Fehlerspeicher (neu, fällig, Gründe), Server, auffällige Journalzeilen je Einheit.
- **Abruf-Protokoll** (`import-trace.mjs`): je Gebiet die Anfragen des letzten Abrufs (`import-debug:<region>`, höchstens 1.500).

12. Befunde beim Schreiben dieser Dokumentation (17.10.2026)

Abweichungen zwischen Code, Konzept und Kommentaren, nur gelesen, nicht zur Laufzeit geprüft. Nichts davon ist hier geändert.

Nr.	Befund	Belege
A1	„PDF laden höchstens 29 Minuten" (Konzept, Admin-Text); der Code rechnet `floor(120·0,4·0,6)` = 28	`nightly.mjs`, `schedules.mjs`
A2	Der Kopf von `nightly.mjs` sagt, `--kind nightly` stelle einen Auftrag „OCR starten" an; `queueJob` wird importiert, aber nie aufgerufen (die Tagesrest-OCR macht der Job `ocr`)	`nightly.mjs`
A3	`fetch.mjs` nennt „letzte 14 Tage = daily", der Code nutzt `DAILY_DAYS` = 7	`fetch.mjs`, `work-failures.mjs`
A4	`ratsmonitor-pdf-nightly.service/.timer` nennen Zusammenfassungen, Fakten, Wortliste und 180 min; tatsächlich 120 min ohne diese Schritte	`deploy/node/ratsmonitor-pdf-nightly.*`
A5	`nacharbeit.mjs` dokumentiert `--include-slow`; die Kommandozeile setzt `includeSlow` immer	`nacharbeit.mjs`
A6	`altbestand-requests.rework.phase` wird gespeichert, aber nicht gelesen; „je Stufe und Grund" wirkt nur über `reopen`	`altbestand-admin.mjs`, `altbestand.mjs`
A7	Das Aufräumen veralteter OCR-Einträge in `text.mjs` läuft nie (Bedingung `!flag('child')` im Zweig der Pakete); fertige Dateien behalten ihren Eintrag, die Nacharbeit schiebt ihn stündlich weiter	`text.mjs`, `nacharbeit.mjs`
A8	`text/internal` „nach dem nächsten Deploy" greift nicht: `failText` schreibt `texts.version = TEXT_VERSION`, `textless` liefert nur Dateien ohne oder mit älterer Fassung; die defekte Datei wird nicht neu gelesen	`pdf-store.mjs`, `text.mjs`
A9	„3 zählende Fehler in Folge": `fetch.mjs` und `text.mjs` rufen `hostOk` nur für schon wartende Server; Erfolge an einem ok-Server mit `fails` 1 bis 2 setzen nicht zurück (der Abruf tut es über `clearFailure`)	`fetch.mjs`, `text.mjs`, `failures.mjs`
A10	Beim OCR-Nachladen gilt „3 Fehler je Server" je Paket (25 Dateien), nicht je Lauf	`text.mjs`
A11	`pdf-store-admin.mjs` beschreibt `ocr` als „OCR aller PDFs"; `job.mjs` übergibt den Auftrag nur an den Job `ocr`	`pdf-store-admin.mjs`, `job.mjs`
A12	`ratsmonitor-ocr.service` hat `TimeoutStopSec=30`, die Kette plant 60 s Gnadenfrist und spricht von 90 s	`deploy/node/ratsmonitor-ocr.service`, `nightly.mjs`
A13	Kommentar in `ratsmonitor-import.timer` sagt „eine Woche zurück"; der Job nutzt 1 Tag, sonntags 1 Monat	`deploy/node/ratsmonitor-import.timer`
A14	Einen Rollenwert `'primary'` gibt es nicht; Hauptdokument heißt „Rolle leer"	`pdf-store.mjs`

Risiken, aus dem Code gelesen:

1. **Die Tagespipeline kann einen Tag ausfallen:** hält jemand `nightly.lock` (Nacharbeit, Altbestand, PDF-Auftrag), endet `nightly.mjs` mit Exit 0; der Zeitplaner vermerkt `ok` mit `lastStart`, danach ist der Job für heute nicht mehr fällig.

2. **OCR der Nacharbeit gegen den Job `ocr`:** läuft der Job `ocr` (bis 3 h), endet `text.mjs` in der Nacharbeit sofort an `ocr.lock`; die Einträge gelten als „nicht versucht“ und rücken stündlich weiter.
3. **„Nicht versucht“ altert nie** (`tries` steigt nicht): Einträge, die der Schritt grundsätzlich nicht anfassen kann (A7, A8, Adresse nicht mehr im Bericht), kommen dauerhaft stündlich wieder.
4. **Die OCR des Altbestands hat kein Ende:** `altbestand-requests.ocr` bekommt nie `finishedAt`; der Job `ocr` liest täglich von 10 bis 22 Uhr den Altbestand, bis jemand anhält.
5. **PDF-Aufträge außerhalb des Fensters** halten `pendingWork('pdf-job')` wahr; der Zeitplaner startet `job.mjs` alle 10 Minuten, der sich jedes Mal ohne Start beendet.
6. **Abruf und KI-Stapel umgehen die Grenze:** `readDocument` und `fetchSource` legen PDFs mit `put` ab, ohne `keeper`, Gebiet und Rang; nur der Sicherungsschritt der Nacht fängt das auf.
7. **Score ohne Titel:** `ranked` ruft `effectiveType` ohne Titel; die Titel-Ausnahme „beginnt mit Anlage“ greift im Score nicht.
8. Mit `--include-slow` sind zu langsame Dateien nach dem ersten Versuch sofort wieder in der Reihe, nach dem zweiten dauerhaft draußen bis `reopen`; das deckt sich mit dem Konzept, `ocr_slow`-Altfälle werden dadurch sofort fällig.

Suche: Suchkarten, Wortliste, Volltextindex

Stand: 17. Oktober 2026. Teil der technischen Dokumentation. Begriffe: Glossar.

Die Suche ist die Grundlage von Startseite, Landkarte, Kalender, Social Media und plenara.X: Jede Auswertung wählt zuerst über die Suche aus (`source()` in `plenara-x-query.mjs`, siehe Auswertungen). Sie liest nie `topics.payload`, sondern nur die **Suchkarten** und ihre Indizes.

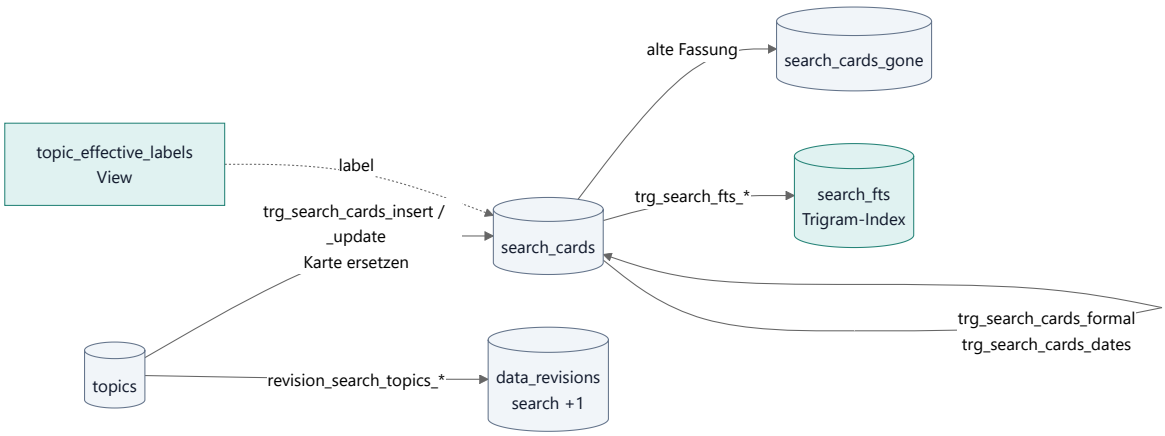
1. Die Suchkarte (`search_cards`)

Eine Zeile je aktivem Bericht (`topics.active=1`, also ohne `identity.mergedInto`). Gepflegt von den Triggern `trg_search_cards_insert` und `trg_search_cards_update` auf `topics` (heutiger Stand `drizzle/0027`), die eine Karte immer **ersetzen, nie ändern**; die vorige Fassung landet in `search_cards_gone`.

Spalte	Herkunft	Beleg
`id`	`topics.id`	`drizzle/0006`
`region_id`, `status`	Spalten von `topics`	
`date`	`substr(event_date,1,10)`, danach „Stand heute“: Trigger `trg_search_cards_dates` setzt bei künftigem Datum `date` auf die letzte Sitzung bis heute und `next_date` auf die nächste anstehende	`drizzle/0024`
`title`	`payload.title`, höchstens 1000 Zeichen	
`gremium`	`payload.committee`	
`teaser`	KI-Kurzfassung, wenn vorhanden (`shortSummary` mit `generatedBy LIKE 'KI-Zusammenfassung%'` oder `contentAnalysis.status='completed'`), sonst die Sätze der Regel-Zusammenfassung (`ruleSummary.short`), sonst `shortSummary`; höchstens 1200 Zeichen	`drizzle/0019`
`search`	`lower(title + teaser + committee)`, Umlaute zu a/o/u, ß zu ss: der **Suchtext**	`drizzle/0019`
`search_exact`	derselbe Text mit Umlauten (für `exact=1`: „Bär“ findet nicht „Bar“)	`drizzle/0014`
`label`	aus dem View `topic_effective_labels` (Rangfolge Hand → KI → Regel-Thema → Titelregel), sonst `unklar`	`drizzle/0027`, `0038`
`formal`	1, wenn `payload.itemKind` in (`formal`, `collection`, `routine`); ohne `itemKind` die alten LIKE-Muster `FORMAL/NOT_FORMAL`. Ändert sich nur `itemKind`, setzt `trg_search_cards_item_kind` die Spalte nach	`drizzle/0017`, `0033`
`next_date`	nächste anstehende Sitzung, sonst NULL	`drizzle/0024`

Das Update-Trigger feuert auch, wenn sich nur das effektive Label ändert. Jede Änderung an `topics` erhöht außerdem den Datenstand `search` in `data_revisions` (Trigger `revision_search_topics_*`, `drizzle/0017`, ohne WHEN-Bedingung): auch „Titel einordnen“ und „Regeln anwenden“ machen damit die Wortliste veraltet, bis „Suche aktualisieren“ läuft.

** `rollCardDates` ** (`server/integrations/search-card-dates.mjs`) rückt Karten nach, deren `next_date` erreicht ist: `date` wird die letzte Sitzung bis heute, `next_date` die nächste oder NULL. Läuft nach jedem Metadaten-Import (`sync.ts`) und in `scripts/refresh-search-words.mjs`.



2. Die Indizes

Wortliste (`search-words.mjs`, Migration `0013`)

Tabelle	Inhalt
`search_words`	je Wort (ab 3 Zeichen aus a–z und 0–9, nicht nur Ziffern; `wordsOf`): Zahl der Karten (`cards`, 501 = „mehr als 500"), Treffer je Ebene (`hits_city`, `hits_district`)
`search_postings`	Karten-IDs je Wort, nur für Wörter mit höchstens `POSTING_MAX` = 500 Karten
`search_word_areas`	vorberechnete Zahl je Gebiet für häufige Wörter
`search_word_facets`	vorberechnete Zahl je Thema und Status für häufige Wörter
`system_state search-words`	Stand: `complete`, `revision` (Datenstand `search` beim Lesen), `hasHits`, gemerkte rowid

Die Liste gilt als **aktuell** (`listCurrent`), wenn `complete` gesetzt ist und `revision` dem heutigen Datenstand `search` gleicht (`SEARCH_REVISION_SQL` in `revision-cache.mjs`; ohne Migration `0017` gilt `content`).

Nachführen (`refreshSearchWords`, Sperre `search-words-lease` in `system_state`, Frist 3 Minuten, beim vollen Aufbau 1 Stunde, vor jedem Abschnitt verlängert):

1. `search_cards_gone` lesen. Mehr als `GONE_MAX` = 5.000 Einträge: nichts abziehen, die vorberechneten Zahlen verfallen.
2. Ersetzte und gelöschte Karten abziehen (IDs, Trefferzahl, Gebiets- und Themenzahlen).
3. Karten an den protokollierten rowids und alle Karten mit größerer rowid als der gemerkten neu lesen.
4. Wörter, die über 500 Karten kommen, verlieren ihre IDs.
5. Stimmt die Bilanz nicht (`hitsOk`), werden die vorberechneten Zahlen gelöscht; der nächste Skriptlauf baut dann voll neu (`fullIfNoHits`).

Voller Aufbau (`buildAll`): bei `--full`, fehlendem oder unvollständigem Stand, passender Karte nicht an der gemerkten rowid, fehlenden Zahlen oder Schemawechsel. Tabellen neu anlegen, alle Karten lesen, „blockierte" seltene Wörter bestimmen (in häufigen Wörtern enthalten, über 300 Wörter oder über 3.000 Karten), für diese die Zahlen zählen, Protokoll bis zum Startstand leeren. Lokal zuletzt 158 s.

Aufrufer: der Abruf (`sync.ts`, `onlyIfBuilt`, höchstens 20.000 Karten je Lauf), `scripts/refresh-search-words.mjs` (Schritt „Suche aktualisieren" des Jobs `rule-texts`), der Dev-Server beim Start. Die Migrationen `0027`, `0034` und `0038` löschen den Stand, damit der nächste Lauf voll aufbaut.

Volltextindex (`search_fts`, Migration `0023`)

FTS5 mit Trigram-Tokenizer als „external content" über `search_cards.search`, mitgeführt von vier Triggern auf `search_cards`. Lokal 650 MB bei 941.000 Karten. Genutzt nur, wenn die Wortliste keine Kandidaten liefert (Abschnitt 3), für Begriffe aus `[a-z0-9]{3,}`, und nur, wenn die Tabelle existiert. Für Seite und Strom nur, wenn ein Vorabtest höchstens `FTS_PAGE_MAX` = 3.000 Treffer zählt; Zähler laufen ohne Grenze über den Index. Die Bedingung `instr(search,?)>0` bleibt zusätzlich bestehen, damit das Ergebnis dasselbe ist wie ohne Index (Test `tests/search-fts.test.mjs`).

3. Eine Anfrage an `GET /api/search`

Route `app/api/search/route.ts` → `guard()` (Stufe des Kontos; Gäste bekommen höchstens 10 Treffer und keine Seite über 1, `guest-search.mjs`) → `monitorSearch` in `server/integrations/monitor-search.mjs`.

Parameter (`parseMonitorSearch`)

Parameter	Bedeutung
`q`	Begriff, höchstens 200 Zeichen. Komma, Semikolon, | und „oder" trennen ODER-Gruppen; Wörter in einer Gruppe sind UND. Normalisiert (klein, ohne Diakritika, ß→ss), Füllwörter fallen weg. Höchstens 8 Gruppen, 12 Wörter je Gruppe, 30 insgesamt
`exact=1`	nur ganze Wörter (GLOB auf `search_exact`), Umlaute bleiben
`area`, `scope=only|with`, `more`	Gebiet (AGS mit 2, 5, 8 oder 9 Stellen), nur das Gebiet oder mit Untergliederungen, bis zu 8 weitere Orte
`within` / `without`	Umkreis als AGS-Liste (höchstens 2.000; der Client schickt die kürzere Liste)
`level`	`city` oder `district` ohne Ortsangabe
`label`	Themenname; alte Namen (`labels-v2`) werden übersetzt
`month`, `from`, `to`	Zeitraum

Parameter	Bedeutung
`status`	einer der acht Status
`noformal=1`	ohne Formalien (`formal=0`)
`sort`	`asc`, `desc`, `relevance` (Titeltreffer zählt dreifach)
`page`, `size`	Seite ab 1 (höchstens 250), 15 oder 20 Treffer
`part`	`''` (alles), `page`, `facets`, `stream` (NDJSON); `x` nur intern für plenara.X
`revision`	Datenstand des Clients; weicht er ab, antwortet die Route mit 409 „Datenstand wurde geändert“ (Weiterblättern braucht dann ein Neuladen)

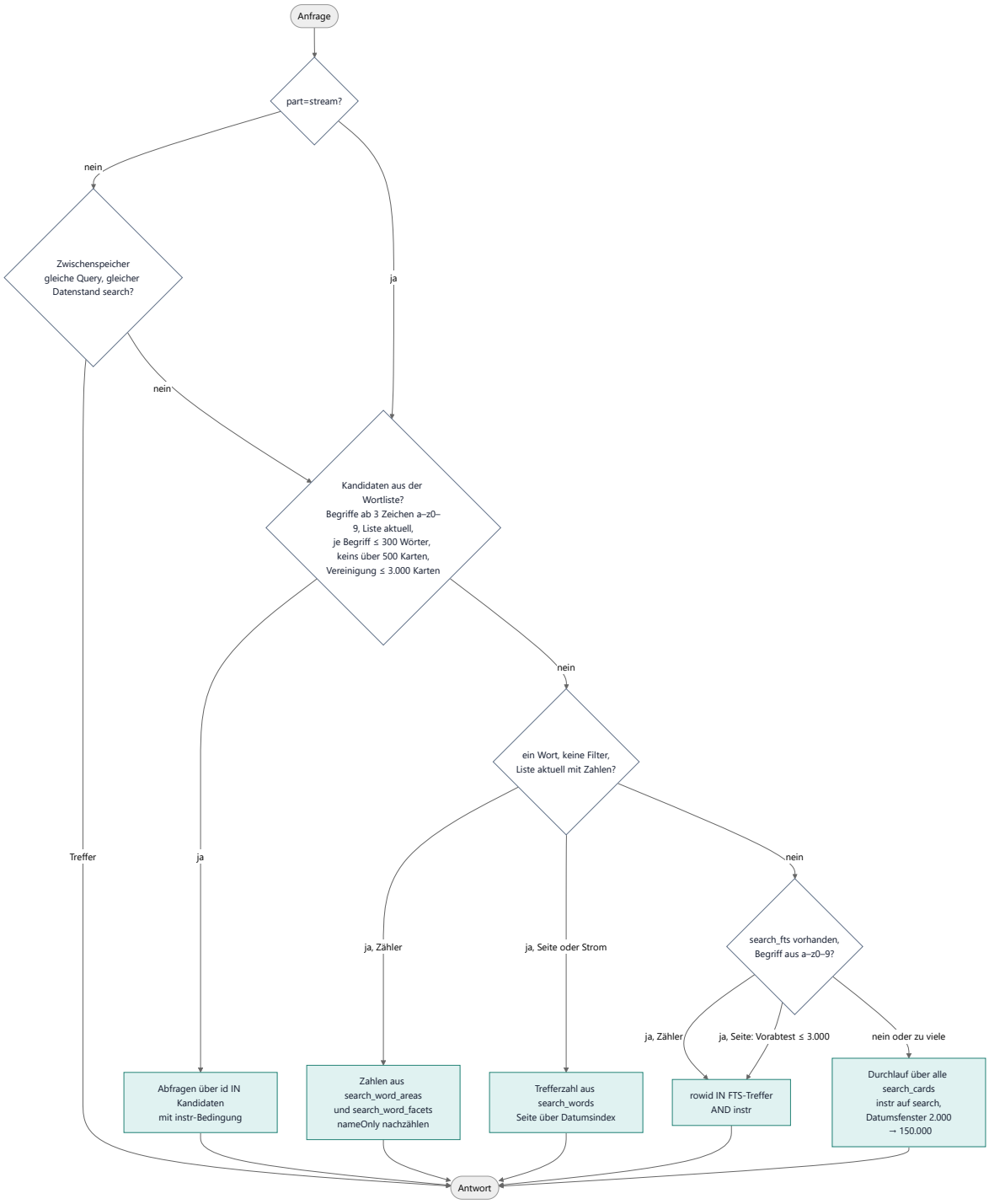
Es gibt keinen Cursor; geblättert wird über `page` und `revision`.

Antwort

Teil	Inhalt
`articles`	je Suchkarte `id`, `date`, `status`, `title`, `teaser`, `gremium`, `regionId`, `ags`, `gemeinde`, `thema` (Labelname, sonst „Sonstiges“) und `steps` (Stationen `{d,s,c}` der Beratungsfolge, nur von derselben Quelle, für den Zeitstrahl der Trefferzeile)
`hasMore`, `page`, `pageSize`, `revision`, `storageAvailable`	Blättern und Stand
`total`	Trefferzahl
`areaCounts`	Zähler je AGS-Präfix (voll, 5, 2 Stellen) für die Landkarte; Thema- und Statusfilter wirken, der Ortsfilter nicht; ein Bericht eines Gemeindeverbands zählt bei jeder Mitgliedsgemeinde
`badgeCounts`	Abzeichen der Landkarte: jede Karte genau einmal, Summe = `total`
`themaCounts`, `statusCounts`	Zähler je Label (ohne Themenfilter) und je Status (ohne Statusfilter)
`monatCounts`	immer leer (Altlast im Client-Typ)

Im Strom (`part=stream`) kommen Zeilen `{known}` oder `{known,total}`, dann `{articles}` mehrfach, zuletzt `{end,hasMore,revision,pageSize}`.

Entscheidungsbaum



Weitere Regeln:

- **Gebietsbedingung:** umfasst die Auswahl höchstens die Hälfte des Katalogs, `region_id IN (…)`, sonst `NOT IN`. Ein Begriff, der in einem Gebietsnamen vorkommt, trifft zusätzlich alle Karten dieses Gebiets (`OR region_id IN (…)`, nur bei nicht leerer Liste). „Nur das Gebiet“ mit dem Schlüssel einer kreisfreien Stadt meint die Stadt selbst (`ownCity`).
- **Formalien:** mit Spalte `formal` die Bedingung `formal=0` (ohne Begriff über den Teilindex `idx_search_cards_noformal`); ohne Spalte 25 LIKE-Muster auf `lower(title)`. Die Muster stehen doppelt (`monitor-search.mjs` und `drizzle/0017`), ein Test hält sie gleich.
- **Strom:** mit Kandidaten oder kleinem Ort (bis 30 Gebiete) eine einzige Abfrage; sonst zuerst `knownWords` („no“ beendet sofort), dann Datumsfenster, die sich von 2.000 Karten verdoppeln bis 150.000.
- **Seite:** `ORDER BY date … LIMIT CAST(? AS REAL)`, damit SQLite nach einer Seite über den Datumsindex abbricht.
- **Zwischenspeicher:** je Datenbankobjekt 200 Einträge (Schlüssel = sortierte Query), gültig solange der Datenstand `search` gleich bleibt; `part=stream` umgeht ihn. Dazu HTTP `Cache-Control: private, max-age=30, stale-while-revalidate=120`.

Die Gebietsabdeckung (`/api/search/coverage`) hat einen eigenen Speicher (5 Minuten).

Gemessen auf der lokalen Kopie (Zähler, ms, ohne Wortliste): „haushalt“ 5.101 → 187 mit Index, „turnhalle“ 2.776 → 17, „kormoran“ 468 → 7 (Belege und weitere Zahlen in `requirements/datenmodell-plan.md`).

4. Einordnung: welches Label die Suche zeigt

`storedClassification` (`shared/analysis-state.mjs`) und der View `topic_effective_labels` (`drizzle/0038`) wählen in dieser Reihenfolge; `tests/effective-labels.test.mjs` prüft, dass beide dasselbe ergeben:

1. **Einordnung von Hand** `labelAssessments.review` (Reviewer, Datum, nicht `stale`, passende `sourceSignature`).
2. **KI-Label** `labelAssessments.ai` (`status='completed'`, `method='ai-local-content-v2'`, Katalog `labels-v2` oder `labels-v3`).
3. **Thema aus den Regeln** `payload.theme` (nur mit `version='themes-v3'` und `evidence` = heutiger Titel; `offen` wird `unklar`).
4. **Titelregel** `payload.classification` (`method` beginnt mit `title-rules-v`, `evidence` = heutiger Titel; erzeugt von `classifyTopic` = Art des Punkts plus Thema nur aus dem Titel).
5. Sonst `unklar` („Sonstiges“).

Labels aus `labels-v2` (16 Werte) werden fest übersetzt (`LEGACY_LABELS`: Klima → Umwelt, Gesundheit → Soziales, Sport → Kultur, Sitzung und Allgemein → unklar). Formalien erkennt die Suche nicht über ein Label, sondern über `formal`.

Die Themenregel selbst (`shared/themes.mjs`): 10 Themen mit je 7 Unterthemen; das Thema ist das des zuerst genannten starken Begriffs im Titel, erst wenn der Titel kein Thema nennt, zählen Betreff und Beschlussvorschlag aus dem Text (`subjectOf`, höchstens etwa 500 Zeichen). Schwache Themen (Finanzen, Verwaltung) zählen nur ohne starken Begriff eines anderen Themas. Gemessene Qualität in `shared/rule-quality.json` (Thema 90 % bei 84 % Abdeckung, frische Stichprobe 14.10.2026).

5. Was die Suche nicht tut

- Sie liest nie `topics.payload` und nie die PDF-Ablage; alles kommt aus der Suchkarte und den Indizes.
- Sie rechnet keine plenara.X-Zahlen; `part:'x'` liefert nur die Karten-IDs, gezählt wird in `x_cases` (Auswertungen).
- Sie schreibt nichts außer ihrem Zwischenspeicher im Prozess.

6. Bekannte Eigenheiten (Stand 17.10.2026)

- `monatCounts` ist immer leer; der Client-Typ führt das Feld noch.
- Die Relevanzsortierung vergleicht den Titel mit Umlauten gegen den normalisierten Begriff; der Titelbonus greift bei Begriffen mit Umlaut vermutlich nicht.
- Solange der Job `rule-texts` zwischen „Regeln anwenden“ und „Suche aktualisieren“ steht, ist die Wortliste veraltet und die Suche läuft über den Volltextindex oder alle Karten.
- Verliert ein Nachführen die Sperre (harter Abbruch), ist die Wortliste bis zum Ablauf der Frist nicht nachführbar.

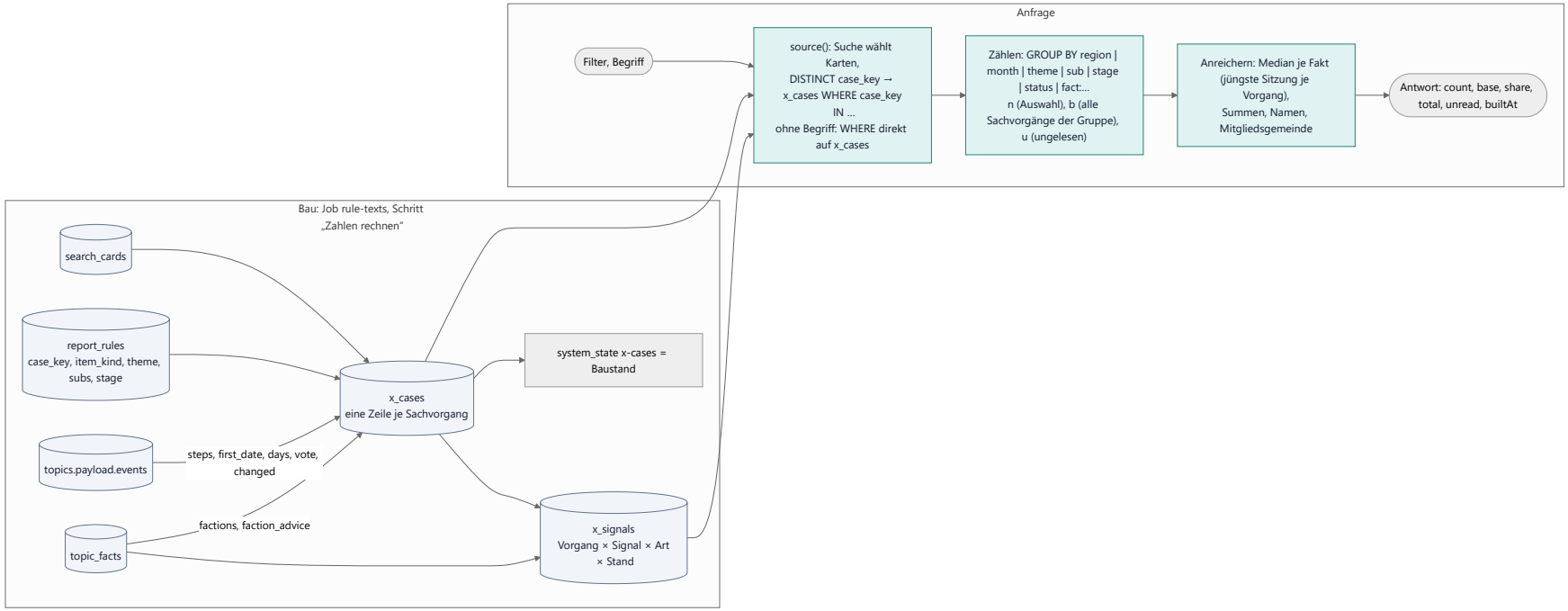
Auswertungen: plenara.X, Vorberechnung, Social Media

Stand: 17. Oktober 2026, gelesen am Code. Teil der technischen Dokumentation. Begriffe: Glossar. Produktsicht und Zielgruppen: `docs/produkt/plenara-x-ueberblick.md`; Konzepte `requirements/plenara-x-konzept.md` (teils überholt, Abschnitt 8), `plenara-x-tools-konzept.md`, Signale `docs/produkt/plenara-x-signale.md`.

1. Ein Weg für alles

plenara.X ist der Analyse-Teil (Stufe Enterprise). Es gibt kein Modell im Ablauf; jede Zahl führt zurück zu Vorgängen und Originalquellen. Seit 15.10.2026 geht jede Auswertung denselben Weg, ohne eigenes Datenmodell je Seite:

1. **Auswahl** über die normale Suche (`source()` in `plenara-x-query.mjs`): dieselben Filter wie auf der Startseite (`q`, Gebiet, Umkreis, Thema, Status, Zeitraum, `exact`, `noformal`), mit Begriff über Wortliste und Volltext (`part:'x'`).
2. **Zählen** in `x_cases`: Vorgänge, keine Suchkarten; Formalien nie; gleiche Titel je Gebiet einmal.
3. **Anreichern** aus `report_rules` (Thema, Unterthemen, „Was geschieht“) und `topic_facts` (Werte, Einordnungen); Signale zusätzlich über `x_signals`.


Anfrage
Filter, Begriff
source(): Suche wählt Karten, DISTINCT case_key → x_cases WHERE case_key IN … ohne Begriff: WHERE direkt auf x_cases
Zählen: GROUP BY region | month | theme | sub | stage | status | fact:… n (Auswahl), b (alle Sachvorgänge der Gruppe), u (ungelesen)
Anreichern: Median je Fakt (jüngste Sitzung je Vorgang), Summen, Namen, Mitgliedsgemeinde
Antwort: count, base, share, total, unread, builtAt
Bau: Job rule-texts, Schritt „Zahlen rechnen“
search_cards
report_rules case_key, item_kind, theme, subs, stage
x_cases eine Zeile je Sachvorgang
system_state x-cases = Baustand
topics.payload.events
steps, first_date, days, vote, changed
factions, faction_advice
topic_facts
x_signals Vorgang × Signal × Art × Stand

2. Der Bau der Zahlen (`scripts/plenara-x/build-x-cases.mjs`, `server/integrations/x-cases.mjs`)

Schritt 3 des Jobs `rule-texts` (nach „Regeln anwenden“ und „Suche aktualisieren“), läuft immer; lokal 48 s. Bricht ab, wenn Migration 0036 fehlt. Eine einzige Transaktion (`BEGIN IMMEDIATE`, `busy_timeout` 60 s):

1. **Vergangene Vorgänge** (`BUILD`): `search_cards` LEFT JOIN `report_rules` je Karte, nur `date <= Bautag` und Art des Punkts `subject` (ohne `report_rules`-Zeile entscheidet `search_cards.formal`); gruppiert nach `coalesce(r.case_key, region_id || '|' || lower(title)) mit` max(date): die übrigen Spalten (label, stage, subs, status, card_id`) stammen von der Karte mit der letzten Sitzung; `cards` = Zahl der Sachpunkt-Karten, `unread` = 1 ohne `report_rules`.
2. **Künftige Vorgänge** (`FUTURE`): dieselbe Abfrage mit `min(date)` und `date > Bautag`, nur Schlüssel, die es unter den vergangenen nicht gibt (nur angesetzt, wie in der Suche).
3. `DELETE FROM x_cases`, Insert aller Zeilen (`month` = `date[0..7]`).
4. ** `buildXSteps`:** je Vorgang den Zeitstrahl `payload.events` der Karte `card_id` lesen → `first_date`, `steps` (Zeilen `Datum<TAB>Status<TAB>Gremium`), `days` (erste Station bis Entscheidung), `vote` (`unanimous`, `majority`, `), changed` (`readResult` aus dem Ergebnistext der letzten entscheidenden Station); dazu `terms` = `termsText(title)` (Wörter ab 4 Buchstaben ohne Füll- und Sitzungswörter, `x-terms.mjs`) und `private_build` = `privateBuild(title)`.
5. ** `buildXFactions`:** Fakten `faction` und `faction_advice` über alle Berichte desselben `case_key` → `factions` (Zeilen `Fraktion<TAB>antrag|anfrage`), `faction_advice` = 1, wenn eine Karte empfiehlt, dem Antrag nicht zu folgen.
6. ** `buildXSignals`:** `DELETE FROM x_signals`, dann je Eintrag von `SIGNALS` ein `INSERT … SELECT DISTINCT case_key, signal, art, phase` (Abschnitt 4).
7. `system_state` `x-cases` = `{builtAt, today, cases, future, unread, steps, factions, signals}`; COMMIT (bei Fehler ROLLBACK).
8. Danach `warmX` (die `X_WARM_SQL`-Abfragen lesen `x_cases` mit vier Indizes und `x_signals` einmal ganz) und für den Reiter „Themen & Texte“ die Zählung je Thema, Unterthema und Fakt sowie `factCoverage` (Gemeinden mit Faktwerten).

Der Webserver hält dieselben Abfragen warm: 5 s nach dem Start und alle 10 Minuten (`serve.mjs`, dazu die 16 Teaser; aus mit `RM_SKIP_WARM=1`).

Spalten von `x_cases`

Spalte	Migration	Bedeutung
`case_key`	0036	`report_rules.case_key` (= `region_id|titleKey(title)`); ungelesen `region_id|lower(title)`
`region_id`, `date`, `month`	0036	Gebiet; letzte Sitzung bis Bautag (künftige: erste anstehende); `date[0..7]`
`label`	0036	Thema = `search_cards.label` der Karte mit der letzten Sitzung (wie der Themenfilter der Suche)
`stage`, `subs`	0036	„Was geschieht“ und Unterthemen aus `report_rules`; `''` = ungelesen
`status`, `card_id`, `cards`, `unread`	0036	Status der Karte, ihre Kennung, Zahl der Karten, ungelesen
`first_date`, `steps`, `days`, `vote`, `changed`	0041	Verlauf aus dem Zeitstrahl (nur der Karte `card_id`)
`terms`, `private_build`	0042	Begriffe aus dem Titel (`Schlüssel<TAB>Wort` je Zeile), privater Bauantrag
`factions`, `faction_advice`	0043	Fraktionen aller Berichte des Vorgangs

Indizes `_region(region_id,date)`, `_date`, `_label`, `_atlas(date,label,month,region_id,unread)`. Es gibt keine Spalten `title_key`, `theme`, `sub`: Thema heißt `label`, Unterthemen `subs`.

3. Die Abfrage `GET /api/x` (`app/api/x/route.ts`, `plenara-x-query.mjs`)

1. **Zugang:** `gated(xGet, {feature:'analytics'})`; Antwort `Cache-Control: private, max-age=300, stale-while-revalidate=600`, `Vary: Cookie`.
2. **Zwischenspeicher `xCached`** (`atRevisionStale`): Schlüssel = Art (`x-v6`), Tagesdatum und sortierte Parameter; Stand = `system_state` `x-cases`; bei älterem Stand (jünger als 6 h) der alte Wert sofort, neue Rechnung im Hintergrund; 20 Filter je Art im Arbeitsspeicher des Hauptprozesses.

3. **Worker-Thread:** mit `RM_X_WORKER` rechnet einer von `RM_X_WORKERS` (2) Threads mit eigener Nur-Lese-Verbindung (`x-offload.mjs`, `x-registry.mjs`, `server/node/x-worker.mjs`); `RM_X_WORKER=0` schaltet ab, dann rechnet der Hauptprozess. Ohne Datei (Dev-Server) ändert sich nichts.
4. ** `parseX` :** eigene Parameter `theme` (Seite: `thema`), `sub` (setzt das Thema), `stage`, `by` (`region|month|theme|sub|stage|status` oder `fact:<Art>`), `list=1`, `value`, `offset`, `view` (`atlas`, `fakten`), `sum` (nur `housing`), `region`; der Rest geht an `parseMonitorSearch` (Filter der Suche); ohne `to` gilt „bis heute“.
5. Ohne `system_state` `x-cases`: **503** „werden gerade aufgebaut“.
6. ** `source()` :** `searchFilters(db, catalog, {...f, to, noformal:false, part:'x'})`. Ohne Begriff gilt der WHERE-Teil der Suche direkt auf `x_cases` (Gebiet, Datum, Label, Status heißen dort gleich); mit Begriff sucht die Suche einmal in `search_cards`, die Karten werden zu `DISTINCT case_key` (aus `report_rules` oder `region|lower(title)`), dann `x_cases WHERE case_key IN json_each(?). Optionen steps, terms, factions (Zusatzspalten) und noPrivate (private_build=0`).
7. **Auswahl innerhalb** (`selection()`): `label = theme`, `subs LIKE ',sub,'`, `stage`, `region_id`.
8. **Zählen:** einfache Gliederung ein `GROUP BY` mit `n` (Treffer), `b` (alle Sachvorgänge der Gruppe unter denselben Suchfiltern, ohne Thema, Unterthema, Stand) und `u` (ungelesen); Unterthemen über `json_each(subs)`, Fakten über `topic_facts ⋈ report_rules ON case_key mit count(DISTINCT case_key); Nenner einmal je Anfrage (memo, WITH sel AS MATERIALIZED`).
9. **Anreichern:** bei Fakten mit Zahl der Median je Gruppe (je Vorgang der Wert der jüngsten Sitzung, `NEWEST_ORDER`; Einheiten `X_FACT_NUM`); `sum=housing` je Vorgang der größte Wert, je Gruppe addiert; Namen, Gemeindeschlüssel, Mitgliedsgemeinde.
10. **Antwort:** `count` (Vorgänge), `base`, `share` (Prozent, eine Stelle), `total`, `unread`, `builtAt`, dazu `sum`, `median`.

Besondere Ansichten:

- ** `view=atlas` ** (`/analytics/atlas?thema=…&sub=…`, `pages/analytics/AtlasPage.tsx`, `XAtlasView.tsx`): `groupsAtOnce` zählt Thema, Monat und Ort in einem Durchgang über `idx_x_cases_atlas`; dazu Unterthemen oder Themen, die 10 jüngsten Vorgänge und `compare` („Ort im Vergleich“: genau ein `area`, ohne `q`, `sub`, `stage`; letzte 12 Monate; Vergleichsgruppe nach Art, Größenklasse, Land und Wilson-Einordnung aus `peer-norms.mjs`, Mindestmengen `NORM_RULE`). Werte aus den Unterlagen nur mit `facts=1` und `X_FACT_GATE`; die Seite fragt sie seit 14.10.2026 nicht mehr ab.
- ** `view=fakten` ** (`/analytics/fakten?fakt=…&wert=…`, `FaktenPage.tsx`): Orte (`by=region`) und `factMaps` für die freigegebenen `MAP_FACTS` (`shared/map-facts.mjs`, Freigabe ab 90 % Qualität: `hebesatz`, `budget`, `fee_rate`, `annual_result`, `land_price`): je Fakt und Gebiet Vorgänge mit Wert, Median, `latest()` (jüngster und voriger anderer Wert, rot bis grün), Mitgliedsgemeinde über `where()`; Haushalt je Einwohner, Wohneinheiten je 1.000 Einwohner.
- ** `list=1&by=…&value=…&offset=…` :** 50 je Seite (`X_LIST_SIZE`), jüngste zuerst, Titel und Gremium aus `search_cards`, Ortsangabe über `where()`, bei Fakten mit Zahl der Wert (`pages/analytics/RadarDrill.tsx`).

Qualitätstore: `MAP_FACTS` (dritter Eintrag) für die Karte; `X_FACT_GATE` / `X_SUM_GATE` nur im Atlas mit `facts=1`. `by=fact:…` und die Listen prüfen keines dieser Tore.

4. Signale (`/api/x/signale`, `plenara-x-signals.mjs`, `/analytics/signale`)

Zugang `feature:'analytics'`, Zwischenspeicher `x-signale-v8`, Auswahl wie `/api/x` über `parseX` und `source()`; Parameter `signal`, `phase`, `art`, `offset`, `first=1`. Schneller Weg `fastSignals`: Zahl je Signal mit `count(DISTINCT case_key)` über `x_signals`, die Vorgänge des gewählten Signals einmal lesen, in JavaScript nach Stand, Art und Ort zählen; „Erstmals hier“ = frühester Vorgang dieses Signals je Gebiet über den ganzen Bestand, wenn der Bestand des Gebiets mindestens ein Jahr früher beginnt (`firstOf`). Ohne `x_signals` Rückfall direkt über `topic_facts` (ein Test prüft die Gleichheit). Antwort `{signals, signal, phases, arts, placeCount, places, total, offset, size 50, builtAt, firstCount?, items}`; `items` sind Karten wie in der Suche mit `place`, `firstHere`, `phase`, `arts`. Keine Kennzahlen (Zahlen gehören zu Fakten).

Die 11 Einträge von `SIGNALS` (Fakt-Bausteine aus `factsOf` in `shared/topic-facts.mjs`, `facts-v27`):

Signal	Fakt (cases)	Art (Filter)	Stand (phase)
vergaben	tender	Leistung	Detail des Fakts
wohnbau	plan_use Wohnen/Mischgebiet, plot_price, area_dev Wohnen	wohnbau_art	Verfahrensschritt plan
gewerbe	plan_use (Gewerbe), land_price, area_dev, retail	Nutzung	plan
wind	wind_step, wind_count, ee_power = Wind	Verfahren	aus wind_step
solar	solar, ee_power = Photovoltaik	Art	x_cases.stage
waerme	heat_plan, energy	Art	Schritt aus heat_plan, sonst „Maßnahme“
schulen	school_build	Einrichtung	x_cases.stage

Signal	Fakt (cases)	Art (Filter)	Stand (phase)
strasse, wasser, breitband	street, water, broadband	Vorhaben	Detail des Fakts
mobil	project ohne „Öffentliches WLAN“	Angebot	x_cases.stage

Nur aus dem Titel: projectsOf, tenderOf, wind_step, area_dev, solar, energy, school_build, retail, street, water, broadband; aus Titel und Text: planOf, planUseOf, heatPlanOf, windCountOf, eePowerOf, Grundstückspreise.

Freigabe: shared/rule-quality.json führt je Signal quality, recall, art, phase; der Reiter „Themen & Texte“ zeigt „offen“ erst ab 90 % Qualität, Art und Stand und 80 % Trefferquote (signalGate in admin-rule-texts.tsx). Die Abfrage und die Seite werten das nicht aus: sie liefern immer alle 11 Kategorien.

5. Die Tools (/api/analytics/*)

Alle mit gated(…, {feature:'analytics'}) außer diffusion (nur gated()), alle über xCached, alle 503 ohne Bau. Mit by=… liefern sie die Liste hinter einer Zahl über listPage (analytics-lists.mjs: Gebiet + titleKey einmal, 50 je Seite, Ortsangabe über where(); Ortsvergleich zusätzlich place=<Schlüssel>).

Tool (Seite, Route, Modul)	Eingabe	Wie gezählt wird	Besonderheit
Ausbreitung (in Trends, diffusion, analytics-diffusion.mjs)	Suchfilter, Begriff Pflicht (je Wort ab 3 Buchstaben), to = heute	source(), je Gebiet min(FIRST_DATE), max(date), count in x_cases, nur Gemeindeebene	Verbände je Mitgliedsgemeinde über where(); Bestandsbeginn aus search_cards; „von Anfang an“ = erste 14 Tage; zählt private Bauanträge mit
Themennetz (/analytics/graph, graph, analytics-graph.mjs)	Suchfilter, Begriff optional	alle Vorgänge über source(steps, terms, noPrivate); Begriffe aus terms, Gremien aus steps, Thema aus label, Land	Gewichtung wie tf-idf gegen search_words.cards; Kanten mit Jaccard, n ≥ 2, w ≥ 0,04; 45 Begriffe, je Art 8 Knoten; Liste by=node
Trends (/analytics/trends, trends, analytics-trends.mjs)	Suchfilter ohne Zeitraum, window 30/90/180	zwei gleich lange Fenster in x_cases über source(terms, noPrivate); Begriffe aus terms	z-Test für Anteile: mindestens 5 Treffer, Verhältnis ≥ 1,4, z ≥ 2,5; Gruppen neu, steigend, fallend; part=spread = Ausbreitung je Begriff über FIRST_DATE; Liste by=term
Ortsvergleich (/analytics/vergleich, compare, analytics-compare.mjs)	area + more, 2 bis 4 Orte, scope=only	je Ort source(steps, terms, noPrivate): gesamt, je 1.000 Einwohner, Themen- und Statusanteile, Gremien, Monate; Vergleichsbasis alle Gebiete mit denselben Filtern (5 Minuten im Speicher)	„typische“ Begriffe z ≥ 3, Verhältnis ≥ 1,8, n ≥ 4; peer-norms.mjs wird hier nicht genutzt (nur im Atlas); Liste by=all|label|committee|term&place=
Beschlüsse (Trends, Reiter *Ausgang*, /analytics/beschluesse, decisions, analytics-decisions.mjs)	Suchfilter, to = heute	source(steps); Endstand aus status; Gremienebene aus jeder Station in steps, je Vorgang, Ebene und Status einmal; Abstimmung, Änderung, Dauer aus vote, changed, days	Beschlussquote = beschlossen / (beschlossen + abgelehnt + vertagt), Abweichung mit Wilson-Spanne; Mindestmengen Thema 20, Land 60, Abstimmung 8; keine Quote für Ausschüsse; Liste by=label|kind|land&outcome=
Beratungswege (/analytics/gremien, network, analytics-network.mjs)	Suchfilter; mit area wird scope=only erzwungen	source(steps), Status consulting bis postponed; Folge der Stationen aus steps (gleiche Gremien hintereinander einmal, mindestens 2 Stationen)	ein Ort: echte Gremiennamen, sonst gremiumType(); höchstens 18 Knoten, 60 Kanten, mindestens 2 je Kante, Median-Tage 0 bis 730; Liste by=edge&value=Von|Nach
Fraktionsprofil (/analytics/fraktionen, fraktionen, analytics-factions.mjs)	Suchfilter, to = heute	source(noPrivate, factions), nur factions <> '', je Vorgang und Fraktion eine Zeile	faction_advice macht „beschlossen“ zu „abgelehnt“; Fraktionen ab 10 Anträgen; Themenindex gegen alle Fraktionsanträge; Liste by=party&outcome=
Newsradar (/analytics/news, news, analytics-news.mjs)	Suchfilter; Standard 30 Tage zurück, 10 Tage voraus; noformal; n 12 (bis 30)	Stufe 1: alle Vorgänge über source(noPrivate, factions) mit Titel, Fakten kita_fee/plan über case_key, Vorbewertung prescore; Stufe 2: für die besten 300 topics.payload lesen	Kern nur aus zitierfähigen Sätzen der Regel-Zusammenfassung; Ausgang über outcomeOf, NO_RESULT (Platzhalter wie „nicht belegt“) ist kein Ausgang; NOT_NEWS filtert Besetzungen; Obergrenze je Gebiet; keine Liste

Ohne private Bauanträge (noPrivate) zählen Themennetz, Trends, Ortsvergleich, Fraktionsprofil und Newsradar; mit ihnen Ausbreitung, Beschlüsse, Beratungswege, Atlas, Fakten und Signale.

Teaser (/api/x/teaser, plenara-x-teaser.mjs, nur gated(), ohne Enterprise): 16 feste Auszüge für die Anwenderseiten (Signale der letzten 6 Monate, Hebesätze mit Median und Quartilen, 3 Meldungen, 5 steigende Begriffe, 4 häufigste Gremienübergänge, Ausgangsquoten), über atRevisionStale am Baustand, vom Webserver vorgeladen.

Mitgliedsgemeinde (where() in member-place.mjs): nur für Gebiete, deren Name mit Samtgemeinde, Verbandsgemeinde, Amt, Verwaltungsgemeinschaft oder (Gemeinde-)Verwaltungsverband beginnt; Reihenfolge: genau eine Mitgliedsgemeinde als Wort im Gremium (kein Verbandsgremium) → Ratsmuster im Gremium („Ortsgemeinderat X“) → „der/des Gemeinde/Stadt … X“ im Titel → genau ein Mitgliedsname als Wort im Titel. Rückgabe {key, label}; genutzt in Listen, Signalen, Fakten-Karte,

Ausbreitung, Newsradar, `fact-coverage`. Bei der Gebietsauswahl der Suche zählt der Schlüssel eines Mitglieds zu seinem Verband (`isPlace`).

Frontend: Seiten unter `components/ratsmonitor/pages/analytics/` (`AnalyticsPages.tsx` zeigt sie nur bei Stufe `enterprise`, sonst `AnalyticsLocked`; `FREE_ANALYTICS` ist leer); gemeinsamer Kopf `XKopf.tsx`, Befund `Befund.tsx`, Ergebnis je Adresse 10 Minuten gemerkt (`useAnalysisRun.ts`), Abfrage aus Suche und Filtern `useAnalyticsQuery.ts`; Adressen in `lib/routes.ts`.

6. Startseite: Karte und Trends (`scripts/startseite-trends.mjs`)

Job `startseite-trends` (05:10): ruft je Begriff aus `components/ratsmonitor/info/startseite-topics.json` die Ausbreitung `diffusion(q, level=city)` auf und legt je Kreis die früheste Gemeinde und den Verlauf je Monat ab (`system_state startseite-trends = {stand, topics}`; mit `--file` auch `public/data/startseite-trends.json`). Route `/api/startseite-trends` (nur `gated()`, Cache 1 h) liefert den Wert oder 404; die Startseite fällt dann auf die mitgelieferte Datei zurück. Der Job läuft vor `rule-texts`, rechnet also auf dem `x_cases` des Vortags.

7. Vorberechnung der Administration (`region-facts.mjs`, `region-series.mjs`, `admin-builds.mjs`)

Unabhängig von `x_cases`; liest `topics`. Die Adminseiten rechnen beim Öffnen nichts (`requirements/admin-performance-konzept.md`).

- **Je Gebiet:** `region_stats` (`count` und `FIGURES` aus `admin-data.mjs`: `firstEvent`, `lastEvent`, `contentSummaries`, `aiLabels`, `aiSummaries`, `ruleSummaries`, `ruleScoreSum`, `qualityPassed`, `pdfArticles`, `conflicts`, `textIssues`, `pendingAnalysis`) und `region_series` (`event_days`, `import_days`, `statuses`). Veraltet, wenn `region_revisions.revision` abweicht oder `v` nicht zu `FIGURES_VERSION` (6) bzw. `SERIES_VERSION` (1) passt. Rechnung kleinste Gebiete zuerst, in Stücken bis 25.000 Berichte, je Stück eine Transaktion; ältere Stände überschreiben nie neuere.
- **Wann:** nach dem Abruf eines Gebiets (1 s), `catchUp` nach den Abrufen (5 s je Schritt, höchstens 400), `POST /api/internal/admin-refresh (Runner, 5 s), Admin-Qualität (8 s), eine offene Adminseite (RegionCatchUp`), lokal `scripts/refresh-admin.mjs` (`--full` setzt `v=0`); der Dev-Server beim Start (`RM_SKIP_ADMIN_REFRESH=1` schaltet ab).
- **Sperre:** `admin-refresh-lease` in `system_state`; belegt → sofort `busy`; Gnadenfrist 20 s; Bauaufträge warten, solange ein Importjob läuft (`heavyJobRunning`).
- **Bauaufträge** (`admin-builds.mjs`, Ziele = Qualitätsprüfungen `quality:<check>`): Zustand in `system_state admin-build:<ziel>` (Bestand beim Start, Cursor, Teilsummen in `admin_agg`); je Schritt Stücke bis 25.000 Berichte, 8 s Budget, 150 ms Pause; jedes Stück ein Batch mit Bedingung „Cursor unverändert" (ein doppelt geschickter Schritt ändert nichts); Fortsetzen innerhalb 1 h, danach Neustart. Ergebnis über `writeStored` (gzip, base64, Teile zu 90.000 Zeichen unter `admin-stored:<name>#nnn`, mit `stockSum` und `content`-Revision); Hochrechnung in `admin-stored`, Qualität in `admin-quality-check`.
- **Lesen:** GETs schreiben nie und lesen `topics.payload` nie über mehr als ein Gebiet (`tests/admin-readonly.test.mjs`); Antworten mit schwachem ETag und Stand (`admin-http.ts`, `admin-stand.mjs`); die Gebietsliste in Teilen `summary`, `areas`, `static` (je Deploy, `STATIC_VER`), `area?id=`.

8. Social Media (`social-topics.mjs`, Admin „Social Media")

Liest die Karten der letzten 30 Tage direkt aus `search_cards` (`formal=0`), nicht aus `x_cases`. Bewertet rein regelbasiert (`fitScore` 0 bis 100: Themenwert aus `shared/social-calibration.json` und `social-ratings.json`, Streit im Vorgang aus `events[].result`, Alltagsnähe, Reichweite, Aktualität, Konkretheit, Verständlichkeit; Abzüge für Einzelstraße, Ortsteil-Gremium, Planungsverfahren). Höchstens 3 Vorschläge je Tag, je Ort und Tag einer, je Ort, Gruppe und Woche einer, Mindesteignung 40 %. `cleanTitle`, `matchTitle` und `evidenceOf` nutzt auch der Newsradar. Lernen: `node scripts/learn-social.mjs` nach Änderungen an `GROUPS`.

9. Befunde beim Schreiben dieser Dokumentation (17.10.2026)

1. **Stand der Signale nicht eindeutig:** `phase` kommt aus einer Unterabfrage `LIMIT 1` ohne `ORDER BY`; tragen mehrere Berichte eines Vorgangs verschiedene Werte, ist offen, welcher gilt.
2. **Zahl der Signale:** Code 11 Einträge, Kommentar 10; `docs/produkt/plenara-x-signale.md` führt Pflege und Einzelhandel als eigene Kategorien (im Code steckt Einzelhandel in `gewerbe`, Pflege fehlt; der Fakt `care` wird gebaut, aber nicht genutzt) und nennt den Cache-Schlüssel `x-signale-v5` (Code `v8`).
3. **Freigabe der Signale wird nicht durchgesetzt:** `rule-quality.json` hat Werte unter 90 % (`sig_tender`, `sig_waerme`, `sig_wasser`), die Abfrage liefert trotzdem alle.

4. ** `X_FACT_GATE` ** im Code: `plan`, `hebesatz`, `faction`, `annual_result`, `plan_use`, `fee_rate`, `heat_plan`, `grant`, `land_price`; `CLAUDE.md` nennt „derzeit plan, hebesatz, faction“. Das Tor greift nur im Atlas mit `facts=1`.
5. **Zugang:** `/api/analytics/diffusion` verlangt kein `feature:'analytics'` (die Startseite nutzt dieselbe Rechnung); `docs/produkt/plenara-x-ueberblick.md` nennt `/analytics/ueber` und `/analytics/tools` als frei, im Code gibt es die Routen nicht mehr (`FRONTEND.md` : keine Übersichtsseiten, 404).
6. **Verlauf aus einer Karte:** `steps`, `first_date`, `terms`, `private_build` kommen nur aus Zeitstrahl und Titel der Karte `card_id`; Kommentare und Überblick sagen „alle Karten eines Vorgangs“. Ob der Zeitstrahl der jüngsten Karte immer alle Gremien enthält, ist ungeprüft.
7. **Ungelesene Sammelpunkte und Routine** zählen mit, bis der Bericht gelesen ist (nur `search_cards.formal` filtert).
8. **Konzept veraltet:** `requirements/plenara-x-konzept.md` beschreibt `x_cases` mit anderen Spalten sowie `x_docs`, `x_stats` und `plenara-x-build.mjs`; so ist es nicht umgesetzt.
9. `server/repositories/analytics.ts` (`getAnalytics`, älterer Weg über `topics.payload`) hat keinen Aufruf in einer Route mehr.
10. Der Kommentar in `deploy/node/ratsmonitor-startseite-trends.service` nennt 04:50, der Zeitplaner 05:10.

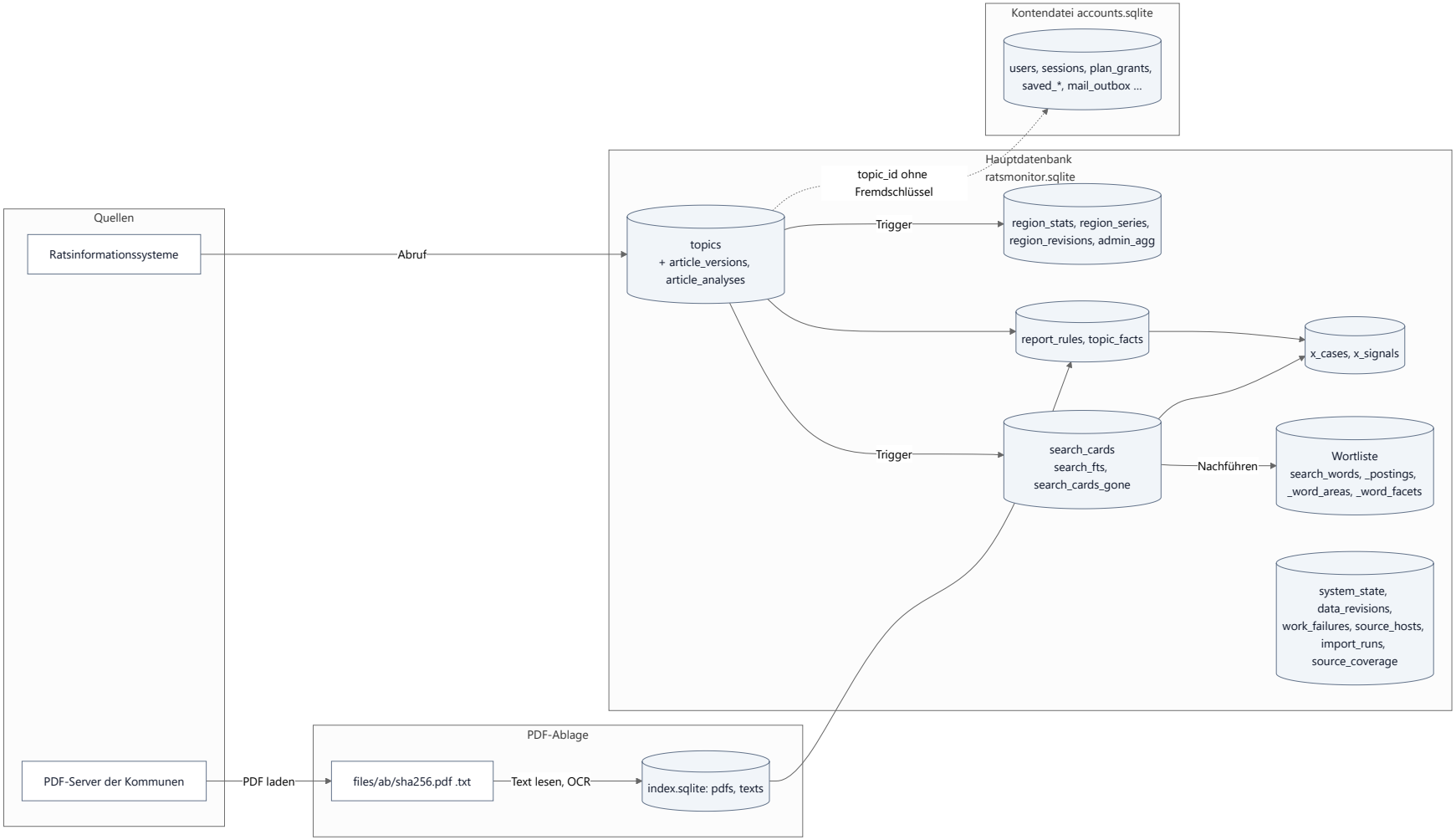
Datenmodell: Speicher, Tabellen, Trigger, Stände

Stand: 17. Oktober 2026, Migrationen `drizzle/0000` bis `0045`, `drizzle-accounts/0001` bis `0002`. Teil der technischen Dokumentation. Begriffe: Glossar. Kein Drizzle-ORM im Code: alle Tabellen sind rohes SQL in den Migrationen, alle Zugriffe `db.prepare(...)` über die D1-Schnittstelle.

1. Die drei Speicher

Speicher	Datei	Schnittstelle	Inhalt
Hauptdatenbank	`ratsmonitor.sqlite` (`DATABASE_FILE`); lokal die Miniflare-Datei unter `.wrangler/state/v3/d1/`	`env.DB` (D1-API, auf dem Node-Server über `server/node/d1-sqlite.mjs`), Migrationen `drizzle/`, Buchführung `d1_migrations`	Berichte, Suchkarten, Indizes, Regeln, Fakten, Vorgänge, Signale, Vorberechnung, Betriebszustand
Kontendatei	`accounts.sqlite` (`ACCOUNTS_FILE`); im Dev-Server in der Hauptdatenbank	`env.ACCOUNTS`, Migrationen `drizzle-accounts/`, Buchführung `accounts_migrations`	Konten, Anmeldesitzungen, Stufen, gespeicherte Suchen und Berichte, Postausgang
PDF-Ablage	Ordner `RM_PDF_STORE` mit `index.sqlite` und `files/`	`server/node/pdf-store.mjs` (nur Node)	Original-PDFs, Textfassungen, Stand je Adresse und Datei

Dazu lesen Skripte und Prozesse mit eigenen Verbindungen auf dieselbe Hauptdatenbank (WAL, `busy_timeout` 30 bis 60 s). Die Kontendatei ist von allen Jobs außer Webdienst und Kontojob abgeschirmt (`InaccessiblePaths` in den systemd-Einheiten) und gehört nie in Kopien, Export oder MCP.


Kontendatei accounts.sqlite
users, sessions, plan_grants,
saved_*, mail_outbox ...
Hauptdatenbank
ratsmonitor.sqlite
topic_id ohne
Fremdschlüssel
Quellen
Ratsinformationssysteme
Abruf
topics
+ article_versions,
article_analyses
Trigger
region_stats, region_series,
region_revisions, admin_agg
report_rules, topic_facts
x_cases, x_signals
Trigger
search_cards
search_fts,
search_cards_gone
Nachführen
Wortliste
search_words, _postings,
_word_areas, _word_facets
system_state,
data_revisions,
work_failures, source_hosts,
import_runs,
source_coverage
PDF-Ablage
PDF-Server der Kommunen
PDF laden
files/ab/sha256.pdf .txt
Text lesen, OCR
index.sqlite: pdfs, texts

Keine Fremdschlüssel in der Hauptdatenbank. Wird ein Bericht gelöscht (der Abruf tut das nie), löschen Trigger nur `search_cards` und `topic_facts`; Zeilen in `article_versions`, `article_analyses`, `report_rules`, `ai_work_items` und `ai_usage` bleiben stehen.

2. Tabellen der Hauptdatenbank

A. Quellenstand (schreibt der Abruf)

Tabelle	Zweck	Schlüssel, wichtige Spalten	schreibt	liest	Migration
`topics`	ein Bericht je Zeile; der fachliche Inhalt steht in `payload` (JSON, Abschnitt 5)	PK `id`; `region_id` (Standard `muenster`), `source` (`city/district`: die Ebene, nicht die Quelle), `event_date`, `updated_at`, `status`, `payload`; generiert `active = json_extract(payload,'$.identity.mergedInto') IS NULL` (VIRTUAL). Teilindizes `idx_topics_canonical_*` hängen an `active=1` (Abfragen müssen genau `active=1` schreiben); weitere `idx_topics_event_date`, `_region_updated`, `_inactive`, `_ai_local_*`	Abruf (`sync.ts` Upsert; nur `$.metadata` per `json_set`), Erstbefüllung (`seed.ts`, `apply-backfill.mjs`), Titelregel (`manual-analysis.mjs`), Regeln (`read-reports.mjs`), KI (`ai-local-work.mjs`, `ai-jobs.mjs`, `prepared-analysis.mjs`)	`repositories/topics.ts` (`/api/topics/[id]`, Artikelseite, `/api/dokument`), `monitor-search.mjs` (Stationen aus `events`), `regions.ts` (`/api/sources`), Admin (`admin-data.mjs`, `region-facts.mjs`), `x-cases.mjs` (Zeitstrahl), Export (`database-transfer.mjs`)	0000, 0001, 0020, 0026, 0028
`source_coverage`	Quellenstand je Gebiet	PK `region_id`; `payload` (`Coverage` plus `importHealth`, `window`, `oparlProbeAt`, `apiCheck`)	Abruf (jeder Versuch), Erstbefüllung	Abruf (Rückstellung), `/api/sources`, Suche (`coverage`), Admin, `scripts/import-gaps.mjs`	0001
`import_runs`	Protokoll der Läufe (Abruf, KI-Übernahme, Analysen)	PK `id`; `started_at`, `finished_at`, `status` (`running`, `completed`, `partial`, `failed`), `details` (JSON: mode, region, trigger, window, count); Indizes auf Start und `details.region`	Abruf (Start, Ende; Läufe älter als 30 Tage löschen), `ai-jobs.mjs`, `manual-analysis.mjs`	Admin (letzte 30), `import-trace.mjs`, `/api/internal/sync`, Nachtbericht, Wege zählen	0000, 0016
`article_versions`	frühere Fassungen eines Berichts als Diff (`diff-v1`)	PK `id`; `topic_id`, `captured_at`, `payload`	Abruf (bei Änderung), `prepared-analysis.mjs`, `ai-local-work.mjs`; gekürzt von `scripts/compact-versions.mjs`, `prune-versions.mjs`	Admin (Zählung), Export, Kürzungsskripte	0001
`article_analyses`	Ergebnisse der Analysen je Bericht als Verlauf	PK `id`; `topic_id`, `kind` (`summary`, `teaser`, `ai-label`, `rule-label`, `ai-local-attempt`), `method`, `input_hash`, `created_at`, `payload`	KI (`ai-local-work.mjs`, `ai-jobs.mjs`), Titelregel (`rule-label`), `prepared-analysis.mjs`, Modus `summaries`	`ai-jobs.mjs`, Admin, Export	0003, 0017

B. Suche (Projektion und Indizes)

Tabelle	Zweck	Schlüssel, Spalten	schreibt	liest	Migration
`search_cards`	Suchkarte je **aktivem** Bericht	PK `id` (= `topics.id`), dazu `rowid` (FTS, Wortliste); `region_id`, `date` (Stand heute), `next_date`, `status`, `label` (View), `title` (≤ 1000), `teaser` (≤ 1200), `gremium`, `search`, `search_exact`, `formal`; Indizes `_date`, `_status`, `_label`, `_facets(region_id,label,status,date)`, `_noformal` (dasselbe `WHERE formal=0`), `_next`	**nur Trigger** auf `topics`; `rollCardDates` (`date`, `next_date`); Nachfüll-Updates der Migrationen	Suche, `/api/sources`, Wortliste, `x-cases.mjs`, alle `analytics-*`, plenara.X, Kunden-MCP, Social Media, `server/accounts/account.mjs` (gespeicherte Berichte)	0006, 0010, 0014, 0017, 0019, 0024
`search_cards_gone`	Protokoll ersetzter und gelöschter Karten für das Nachführen der Wortliste	ohne PK; `card_rowid`, `id`, `region_id`, `label`, `status`, `search`	Trigger auf `topics`	`search-words.mjs` (`readGone`); geleert vom Nachführen, vom vollen Aufbau und von Migrationen	0015

Tabelle	Zweck	Schlüssel, Spalten	schreibt	liest	Migration
search_fts (virtuell)	FTS5-Volltextindex über search_cards.search, Trigram, external content	fts5(search, content='search_cards', content_rowid='rowid', tokenize='trigram')	Trigger trg_search_fts_* auf search_cards	Suche (Rückfall bei fehlenden Kandidaten)	0023
search_words	Wortliste: je Wort die Kartenzahl (501 = „mehr als 500“)	PK word; cards; vom Code zusätzlich hits_city, hits_district (Migration 0013 kennt sie nicht; der volle Aufbau legt die Tabelle neu an)	search-words.mjs (Abruf mit onlyIfBuilt, Skript refresh-search-words.mjs)	Suche (knownWords, candidateCards, precomputed*), Themennetz (tf-idf)	0013
search_postings	Karten-IDs je seltenem Wort (bis 500 Karten)	PK (word, card_id) WITHOUT ROWID	wie oben	Suche (Kandidaten)	0013
search_word_areas	Trefferzahl je Gebiet für häufige Wörter	PK (word, region_id); n	wie oben	Suche (vorberechnete Zahlen)	0013
search_word_facets	Trefferzahl je Ebene, Label, Status für häufige Wörter	PK (word, kind, label, status); n	wie oben	Suche	0013

C. Regeln und Auswertungen

Tabelle	Zweck	Schlüssel, Spalten	schreibt	liest	Migration
report_rules	Stand der Regeln je Bericht; gleicher key = nichts neu rechnen	PK topic_id; key, version (READ_VERSION), summary_issue, read_at; case_key (= region_id|titleKey(title)), item_kind, theme, subs (kommagetrennt), stage; Indizes _theme (WHERE item_kind='subject'), _case	read-reports.mjs	read-reports.mjs, x-cases.mjs, plenara.X, Newsradar, fact-coverage.mjs, Wege zählen	0032, 0035
topic_facts	Fakten je Bericht	PK (topic_id, kind, value); detail, method (facts-v27:text|title); Index (kind, value); Trigger trg_topic_facts_delete	read-reports.mjs (erst DELETE … method LIKE 'facts-%', dann Insert), Modus summaries	Fraktionsprofil, Newsradar, plenara.X (Fakten, Signale), x-cases.mjs (Fraktionen), fact-coverage.mjs	0025
x_cases	eine schmale Zeile je Sachvorgang für plenara.X	PK case_key; region_id, date, month, label, stage, subs, status, card_id, cards, unread; first_date, steps, days, vote, changed (0041); terms, private_build (0042); factions, faction_advice (0043); Indizes _region(region_id,date), _date, _label, _atlas(date,label,month,region_id,unread)	x-cases.mjs (DELETE, dann Insert in einer Transaktion), Schritt „Zahlen rechnen“; nie im Abruf	/api/x, /api/x/signale, /api/x/teaser, alle /api/analytics/*, fact-coverage.mjs, serve.mjs (warmhalten)	0036, 0041 bis 0045
x_signals	Signale je Vorgang vorberechnet	ohne PK; case_key, signal, art, phase; Indizes (signal, case_key), (case_key)	x-cases.mjs (buildXSignals)	/api/x/signale, warmhalten	0039

D. Vorberechnung der Administration

Tabelle	Zweck	Schlüssel, Spalten	schreibt	liest	Migration
region_revisions	Änderungszähler je Gebiet	PK region_id; revision	**nur Trigger** region_revision_topics_*	region-facts.mjs, region-series.mjs, admin-stand.mjs (stockSum, ETags), admin-builds.mjs, quality-check.mjs	0011
region_stats	Kennzahlen je Gebiet (FIGURES, FIGURES_VERSION 6) mit dem Zählerstand ihrer Berechnung	PK region_id; revision, computed_at, stats (JSON)	refreshRegionFacts (Abruf 1 s, catchUp, /api/internal/admin-refresh, scripts/refresh-admin.mjs)	admin-data.mjs, admin-stand.mjs, area-figures.mjs; Routen admin/areas, summary, coverage	0011
region_series	Tageswerte je Gebiet (Sitzungstag, Aufnahmetag, Zählung nach Status), SERIES_VERSION 1	PK region_id; revision, v, computed_at, event_days, import_days, statuses	wie oben	Routen admin/timeline, estimate, coverage	0016
admin_agg	Teilsummen laufender Bauaufträge (heute nur Liste host der Qualitätsprüfung)	PK (build, list, g, k) WITHOUT ROWID; n, a, b, c	admin-builds.mjs, quality-check.mjs	quality-check.mjs	0016

E. KI

Tabelle	Zweck	Schlüssel, Spalten	schreibt / liest	Migration
ai_work_items	Arbeitsposten der lokalen KI je Bericht	PK topic_id; input_hash, state (processing, completed, stale, failed …), agent, claim_token, lease_expires_at, attempts, last_error, read_state, updated_at, last_result_*, last_receipt; Index (state, lease_expires_at)	ai-local-work.mjs (list, claim, read, renew, release, submit, fail) über /api/admin/ai-local, MCP intern, scripts/ai-local.mjs; liest rule-texts.mjs (ai-counts), Debug	0028
ai_usage	Tokenverbrauch je KI-Versuch	PK id; topic_id, region_id, method, agent, model, kinds, created_at, outcome, Zähler je Art	shared/ai-usage.mjs (insertUsage); liest scripts/ai-usage.mjs; mcp-runtime.mjs verlangt die Tabelle	0007, 0029, 0030
ai_dispatches, ai_job_articles	Altweg der KI-Aufträge per Datei: Export-Zeitpunkt je Bericht und Art; eingefrorene Berichte eines Auftrags mit „übernommen“	PK (topic_id, kind) bzw. (job_id, topic_id)	nur ai-jobs.mjs (/api/admin/ai-job, scripts/ai-job.mjs); Manifest in system_state admin-ai-job	0005

F. Betrieb

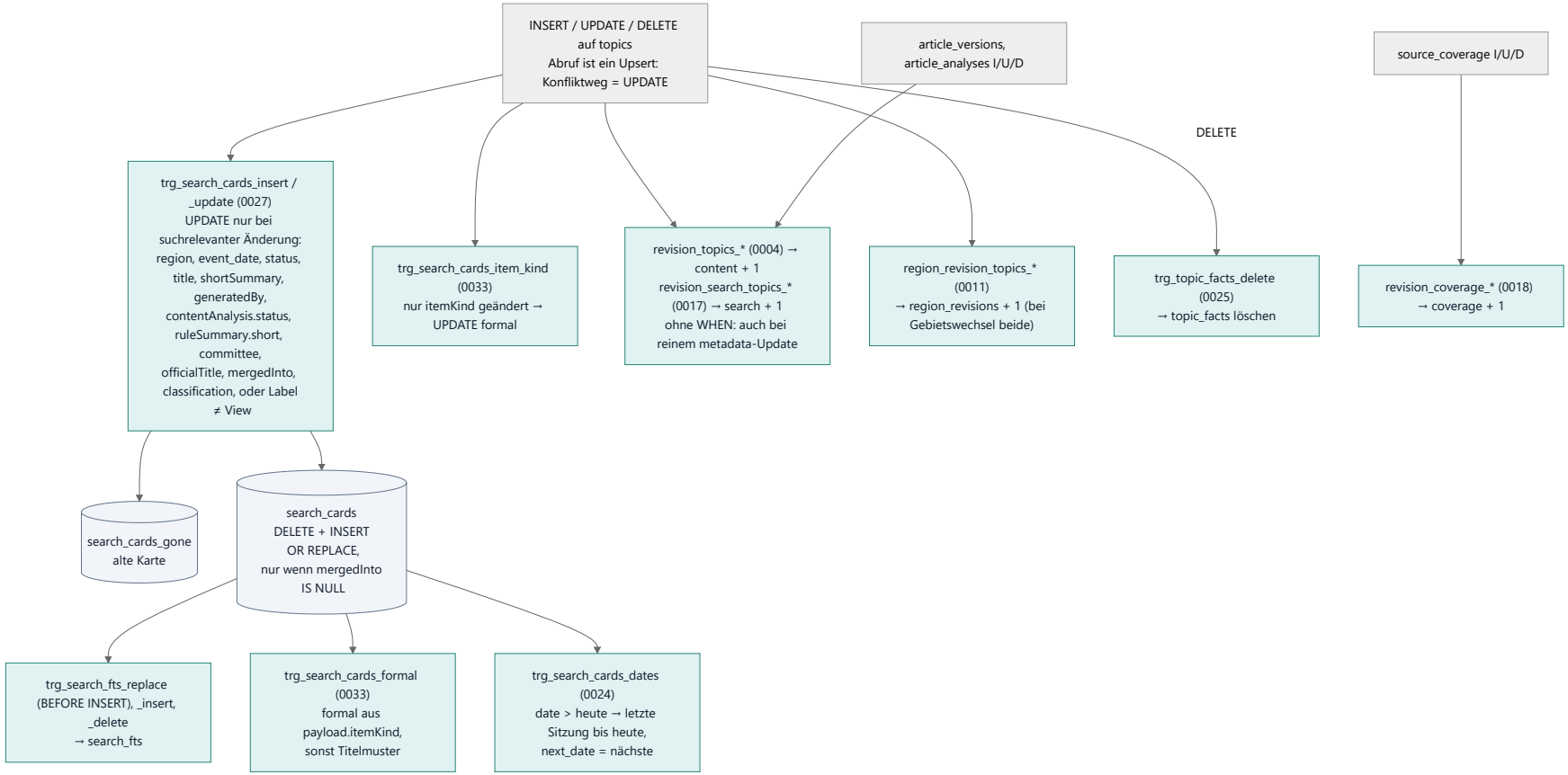
Tabelle	Zweck	Schlüssel, Spalten	schreibt / liest	Migration
system_state	Schlüssel-Wert-Speicher (Abschnitt 4)	PK key; value (meist JSON)	viele Module	0000
data_revisions	globale Datenstände	PK id ∈ {content, search, coverage}; revision	Trigger und einzelne Skripte (Abschnitt 3)	0004, 0017, 0018
work_failures	Fehlerspeicher je Schritt und Gegenstand	PK (step, key); host, class, reason, http, tries, first_at, last_at, next_try_at, event_date, origin, release; Indizes next_try_at, host	work-failures.mjs für Abruf (noteImport) und Ablage (failures.mjs); Nacharbeit, Altbestand, job.mjs; liest Debug, Altbestand, pipeline-worker.mjs, text.mjs	0040
source_hosts	Serverstufe je Server	PK host; state, fails, level, next_try_at, last_ok_at, last_fail_at, reason, updated_at	work-failures.mjs, failures.mjs, job.mjs; liest Abrufer, Debug, stats.mjs	0040
push_subscriptions	Web-Push-Abonnements	PK id; endpoint, auth, p256dh, created_at, last_sent_at	/api/push/subscribe, push.ts	0000
d1_migrations	Buchführung der Migrationen (wie Wrangler)	id, name UNIQUE, applied_at	scripts/node-migrate.mjs; serve.mjs prüft die Tabelle beim Start	(Code)

View `topic_effective_labels` (Fassung `0038`)

Spalten `id`, `label`, nur für `topics.active=1`. Rangfolge: Einordnung von Hand → KI-Label (`ai-local-content-v2`, Katalog `labels-v2`/`v3`, passende `sourceSignature`) → `payload.theme` (`themes-v3`, `evidence` = heutiger Titel; `offen` → `unklar`) → Titelregel (`title-rules-v*`, `evidence` = Titel). Alte Labels werden übersetzt. Gelesen nur von den Suchkarten-Triggern und den Nachfüll-Updates der Migrationen; die JavaScript-Entsprechung ist `storedClassification` (`shared/analysis-state.mjs`), Test `tests/effective-labels.test.mjs`. `THEMES_VERSION` steht fest im View: ein Wechsel braucht eine neue Migration.

3. Trigger-Kette und Datenstände

Die Ausführungsreihenfolge mehrerer Trigger auf demselben Ereignis legt SQLite nicht fest.


INSERT / UPDATE / DELETE
auf topics
Abruf ist ein Upsert:
Konfliktweg = UPDATE
article_versions,
article_analyses I/U/D
source_coverage I/U/D
DELETE
trg_search_cards_insert /
_update (0027)
UPDATE nur bei
suchrelevanter Änderung:
region, event_date, status,
title, shortSummary,
generatedBy,
contentAnalysis.status,
ruleSummary.short,
committee,
officialTitle, mergedInto,
classification, oder Label
≠ View
trg_search_cards_item_kind
(0033)
nur itemKind geändert →
UPDATE formal
revision_topics_* (0004) →
content + 1
revision_search_topics_*
(0017) → search + 1
ohne WHEN: auch bei
reinem metadata-Update
region_revision_topics_*
(0011)
→ region_revisions + 1 (bei
Gebietswechsel beide)
trg_topic_facts_delete
(0025)
→ topic_facts löschen
revision_coverage_* (0018)
→ coverage + 1
search_cards_gone
alte Karte
search_cards
DELETE + INSERT
OR REPLACE,
nur wenn mergedInto
IS NULL
trg_search_fts_replace
(BEFORE INSERT), _insert,
_delete
→ search_fts
trg_search_cards_formal
(0033)
formal aus
payload.itemKind,
sonst Titelmuster
trg_search_cards_dates
(0024)
date > heute → letzte
Sitzung bis heute,
next_date = nächste

Kurzform: `topics` → { `search_cards` (→ `search_fts`, `formal`, `date`) + `search_cards_gone` } · `data_revisions` `content` und `search` · `region_revisions` · bei DELETE `topic_facts`. `rollCardDates` ändert `search_cards` direkt und erhöht `search` und `content` selbst.

Datenstände (`data_revisions`)

Stand	erhöht von	geprüft von
`content`	Trigger auf `topics`, `article_versions`, `article_analyses`; `read-reports.mjs` (wenn gelesen), `search-card-dates.mjs`; Migrationen 0019, 0024, 0027, 0034, 0037, 0038	`CONTENT_REVISION_SQL` (`revision-cache.mjs`): `atRevision`/`atRevisionStale` als Standard (Admin-Daten, Gebietswerte, Ausbreitung), `processing-status.mjs`, `admin-stored.mjs`, Beleg in `ai-jobs.mjs` und `database-restore.mjs`
`search`	Trigger auf `topics` (0017, Startwert aus `content`), `search-card-dates.mjs`, dieselben Migrationen	`SEARCH_REVISION_SQL` (`search-words.mjs`; ersatzweise `content`): Suche (Stand in der Antwort, 409 bei Abweichung des Parameters `revision`), Gültigkeit der Wortliste (`system_state search-words`)
`coverage`	nur Trigger auf `source_coverage` (0018)	`SOURCES_REVISION_SQL` = `content` + `coverage`: `/api/sources`, `admin-stand.mjs` (geht in die Admin-ETags ein), Export
Bau von `x_cases`	`system_state x-cases` ({`builtAt, today, cases, future, unread, steps, factions, signals`}), geschrieben von `buildXCases`	`X_REVISION_SQL` (`x-cases.mjs`): `xCached` für `/api/x`, `/api/x/signale`, `/api/analytics/*`, `/api/x/teaser` (`atRevisionStale`: bis 20 Filter je Art, alter Wert bis 6 h, Rechnung im Hintergrund)
je Gebiet	`region_revisions` (Trigger)	`region-facts.mjs` rechnet nur abweichende Gebiete; `admin-stand.mjs` bildet `stockSum` und Stempel für ETags (`server/repositories/admin.ts`, 304 aus `admin-etag.mjs`)

Zwischenspeicher im Prozess: `server/services/read-cache.mjs` (30 s, 3 Einträge, geleert mit `invalidateReads()` nach Schreibvorgängen), `cachedSearch` in `monitor-search.mjs` (200 Einträge am Datenstand `search`), `atRevision*` in `revision-cache.mjs`.

4. Schlüssel in `system_state`

Bereich	Schlüssel	Zweck
Abruf	`admin-pipeline-job`, `pipeline-lease`	Abrufauftrag und seine Sperre (30 s)
	`import-lock`, `import-run:<gebiet>`	Importsperre (`<expiry>` exklusiv oder `shared`), Marke je Gebiet; auch von KI-Übernahme, Analysen, Titelregel genutzt
	`import-marks:<gebiet>`, `import-list:<gebiet>`	gelesene Sitzungen, gemerkte Sitzungsliste
	`import-debug:<gebiet>`	Anfragen des letzten Abrufs
	`latest-decision`	letzter erkannter Beschluss (Push, nur Münster)
	`seed-init-lock`, `regional-v5-…`, `nrw-<rev>`, `history-backfill-*`, `last-data-restore`	Sperren und Marken der Erstbefüllung, Nachladung, Wiederherstellung
Suche	`search-words`, `search-words-lease`	Stand ({`rowid, topId, revision, complete, counted, hasHits, words, postings, at`}) und Sperre der Wortliste
Regeln, plenara.X	`x-cases`	Baustand von `x_cases` (= X-Revision)
	`rule-texts-last`, `rule-texts-all`	Ergebnis je Schritt, Wunsch „Alle neu erstellen“
	`ai-counts`	KI-Stand (Zustände, neuer Text, ohne KI-Fassung)
	`pipeline-trace`	Wege der Berichte
	`startseite-trends`	vorgerechnete Trends der Startseite
Zeitplaner	`schedules`, `schedule-runs`, `schedule-now`, `schedule-stop`, `schedule-log`, `systemd-units`	Einstellungen, Läufe, Wünsche, Anhalten, Laufbuch (≤ 400), Zustand der Einheiten
Ablage	`pdf-store-stats`, `pdf-store-job`, `pdf-store-progress-chain`, `pdf-store-progress-ocr`, `pdf-store-runs`, `pdf-store-stop`, `pdf-store-count`, `pdf-store-heartbeat`, `pdf-store-errors-csv:N`, `pdf-daily-last`	Stand, Aufträge, Fortschritt je Bahn, Läufe (≤ 50), Anhalten, Zählwunsch, Lebenszeichen, Fehlerliste, Ergebnis des Tageslaufs
Nacharbeit, Altbestand	`nacharbeit-last`, `altbestand-requests`, `altbestand-last`, `altbestand-rework-last`	Ergebnisse und Wünsche
Speicher	`server-disk`, `server-disk-count`	Plattenbelegung, Messwunsch
Administration	`admin-refresh-lease`, `admin-build:<ziel>`, `admin-stored:<name>` (+ `#000`…), `admin-quality-check`, `admin-owner-v1`, `admin-todos-v1`	Sperre der Rechenschritte, Bauaufträge, gepackte Auswertungen (gzip, base64, Teile zu 90.000 Zeichen), Qualitätsprüfungen, alter Admin-Besitzer, To-do-Board
KI (alt)	`admin-ai-job`, `ai-job-lease`, `last-ai-apply`	Manifest, Sperre, Übernahmebeleg des Dateiwegs
Altlast	`coverage`	wird in `repositories/topics.ts` gelesen; kein Schreiber im Code gefunden

5. Das Payload eines Berichts (`topics.payload`)

Typ `StoredTopic` (`server/types.ts`) erweitert `TopicDetail` und `TopicCard` (`shared/types.ts`). Die Felder nach Herkunft:

Herkunft	Felder
Abruf (Leser und `refreshMetadata`)	`id`, `source`, `regionId`, `public`, `title`, `officialTitle`, `status`, `category`, `committee`, `eventDate`, `updatedAt`, `reference`, `referenceStatus`, `documents[]`, `events[]` (`date`, `committee`, `status`, `description`, `url`, `result`, `created`, `modified`, `attendance`, `decision`), `relevanceReason`, `sourceUrl`, `sourceText`, `documentText`, `hasDocumentText`, `documentSource`, `documentIssue`, `paperType`, `sourceData` (`version`, `method`, `fetchedAt`, `records[]`, `detailStatus`, `issues`), `identityLinks`, `identityRecords`, `identity` (`version`, `mergedInto?`, `conflict?`), `metadata` (`version`, `firstImportedAt`, `sourceModifiedAt`, `lastFetchedAt`, `lastProcessedAt`), `quality`; automatischer Text `shortSummary`, `longSummary`, `generatedBy: 'Automatischer Quellenüberblick'`
Regeln (ohne Modell)	`classification` (Titelregel, `method` `title-rules-v7`, `version` `labels-v3`, `evidence` = Titel), `labelAssessments.rule`, `analysisFeatures` (Titelregel, `manual-analysis.mjs`); `itemKind`, `theme` (`id`, `subs`, `source`, `version`, `evidence`), `stage`, `ruleSummary` (`short`, `long`, `score`, `generatedBy`, `generatedAt`, `source`), `ruleSummaryTried` (alt; wird gelöscht), `summaryAttemptedAt`, `summaryMethod`, `summaryIssue` (`read-reports.mjs`, Modus `summaries`)
KI	`shortSummary`, `longSummary` (überschrieben), `generatedBy: 'KI-Zusammenfassung'` (Altfälle mit Zusatz), `summaryGeneratedAt`, `summaryMethod`, `summaryModel`, `contentAnalysis` (seit 0021 ohne Text; der Text steht in `article_analyses` `summary`), `labelAssessments.ai` (`Classification` mit `status`, `sourceSignature`), `aiAttempts`, `metadata.lastProcessedAt`
Von Hand	`labelAssessments.review` (`primary`, `reviewer`, `reviewedAt`, `reason`, `status?`, `sourceSignature?`); gelesen vom View und von `storedClassification`; ein Schreiber im Code wurde nicht gefunden (nur Tests)
entfernt	`weightedKeywords` (0019), `summaryEvidence` (0022, wo `contentAnalysis.evidence` vorhanden ist)

`preserveArticleContent` (`shared/article-record.mjs`) behält beim erneuten Abruf `contentAnalysis`, `labelAssessments`, `metadata`, `ruleSummary` und vorhandene KI-Texte; ändert sich die Quelle (`analysisSignature`), werden `contentAnalysis` und `labelAssessments.ai` `stale`.

6. Kontendatei (`drizzle-accounts/`)

Geheimnisse nur als SHA-256, Adressen nur dort im Klartext, wohin versendet wird, keine IP-Adressen.

Tabelle	Zweck
`users`	Konten (`id` `u_…`) mit bestätigter E-Mail, Name, Organisation, Kundentyp, Status `active`/`blocked`
`user_roles`	Rollen je Konto (derzeit nur `admin`)
`plan_grants`	Freischaltungen der Stufen `basic`/`pro`/`enterprise` mit Quelle `manual`/`beta`/`stripe` und Laufzeit; ohne Zeile gilt Gast; `end_notice_at` vermerkt den Hinweis vor dem Ablauf
`consents`	Zustimmungen (`agb`, `alter18`) je Fassung mit Zeitpunkt, Widerruf, Fassung der Datenschutzerklärung
`invites`	Einladungen je E-Mail mit Stufe, Laufzeit, Notiz, Einladendem, Ablauf
`login_requests`	Zählprotokoll der Anmeldeanfragen (HMAC von Adresse und IP) für die Begrenzung; bereinigt vom Kontojob
`login_flows`	laufende Anmeldeabläufe mit Versuchszähler und Ablauf (Flow-Cookie)
`login_tokens`	Einmallinks und -codes (`id` = Hash des Tokens, `code_hash` aus Zweck, Flow und Code) für `login`, `signup`, `delete`, `export`, mit Ziel `next` und `used_at`
`sessions`	Anmeldesitzungen (Hash des Cookie-Werts), gleitender und absoluter Ablauf, Admin-Host-Kennzeichen, Gerät; seit 0002 `admin_verified_at` (zweiter Faktor, 12 h) und `view_tier` („Ansehen als“)
`saved_searches`	gespeicherte Suchen je Konto mit Zustand, Abfrage, Mail-Benachrichtigung
`saved_articles`	gemerkte oder gefolgte Berichte mit Momentaufnahme und letztem Status; `topic_id` verweist ohne Fremdschlüssel auf die Hauptdatenbank
`user_settings`	Wochenmail (an/aus, Wochentag), Kalenderauswahl als JSON
`calendar_feeds`	geheime Kalender-Abo-Kennungen je Konto mit Widerruf
`auth_events`	Protokoll sicherheitsrelevanter Ereignisse
`admin_audit`	ändernde Admin-Zugriffe, stundenweise verdichtet (`UNIQUE (user, path, action, hour)`)
`mail_outbox`	Warteschlange ausgehender Mails mit Status, Versuchen, Verfall, Kontingentklasse; nach dem Versand werden Adresse und Inhalt gelöscht
`dev_mail`	im Dev-Betrieb abgefangene Mails (Postfach `/konto/postfach`)
`user_totp`	TOTP-Schlüssel der Admins, AES-GCM-verschlüsselt mit `ACCOUNTS_SECRET`; `last_step` verhindert doppelte Codes

7. PDF-Ablage (`index.sqlite`)

Tabelle	Spalten	Zweck
`pdfs`	PK `url`; `sha256`, `bytes`, `pages`, `status` (`ok`, `error`, `unsupported`, `text-only`), `http`, `error`, `fetched_at`, `region_id`, `topic_id`, `doc_type`, `formal`, `event_date`, `role` (`other` = Anlage, leer = Hauptdokument); Indizes `sha`, (`status, fetched_at`)	eine Zeile je **Adresse**

Tabelle	Spalten	Zweck
texts	PK sha256; status, method (text, ocr, mixed), version (TEXT_VERSION), pages, ocr_pages, ocr_needed, chars, error, made_at, head_type; Teilindex WHERE ocr_needed>0	eine Zeile je **Datei** (Textfassung)
meta	key, value	doc-type-version, failures-seeded
text_hashes, facts_keys		Altlasten der Skripte summaries.mjs und facts.mjs (ersetzt durch report_rules)
ocr_refetch, ocr_slow		Altlasten, einmal in den Fehlerspeicher übertragen, danach nur noch ohne Fehlerspeicher geschrieben

Dateien `files/<sha256[0..1]>/<sha256>.pdf` und `.txt` (Seiten getrennt durch `\f`); gleiche Bytes liegen einmal, auch unter mehreren Adressen. Das Laufprotokoll liegt nicht hier, sondern in `system_state` `pdf-store-runs`.

8. Migrationen

- Reihenfolge: `scripts/node-migrate.mjs` sortiert die Dateinamen; von den beiden Dateien mit Nummer `0017` läuft `0017_search_cards_label_version.sql` vor `0017_search_formal_revision.sql`. Buchführung in `d1_migrations`; eine lokal migrierte Datenbank gilt als auf demselben Stand wie der Server.
- Dauer auf echten Daten (lokale Kopie mit 941.000 Karten, Server etwa das Anderthalbfache): 0017 142 s (Spalte `formal`), 0019 45 s, 0020 28 s, 0023 43 s (+650 MB FTS), 0024 3,8 s; Einzelheiten und die Reihenfolge nach dem Einspielen (Wortliste `--full`, Gebietswerte neu, `rule-texts --all`) in `docs/betrieb/node-server.md`, Abschnitt „Migrationen“.
- Die Migrationen 0019, 0027, 0034, 0037 und 0038 löschen den Stand der Wortliste und leeren `search_cards_gone`, damit der nächste Lauf voll aufbaut; 0034 und 0037 setzen die Labels aller Karten neu.
- `deploy/node/deploy.sh` prüft nur (`--check`) und bricht bei fehlenden Migrationen ab; eingespielt wird von Hand mit angehaltenem Importprozess. Die Kontendatei migriert das Skript selbst (nur additiv, vorher verschlüsselte Sicherung).

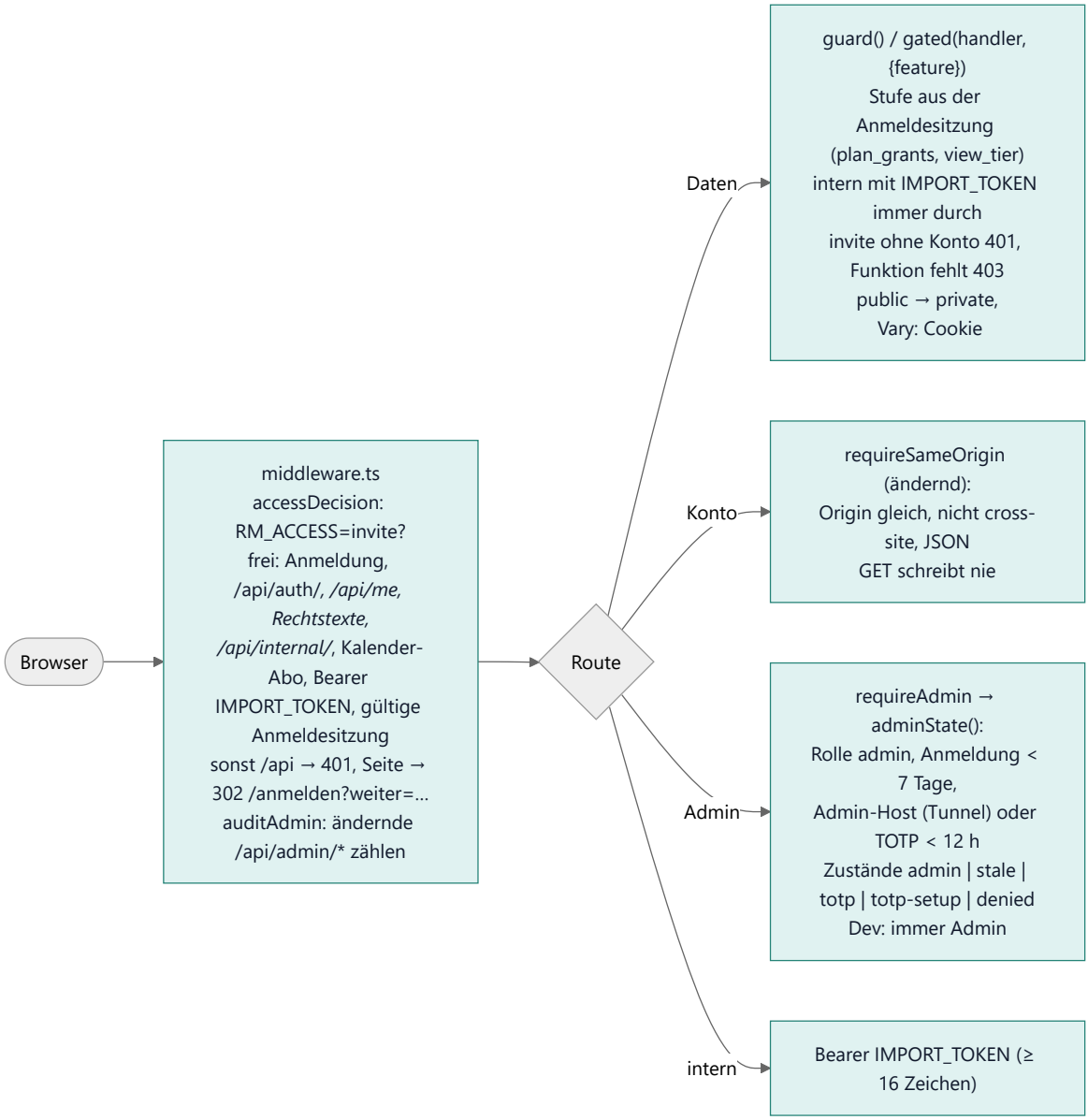
9. Befunde beim Schreiben dieser Dokumentation (17.10.2026)

1. `labelAssessments.review` (Einordnung von Hand) hat keinen Schreiber im Code; nur View, `storedClassification` und Tests lesen es.
2. `system_state` `coverage` wird gelesen (`repositories/topics.ts`), aber nirgends geschrieben.
3. `search_words.hits_city` / `hits_district` legt nur der Code an; auf einer Datenbank, die nur Migrationen gesehen hat, fehlen sie, bis `schemaOk` den vollen Aufbau auslöst.
4. `scripts/pdf-store/summaries.mjs` und `facts.mjs` ruft kein Job mehr auf; sie schreiben aber noch `topics`, `article_analyses`, `topic_facts`, `data_revisions` sowie `text_hashes` / `facts_keys`. Fakt-Art `funding` (Kommentar in 0025) gibt es im heutigen `factsOf` nicht mehr.
5. `requirements/plenara-x-konzept.md` beschreibt `x_cases` mit `id`, `member_id`, `routine`, `text_state` sowie `x_docs`, `x_stats` und `plenara-x-build.mjs`; so ist es nicht umgesetzt.
6. Zwei Schreibweisen des Vorgangsschlüssels: gelesene Berichte `titleKey` (Satzzeichen zu Leerzeichen), ungelesene `lower(title)` (`x-cases.mjs`, `plenara-x-query.mjs`); derselbe Titel mit Satzzeichen kann so vorübergehend zwei Vorgänge ergeben.

Zugang, Konten, Oberfläche, KI-Weg, Tests

Stand: 17. Oktober 2026, gelesen am Code. Teil der technischen Dokumentation. Begriffe: Glossar. Konzept der Konten: `requirements/konten-konzept.md`; Frontend-Karte: `components/ratsmonitor/FRONTEND.md`; KI-Vertrag: `requirements/ai-processing.md`.

1. HTTP-Schicht: wer darf was



- **`middleware.ts`** ruft `accessDecision` (`server/accounts/access.mjs`): die Zugangssperre greift nur bei `RM_ACCESS=invite` und nicht im Dev; frei sind statische Dateien, Anmeldung, `/registrieren`, `/api/auth/*`, `/api/me`, Rechtstexte, `/api/internal/*`, `/api/calendar/feed/*`, `/api/dev/*`, Bearer `IMPORT_TOKEN` und gültige Anmeldesitzungen. Für `/admin`, `/api/admin`, `/api/auth`, `/api/me`, `/anmelden`, `/konto` setzt sie `private, no-store` und `X-Robots-Tag: noindex`. `auditAdmin` zählt ändernde Admin-Anfragen je Konto, Pfad und Stunde in `admin_audit`.
- **`server/services/session.ts`:** `requestTier()` liest die Stufe aus der Anmeldesitzung (ohne Kontendatei `accounts:false`, dann keine Stufengrenzen: Cloudflare-Build); `guard()` lässt interne Aufrufe immer durch, antwortet bei `invite` ohne Konto 401 und bei fehlender Funktion der Stufe 403; `gated(handler, {feature:'analytics'|'calendar'})` umhüllt eine Datenroute und macht `public` zu `private` mit `Vary: Cookie`; `adminState()` liefert `admin`, `stale` (Anmeldung älter als 7 Tage), `totp`, `totp-setup` oder `denied`; der alte Kopfzeilenweg (`Oai-Authenticated-User-*` aus Caddy, `admin-owner-v1`) nur mit `ADMIN_LEGACY_HEADER=1` oder solange keine Kontendatei eingerichtet ist.

- **Admin-Routen:** `adminRead()` = `requireAdmin` + `readResponse` mit schwachem ETag (304 bei passendem `If-None-Match`, ohne den Body zu bauen; `admin-etag.mjs`); Kopfzeilen `private, no-cache`; `/api/admin/static?v=` für immer im Cache.
- **Kontorouten:** `requireSameOrigin` (`server/accounts/http.mjs`): `Origin` gleich, nicht `cross-site`, `Content-Type` JSON, sonst 403 bzw. 415. Cookies `__Host-rm_session` und `__Host-rm_flow` (https), `rm_session` im Tunnel, `rm_dev_session` im Dev. `clientIp` nimmt den letzten Eintrag von `X-Forwarded-For`, IPv6 auf /64 gekürzt; gespeichert wird nie eine IP, nur HMACs für Grenzen.

API-Routen (78 `route.ts` unter `app/api/`)

Daten (öffentlich, mit `guard`/`gated`; Gäste gekürzt)

Route	Zweck
`GET /api/search`, `/api/search/coverage`	Suche (auch NDJSON-Strom), Gebiete mit Berichten je Ebene
`GET /api/sources`	Quellen und Abdeckung
`GET /api/topics/[id]`	ein Bericht im Detail (folgt `mergedInto`)
`GET /api/dokument`	Originalunterlage beim Klick durchreichen (`document-proxy.mjs`: nur https, nur öffentliche Ziele, ohne zu speichern)
`GET /api/startseite-trends`	vorgerechnete Startseite (404 = mitgelieferte Datei nehmen)
`GET /api/x`, `/api/x/signale`	plenara.X (`feature:'analytics'`); `/api/x/teaser` ohne Enterprise
`GET /api/analytics/{compare,decisions,fraktionen,graph,network,news,trends}`	Tools (`feature:'analytics'`); `diffusion` nur `gated()`
`GET /api/calendar`, `/api/calendar/ics`, `/api/calendar/feed/[token]`	Sitzungstermine als JSON und ICS (`feature:'calendar'`), Abo mit Token statt Cookie
`/api/push/config`, `/api/push/latest`, `POST|DELETE /api/push/subscribe`	Web-Push (eigene Herkunftsprüfung)

Konto (`POST` mit `requireSameOrigin`)

Route	Zweck
`/api/auth/link`, `peek`, `verify`, `code`	Anmeldelink anfordern (immer 202), Link prüfen ohne Verbrauch, Link einlösen, Code einlösen (Flow-Cookie)
`/api/auth/logout`, `refresh`	abmelden, Anmeldesitzung verlängern (GET schreibt nie)
`/api/auth/totp`, `totp/setup`, `totp-login`	zweiter Faktor der Admins (12 h), Einrichtung nur im Tunnel, Anmeldung mit Adresse und App-Code (`LOGIN_TOTP=1`)
`GET /api/me`	Stufe, Rollen, Rechte, Zugangsmodus
`/api/me/searches`, `articles`, `settings`, `sessions`, `calendar-feed`, `import`, `export`, `delete`, `view-tier`	gespeicherte Suchen und Berichte, Einstellungen, Geräte, Abo-Link, Browserdaten übernehmen, Datenauskunft, Konto löschen (mit Code), Admins „ansehen als"

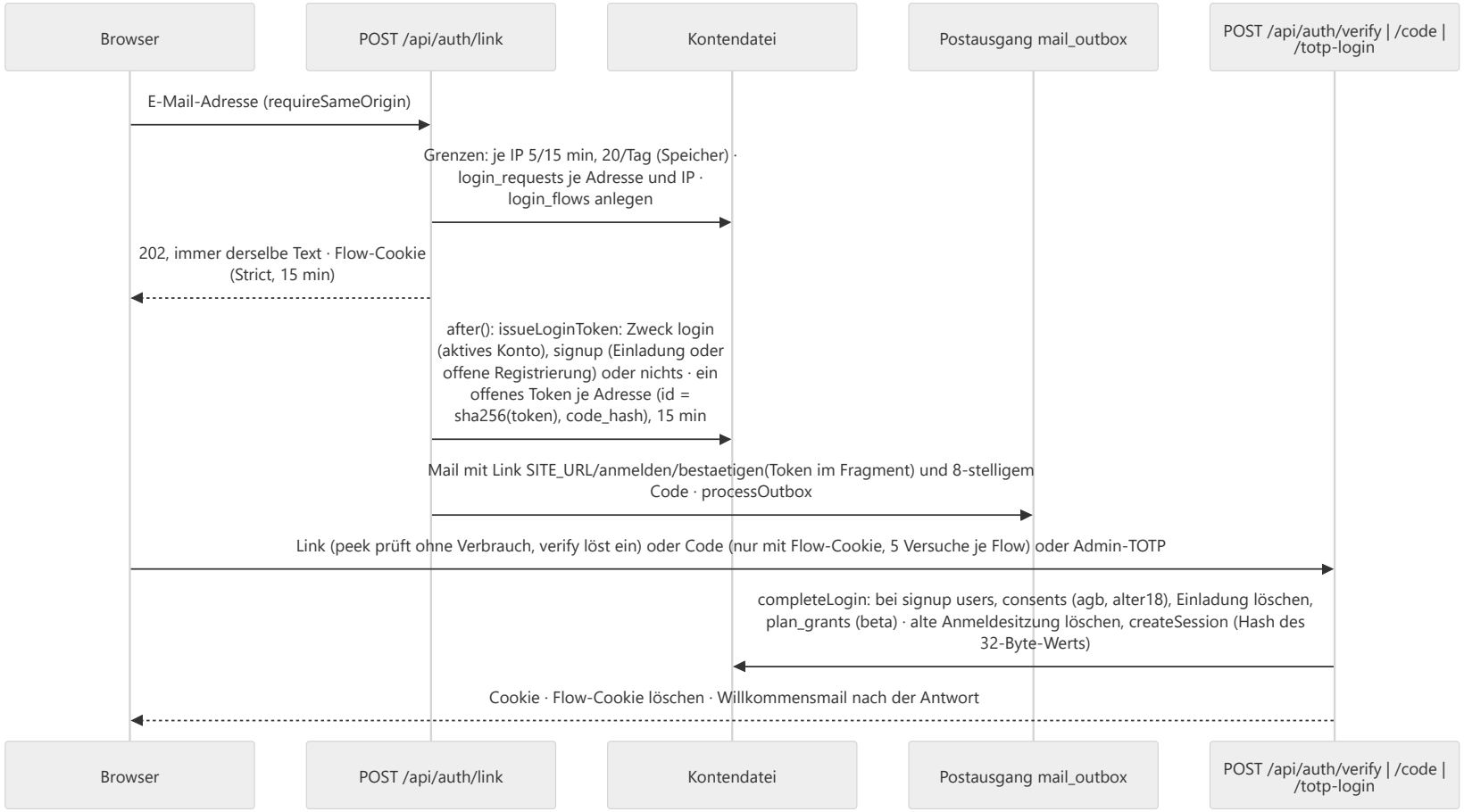
Administration (29 Routen, alle `requireAdmin`): `ai-local`, `ai-job`, `altbestand`, `analyse`, `area`, `areas`, `static`, `atlas`, `coverage`, `timeline`, `summary`, `claim` (alt), `database` (Export), `debug`, `run-debug`, `estimate`, `export` (CSV), `import`, `pipeline`, `pdf-ablage`, `pdf-ablage/fehler`, `prepared-analysis`, `quality`, `refresh`, `review`, `social`, `speicher`, `todos`, `zeitplaene`.

Intern (Bearer `IMPORT_TOKEN`): `GET /api/internal/health` (Hauptdatenbank, Kontendatei, Migrationen, Mails, Zugang; vom Deploy geprüft), `POST /api/internal/outbox` (Postausgang, jede Minute vom Webserver), `/api/internal/sync` (älterer Abrufweg), `/api/internal/admin-refresh` (Rechenschritte für den Runner).

Dev (sonst 404): `POST /api/dev/login`, `GET /api/dev/mails`, `PATCH /api/dev/tier`.

2. Konten und Anmeldung (`server/accounts/`)

Ohne Passwort: E-Mail-Link oder achtstelliger Code; Tokens, Codes und Anmeldesitzungen nur als SHA-256 gespeichert; Antwort und synchrone Arbeit von `/api/auth/link` sind für alle Adressen gleich, alles Adressabhängige läuft nach der Antwort (`after()`).


Browser
POST /api/auth/link
Kontendatei
Postausgang mail_outbox
POST /api/auth/verify | /code | /totp-login
E-Mail-Adresse (requireSameOrigin)
Grenzen: je IP 5/15 min, 20/Tag (Speicher) · login_requests je Adresse und IP · login_flows anlegen
202, immer derselbe Text · Flow-Cookie (Strict, 15 min)
after(): issueLoginToken: Zweck login (aktives Konto), signup (Einladung oder offene Registrierung) oder nichts · ein offenes Token je Adresse (id = sha256(token), code_hash), 15 min
Mail mit Link SITE_URL/anmelden/bestaetigen(Token im Fragment) und 8-stelligem Code · processOutbox
Link (peek prüft ohne Verbrauch, verify löst ein) oder Code (nur mit Flow-Cookie, 5 Versuche je Flow) oder Admin-TOTP
completeLogin: bei signup users, consents (agb, alter18), Einladung löschen, plan_grants (beta) · alte Anmeldesitzung löschen, createSession (Hash des 32-Byte-Werts)
Cookie · Flow-Cookie löschen · Willkommensmail nach der Antwort
Browser
POST /api/auth/link
Kontendatei
Postausgang mail_outbox
POST /api/auth/verify | /code | /totp-login

- **Anmeldesitzung** (`sessions.mjs`): im Cookie ein 32-Byte-Zufallswert, in der Datenbank nur sein Hash; Laufzeiten Nutzer 30 Tage Leerlauf und 90 Tage absolut, Admin-Host 12 h und 7 Tage; verlängert über `POST /api/auth/refresh`. Stufe nur aus `plan_grants` (`effectiveTier`); ohne Zeile Gast (`ACCOUNT_DEFAULT_TIER`).
- **Stufen** (`shared/plans.mjs`, eine Quelle für Server und Oberfläche; `docs/produkt/stufen.md` wird von Hand mitgezogen):

Stufe	Treffer	Filter	Merkliste / Suchen / Benachrichtigungen	Mails	Kalender	plenara.X
`guest`	10	nein	0 / 0 / 0	0	nein	nein
`basic`	alle	ja	1 / 1 / 1	1	nein	nein
`pro`	alle	ja	100 / 100 / 100	1	nein	nein
`enterprise`	alle	ja	100 / 100 / 100	5	ja	ja

  Preise sind Platzhalter (`PRICES` alle `null`, „Preis folgt"). Gastsuche (`guest-search.mjs`): erste Seite, höchstens 10 Treffer, gekürzt nach dem Zwischenspeicher, auch im Strom.
- **Admin:** Rolle `admin` in `user_roles`; `admin='ok'` nur bei Anmeldung jünger als 7 Tage **und** entweder Admin-Host per http (`ADMIN_HOSTS`, Standard Tunnel `127.0.0.1:8080`) oder bestätigtem TOTP (`admin_verified_at` jünger als 12 h). TOTP nach RFC 6238 (SHA1, 30 s, 6 Ziffern), Schlüssel AES-GCM-verschlüsselt in `user_totp` mit `ACCOUNTS_SECRET`; Einrichten nur im Tunnel oder per Kommandozeile `totp-setup`; 20 Fehlversuche am Tag. Admins können die Seite „ansehen als" andere Stufe (`sessions.view_tier`, höhere Stufen nur mit voller Admin-Freigabe).
- **Postausgang** (`mail.mjs`, Vorlagen mit Maskierung in `mail-templates.mjs`: login, signup, welcome, code_locked, delete_code, deleted, export_code, exported, invite, tier_ending, inactive, operator): `sendMail` schreibt in `mail_outbox` (Kontingente je Stunde login 100, signup 20, account 50); `processOutbox` nach der Antwort, jede Minute über `POST /api/internal/outbox` und beim Start; bis 3 Versuche, danach werden Adresse und Inhalt gelöscht. Transporte `http` (POST mit `MAIL_API_KEY` an `MAIL_API_URL`), `log` (verschickt nichts; die Anmeldeseite bietet dann keine Mail-Anmeldung), `outbox` (Dev-Postfach `dev_mail`).
- **Werkzeuge:** `scripts/accounts/manage.mjs` (auf dem Server `ratsmonitor-accounts`): `invite`, `create`, `role`, `tier`, `block`, `unblock`, `revoke`, `delete`, `export`, `list`, `stats`, `login-link` (Notzugang), `reapply-deletions`, `totp-setup`, `totp-reset`; Adressen nur über `--email-stdin`. `maintenance.mjs` (Job `accounts`, 00:10): Fristen, „Testzugang endet" 14 Tage vorher, Hinweis bei Inaktivität, Postausgang prüfen; danach verschlüsselte Sicherung (`accounts-backup.sh`, age, 7 Stände). `dev-setup.mjs` beim Dev-Start: Kontotabellen in die lokale Datenbank, Testkonten `basic@`, `pro@`, `enterprise@`, `admin@plenara.test` (`RM_SKIP_ACCOUNTS=1` schaltet ab); anmelden ohne Mail über `/api/dev/login` oder den DEV-Umschalter in der Kopfzeile.
- Grenzen im Speicher (`rate-limit.mjs`): Gleitfenster je Name und Schlüssel, nach einem Neustart leer.

Tests: `tests/accounts-login.test.mjs`, `accounts-data.test.mjs`, `accounts-rules.test.mjs` (Hilfen `tests/helpers/accounts.mjs`).

3. Oberfläche

Aufbau

- `app/layout.tsx`: Metadaten und Open Graph mit `SITE_URL`, Schriften (IBM Plex Sans), Dunkelmodus-Skript.
- `app/(monitor)/layout.tsx` rendert `{children}` und daneben `<MonitorApp/>`. Die Seitendateien unter `app/(monitor)/*/page.tsx` sind Stubs (`return null`) mit Metadaten; gerendert wird clientseitig in `components/ratsmonitor/App.tsx` (Pfadprüfung `isKnownPath`, Seitenmodule nachgeladen). `/beschluss/[id]` liest zusätzlich serverseitig `getTopic` für Vorschau und Metadaten. Neue Route: Stub anlegen und in `info/InfoPages.tsx` bzw. `pages/PersonalPage.tsx` eintragen (`FRONTEND.md`).

Pfad	Zweck
`/`	Startseite (`pages/OverviewPage.tsx`): Suche, Landkarte, Trefferliste; lädt drei Abfragen vorab
`/suche?q=…&ort=…&thema=…`	Suche in der Adresse (`lib/searchUrl.ts`, `state/search.tsx`)
`/beschluss/[id]`	Artikelseite eines Berichts (`pages/DetailPage.tsx`); `/thema/[id]` leitet dorthin
`/analytics/{atlas,signale,fakten,trends,beschluesse,graph,vergleich,gremien,fraktionen,news}`	plenara.X (`pages/analytics/`), sonst 404; Methodik unter jeder Auswertung

Pfad	Zweck
/anwender/[slug]	Branchen- und Anwenderseiten (info/BranchenPages.tsx, Daten info/content.ts)
/konto/{profil,suchen,artikel,kalender} (+ postfach nur Dev)	Kontoseiten (pages/PersonalPage.tsx)
/anmelden, /anmelden/bestaetigen, /registrieren → /anmelden	Anmeldung (info/AccountPages.tsx)
/preise, /faq, /videos, /datenabdeckung, /funktionen/benachrichtigungen, /kontakt	Infoseiten (info/InfoPages.tsx)
/impressum, /datenschutz, /agb, /widerruf	Rechtstexte (pages/LegalPage.tsx, Fassungen shared/legal-versions.mjs)
/admin?seite=…	Administration (app/admin/page.tsx)

- ** components/ratsmonitor/ :** App.tsx (Provider, Routing), components/ (Kopfzeile, Suche, Filter, map/MapPanel.tsx mit lib/geo/mapEngine.ts, results/), pages/ (Detail, Konto, Rechtliches, analytics/), info/ (Infoseiten, content.ts), lib/ (account.ts, fetchJson.ts, routes.ts, tier.ts, entitlements.ts, place.ts, searchLogic.ts), state/ (search.tsx, account.tsx, data.tsx, ui.tsx), menu/MainMenu.tsx. Datei- und Klassenverzeichnis: INDEX.md (nur per grep, neu mit node scripts/frontend-index.mjs).
- **Datenzugriff:** kein einzelner Client. lib/fetchJson.ts (fetchJson, readJson, ApiError mit deutschen Fehlertexten), lib/account.ts api() für /api/me/* (optimistisch mit Rücknahme, Abgleich zwischen Tabs), die Suche über state/search.tsx; daneben rohes fetch in vielen Komponenten.
- **Stile:** app/globals.css (Tailwind 4, Tokens; nicht ändern), app/ratsmonitor.css (App, Block „Farbkonzept“ Petrol klar mit --rm-*), app/ratsmonitor-info.css (ri-*), app/design-styles.css (wirkt auch auf Admin und Analysen), app/admin.css, app/analyse-admin.css. Designregeln und Prüfbefehle in FRONTEND.md.
- **Kalender:** calendar-events.mjs (höchstens 200 Gebiete, 100 Tage), calendar-ics.ts; Abo-Link mit Token aus calendar_feeds.
- **Web-Push:** public/sw.js holt bei push /api/push/latest; Server push.ts (dispatchDecisionPush aus dem Abruf, nur Münster); im Frontend wurde keine Registrierung des Service Workers und kein Aufruf von /api/push/* gefunden (Abschnitt 6).

Administration (/admin?seite=…, components/admin-*.tsx)

Reiterliste in admin-chrome.tsx, Aliase pdf-import → pdf, jobs → zeitplaene; ?seite=mobil zeigt eine schlanke Ansicht. Bausteine: admin-chrome.tsx (Rahmen), admin-ui.tsx, admin-stand.tsx (Standzeile), admin-store.ts (Teile laden, useRegionCatchUp), admin-catch-up.tsx, admin-texts.ts, admin-colors.ts, admin-totp.tsx, admin-signout.tsx.

Reiter	Komponente	Zweck
uebersicht	admin-overview.tsx	Übersicht
atlas	admin-atlas.tsx	Lückenatlas (live, alle 5 min)
abruf	admin-loader.tsx → admin-dashboard.tsx, admin-processing.tsx, admin-ai-local.tsx	Abruf & Verarbeitung (Abrufauftrag, Titelregel, KI-Arbeitsliste)
pdf	admin-pdf.tsx, admin-pdf-parts.tsx	PDF-Import (Stand, Aufträge, Server, Läufe)
regeltexte	admin-rule-texts.tsx	Themen & Texte (Ergebnis je Schritt, Qualität der Regeln, Freigaben)
qualitaet	admin-quality-check.tsx	Qualität & Betrieb (Bauaufträge)
zeitplaene	admin-schedules.tsx	Jobs & Zeitpläne
debug	admin-debug.tsx, admin-pipeline-parts.tsx	Pipeline-Debug (vorübergehend)
altbestand	admin-altbestand.tsx	Altbestand
speicher	admin-storage.tsx	Speicher & Sicherungen
hochrechnung	admin-forecast.tsx, admin-estimate.tsx	Hochrechnung für Deutschland
social	admin-social.tsx	Social-Media-Vorschläge
todo	admin-todo.tsx	To-do-Board (docs/todo/todos.json, auf dem Server admin-todos-v1)

Alle Admin-Seiten lesen nur gespeicherte Stände (system_state, kleine Tabellen), nie alle Berichte; GETs schreiben nie.

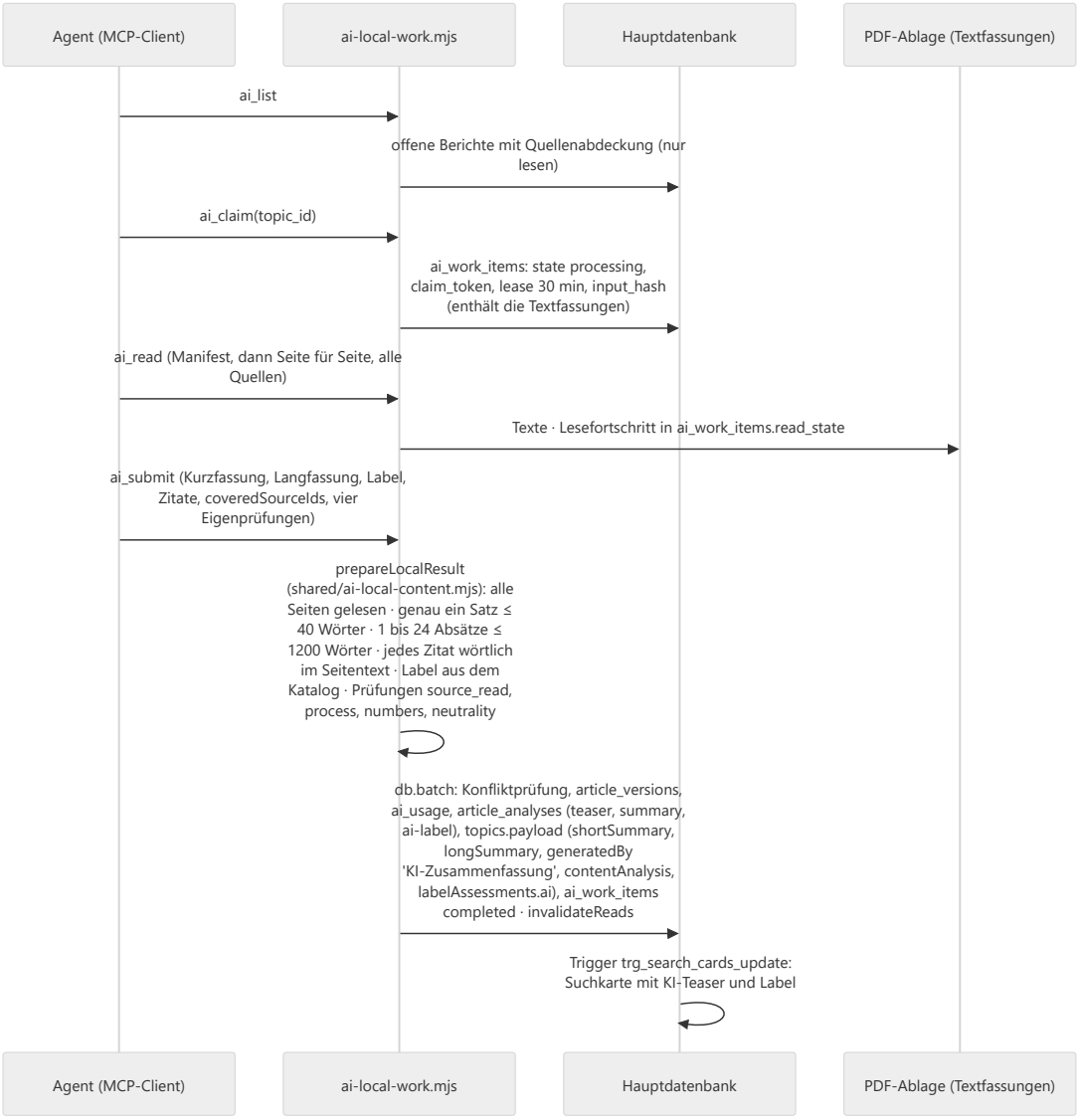
4. KI-Weg (ai-local-content-v2)

Regeln und KI sind zwei getrennte Ergebnisse; keines überschreibt das andere. Die KI startet nie von selbst: ein Agent (Modell mit Werkzeugzugriff) arbeitet über den internen MCP-Dienst und liest nur lokal gespeicherte Textfassungen. Die Fachlogik

steckt in `createAiLocalWork` (`server/integrations/ai-local-work.mjs`, Operationen `list`, `claim`, `read`, `renew`, `release`, `submit`, `fail`, Reservierung 30 Minuten); Quellen liefert `server/node/ai-local-sources.mjs` aus der PDF-Ablage sowie `documentText` / `sourceText`.

Drei Zugänge auf dieselbe Logik:

Zugang	Start	Hinweis
MCP intern (STDIO, kein Port)	`npm run mcp:internal` → `server/mcp/stdio.mjs` → `openMcpRuntime('internal')`; Werkzeuge `ai_list`, `ai_claim`, `ai_read`, `ai_renew`, `ai_release`, `ai_submit`, `ai_fail` (`server/mcp/server.mjs`)	Konfiguration mit `databaseFile`, `sourceDirectory`, `identity`, `requestsPerMinute` (`RM_MCP_CONFIG`, Vorlagen `config/mcp/`); läuft aus dem Quelltext, wird nicht gebaut; keine Unit auf dem Server
Kommandozeile	`scripts/ai-local.mjs list|claim|read|renew|release|submit|fail`	Datenbank aus `DATABASE_FILE`, sonst die Miniflare-Datei
HTTP	`GET|POST /api/admin/ai-local` (`requireAdmin`, `requireSameOrigin`, Body ≤ 3 MB), Reiter „Abruf & Verarbeitung“	braucht `env.AI_LOCAL_SOURCES` (nur Node); im Dev fängt `build/ai-local-dev-plugin.mjs` die Route ab (nur Loopback)



Kommt neuer Text dazu (OCR, Nacharbeit, Altbestand), ändert sich der Eingabehash: der Bericht wird `stale` und erscheint wieder in der Liste. Ein erneut gesendetes gleiches Ergebnis ist idempotent. `ai_fail` schreibt `article_analyses` `ai-local-attempt`, die alten Inhalte bleiben. Der Job `rule-texts` zählt den Stand (`ai-counts`); Pipeline-Debug und Altbestand zeigen ihn.

Weitere Wege: `ai-jobs.mjs` und `scripts/ai-job.mjs` (Altweg per Datei, nur für vorbereitete Altaufträge), `prepared-analysis.mjs` (festes Bündel Billerbeck, `codex-direct-v1`), Modus `summaries` von `runSync` mit OpenAI (`ai-summary.mjs`, nur von Hand, nur mit `OPENAI_API_KEY`; siehe Abruf, Befund 6). `manual-analysis.mjs` ist keine KI, sondern die Titelregel in Paketen. Verbrauch: `ai_usage`, Auswertung `scripts/ai-usage.mjs` mit Preisen aus `scripts/ai-prices.json`.

Kundendienst (`npm run mcp:customer`, `mcp-customer-data.mjs`): reine Recherche mit `reports_regions`, `reports_search`, `reports_get`, `reports_aggregate`; Datenbank schreibgeschützt (`PRAGMA query_only=ON`), feste Liste `allowedRegionIds` aus der

betreibereigenen Startkonfiguration, höchstens 366 Tage und 100 Ergebnisse je Abfrage, keine Rohtexte, keine Pfade. Kundenrechte stammen nie aus der Datenbank; keine interne Verbindung an Kunden geben.

5. Tests, Lint, Typen

- 175 Dateien `tests/*.test.mjs`, Start `node --test tests/*.test.mjs` (kein npm-Skript; gelegentlich zufällig rot: „marks of read meetings", „every import leaves a record"). Hilfen in `tests/helpers/`: `d1.mjs` (In-Memory-`node:sqlite` mit allen Migrationen), `accounts.mjs` (Kontext, Anfragen wie aus dem Browser, Dev-Postfach), `analysed.mjs`. Fixtures in `tests/fixtures/` (nie ganz lesen).
- Gleichheitstests sichern doppelte Logik: Formalien-Muster JS gegen SQL (`search-formal`), Datum „Stand heute" (`search-card-dates`), Label-Rangfolge JS gegen View (`effective-labels`), Nachführen der Wortliste gegen vollen Aufbau (`search-cards-gone`), FTS gegen `instr` (`search-fts`), Signale mit und ohne `x_signals`, warmgehaltene Abfragen (`x-warm`), Stufen gegen `docs/produkt/stufen.md` (`accounts-rules`), Admin-GETs ohne Schreiben (`admin-readonly`).
- Lint `npm run lint` (`eslint-config-next`), Typen `node node_modules/typescript/bin/tsc --noEmit --incremental false`.

6. Befunde beim Schreiben dieser Dokumentation (17.10.2026)

1. **Web-Push ist nicht verdrahtet:** Service Worker, Routen und `push.ts` existieren, im Frontend fehlen Registrierung und Abo-Aufruf.
2. `tests/accounts-tiers.test.mjs` (genannt in `shared/plans.mjs`) und `tests/accounts-session.test.mjs` (genannt in `server/accounts/dev.mjs`) gibt es nicht; den Vergleich mit `docs/produkt/stufen.md` macht `tests/accounts-rules.test.mjs`.
3. `FRONTEND.md` nennt `/analytics/methodik`; die Route steht weder in `ANALYTICS_PATHS` noch als Stub, die Methodik erscheint als `MethodikUnten` unter jeder Auswertung.
4. `package.json` verlangt Node ≥ 22.13, `serve.mjs` bricht unter 22.16 ab.
5. Die Startseite, Infoseiten und Admin greifen an vielen Stellen mit rohem `fetch` zu; ein gemeinsamer Client existiert nur für Konto (`lib/account.ts`) und Suche (`state/search.tsx`).